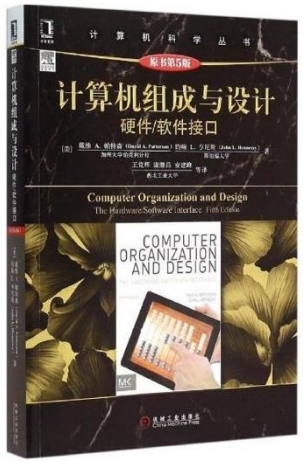
计算机组成原理

Computer Organization and Design

——The Hardware and Software Interface 5th



首批国家级一流本科课程
天津市创新创业示范课程

R0-00 现代计算机的设计

R1-01 什么是后PC时代?

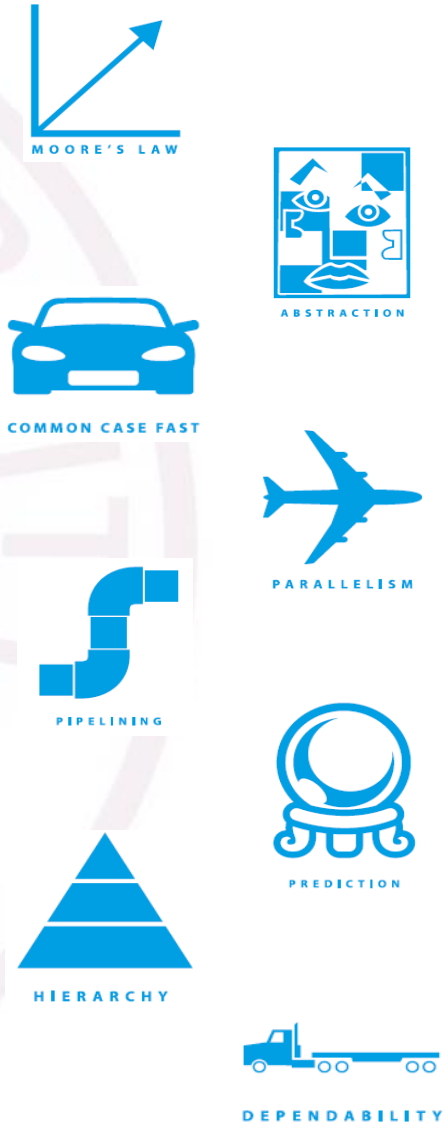
The Computer Revolution：

- Progress in computer technology
  - Underpinned by Moore's Law
- Makes novel applications feasible
  - Cell phones, WWW, Search Engines
- Computers are pervasive
  - Personal Mobile Device, Cloud computing, SaaS

R1-01

计算机设计的八个伟大思想/原则?

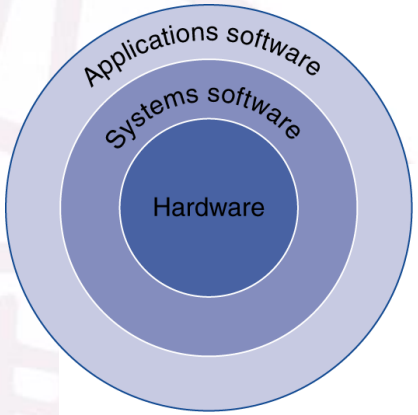
- Design for ***Moore's Law***
- Use ***abstraction*** to simplify design
- Make the ***common case fast***
- Performance *via* ***parallelism***
- Performance *via* ***pipelining***
- Performance *via* ***prediction***
- ***Hierarchy*** of memories
- ***Dependability*** *via* redundancy

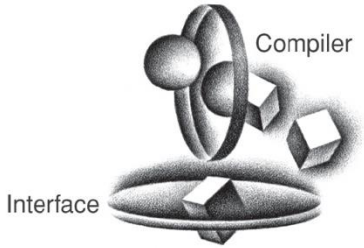


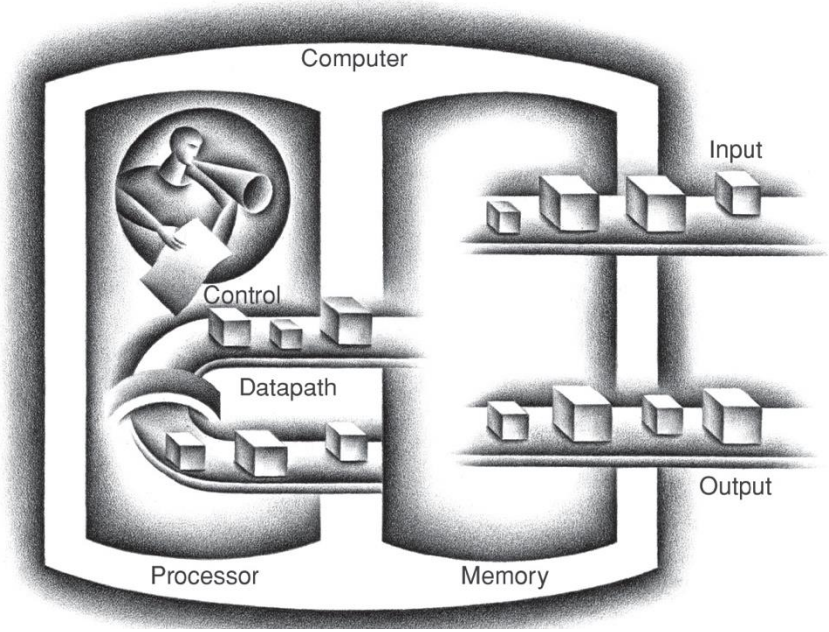
R1-02 什么是计算机的性能?

Response Time and Throughput:

- Response time
  - How long it takes to do a task
- Throughput
  - Total work done per unit time
    - e.g., tasks/transactions/… per hour
- They are affected by
  - Algorithm, Programming Language,
  - Compiler, ICA, Memory System
- We'll focus on response time for now..







单选题 1分

设置

CPU、内存、键盘、鼠标、显卡、NPU 属于传统计算机中的什么部分呢?

A DataPath

B Input

C Output

D Memory

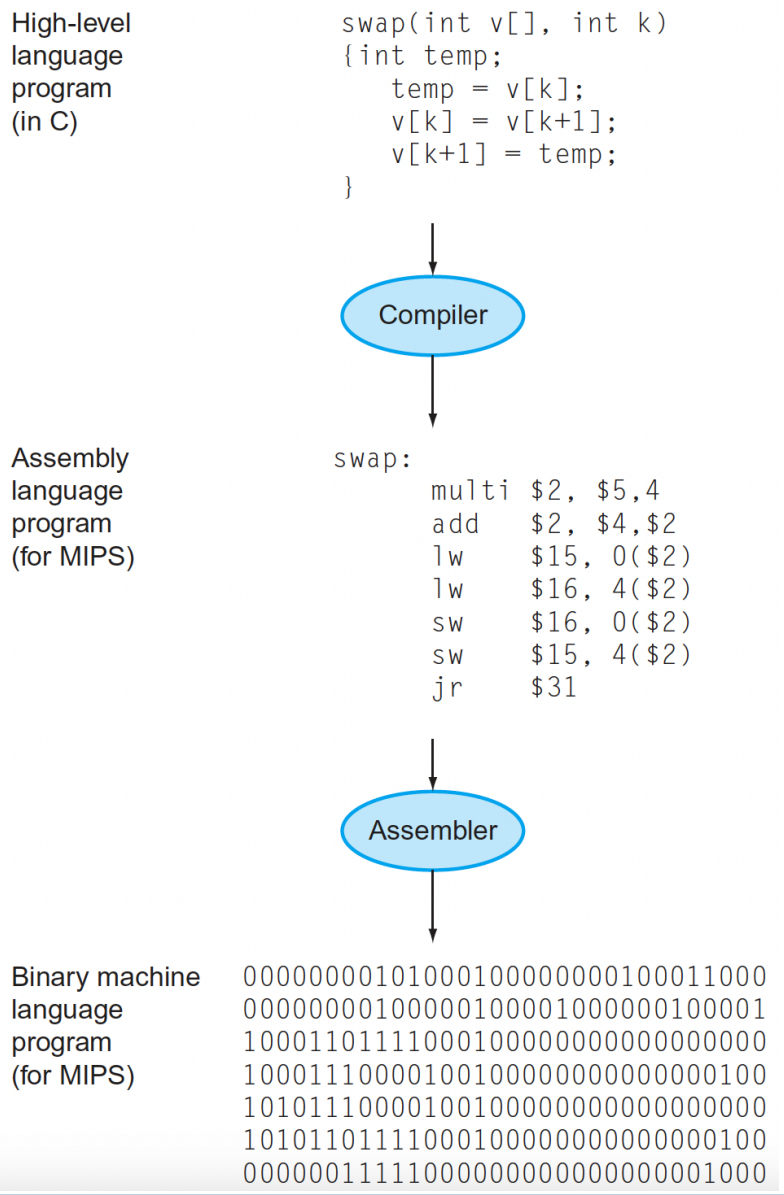
提交

R1-02

什么是计算机的性能?

Response Time and Throughput:

- Response time
  - How long it takes to do a task
- Throughput
  - Total work done per unit time
    - e.g., tasks/transactions/… per hour
- They are affected by
  - Algorithm, Programming Language,
  - Compiler, ICA, Memory System
- We'll focus on response time for now…



R1-02 计算机的性能如何量化?

量化评价的目的是比较

- Define Performance = 1/Execution Time
- "X is *n* time faster than Y"

$$\text{Performance}_X / \text{Performance}_Y = \text{Execution time}_Y / \text{Execution time}_X = n$$

- Example: time taken to run a program
  - 10s on A, 15s on B
  - Execution Time_B / Execution Time_A = 15s / 10s = 1.5
  - So A is 1.5 times faster than B

R1-03

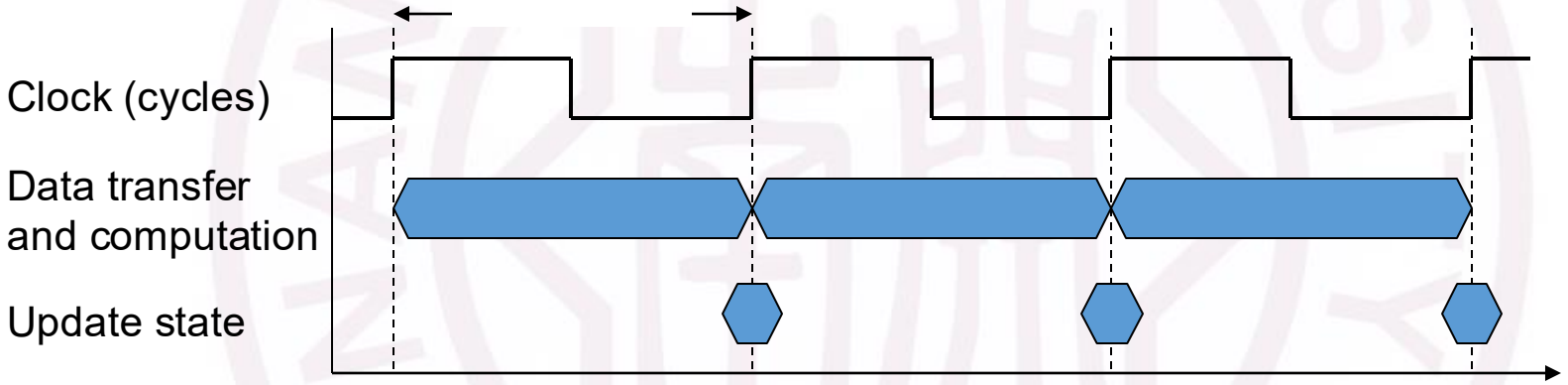
任务的执行时间如何界定？从哪开始？到哪结束？为什么不行？

- Elapsed time
  - Total response time, including all aspects
    - Processing, I/O, OS overhead, idle time
  - Determines system performance
- CPU time
  - Time spent processing a given job
    - Discounts I/O time, other jobs' shares
  - Comprises *user CPU time* and *system CPU time*
  - Different programs are affected differently by CPU and system performance

R1-03

任务的执行时间如何界定？时钟周期？时钟频率？

- Operation of digital hardware governed by a constant-rate clock



- Clock period: duration of a clock cycle
  - e.g., 250ps = 0.25ns = 250×10^{-12}s
- Clock frequency (rate): cycles per second
  - e.g., 4.0GHz = 4000MHz = 4.0×10^{6}kHZ = 4.0×10^{9}Hz

R1-03 任务的执行时间如何界定?

- Clock period: duration of a clock cycle
  - e.g., 250ps = 0.25ns = 250×10^{-12}s
- Clock frequency (rate): cycles per second
  - e.g., 4.0GHz = 4000MHz = 4.0×10^{9}Hz

$$\text{CPU Time} = \text{CPU Clock Cycles} \times \text{Clock Cycle Time}$$

$$= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}$$

R1-04 指令集和CPU时间关系?

$$\text{CPU Time} = \text{CPU Clock Cycles} \times \text{Clock Cycle Time}$$

$$= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}$$

$$\text{Clock Cycles} = \text{Instruction Count} \times \text{Cycles per Instruction}$$

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

$$= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

R1-04

如何获得CPI?

- If different instruction classes (n) take different numbers of cycles

$$\text{Clock Cycles} = \sum_{i=1}^{n} (\text{CPI}_i \times \text{Instructio n Count}_i)$$

- Weighted average CPI

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instructio n Count}} = \sum_{i=1}^{n} \left(\text{CPI}_i \times \underbrace{\frac{\text{Instructio n Count}_i}{\text{Instructio n Count}}}_{\text{Relative frequency}} \right)$$

R1-04 功率和性能的关系？功率墙问题

For CMOS, the primary source of energy consumption is so-called dynamic energy—that is, energy that is consumed when transistors switch states from 0 to 1 and vice versa.

- Dynamic energy:

$$Energy \propto Capacitive\ load \times Voltage^2$$

$$Energy \propto 1/2 \times Capacitive\ load \times Voltage^2$$

- Power:

$$Power \propto 1/2 \times Capacitive\ load \times Voltage^2 \times Frequency\ switched$$

R1-05 面对功率墙我们该怎么办？

- Q1-19 新形式——单处理器到多处理器
- Q1-20 新材料——光进铜退、超导计算机、碳基芯片
- Q1-21 新架构——类脑计算
- Q1-22 新机理——量子计算机

R2-01 如何开始设计一台计算机?

R2-02 贯穿式设计——异构衔接之处?

- Instruction Set
  - CISC（Complex Instruction Set Computer）复杂指令系统计算机
  - RISC（Reduced Instruction Set Computer）精简指令集计算机

- X86——目前PC和服务器市场份额最大
- ARM——以智能手机为代表的嵌入式市场（ARMv8更加接近MIPS）
- MIPS——以网络设备为代表的嵌入式市场
- PowerPC——IBM维护的服务器/游戏主机
- RISC-V——冉冉升起的新兴开源指令集

R2-03 如何开始设计指令?

R2-04 指令操作的本质是什么?

将数从某处取出放到某处

数据 ↓ 位置

- Basic principles of hardware design:
  - Simplicity favors regularity
  - Smaller is faster
  - Good design demands good compromises
  - Make the common case fast

R2-05 MIPS指令集是如何设计的？——算术指令

- ***Arithmetic instructions* use *register operands***
- MIPS has a 32 × 32-bit register file
  - Use for frequently accessed data
  - 32-bit data called a "word"
  - $t0, $t1, ..., $t9 for temporary values
  - $s0, $s1, ..., $s7 for saved variables
- MIPS assembly language
  - Operation rd, rs, rt
  - Example: add c, a, b

Registers are faster to access than memory

rd: Destination Register number
rs: first Source Register number
rt: Target Register (Second Source Register) number

R2-05 MIPS指令集是如何设计的？——数据传送

- ***Data transfer instructions***
  - Load values from memory into registers
  - Store result from register to memory
  - Memory is **byte** addressed
  - Example: `A[12] = h + A[8];`

```
lw  $t0, 32($s3)    # load word, get A[8]
add $t0, $s2, $t0
sw  $t0, 48($s3)    # store word
```

R2-05 MIPS指令集是如何设计的？——立即数操作

Make the common case fast

- ***Immediate Operands***
  - Constant data specified in an instruction

    addi $s3, $s3, 4
  - No subtract immediate instruction

    addi $s2, $s1, -1
  - MIPS register 0 ($zero) is the constant 0

    `add $zero, $t2, $s1 # but` cannot be overwritten
  - $zero is useful for common operations

    `add $t2, $s1, $zero`

R2-05

MIPS指令集是如何设计的？——逻辑操作

- ***Logical Operations in bit***

Operation	C	Java	MIPS
Shift left	<<	<<	sll
Shift right	>>	>>	srl
Bitwise AND	&	&	and, andi
Bitwise OR	|	|	or, ori
Bitwise NOT	~	~	nor

- `sll $t2, $s0, 4`
- `srl $t2, $s0, 4`
- `and $t0, $t1, $t2`
- `or $t0, $t1, $t2`
- `nor $t0, $t1, $t2`

$t1	0000 0000 0000 0000 0011 1100 0000 0000
$t0	1111 1111 1111 1111 1100 0011 1111 1111

R2-06

MIPS指令集是如何设计的？——决策操作

- ***Conditional Operations***：Branch to a labeled instruction if a condition is true
  - beq rs, rt, L1 # if (rs == rt) branch to instruction labeled L1
  - bne rs, rt, L1 # if (rs != rt) branch to instruction labeled L1
  - slt rd, rs, rt # if (rs < rt) rd = 1; else rd = 0;
  - j L1 # unconditional jump to instruction labeled L1
  - jr $ra # For procedure *return*, switch ($ra: return address，PC)
  - jal ProcedureLabel # For procedure *call*
- for, while：

```
slt $t0, $s1, $s2     # if ($s1 < $s2)
bne $t0, $zero, L     #   branch to L
```

R2-07

正负数与有无符号数在二进制下是如何表示的?

不足32bit的数据装入32bit寄存器会发生什么?

- two's complement representation
  - +: itself, - : Invert and add 1 (except the leading bit)
  - leading 0s mean +, and leading 1s mean -
- Replicate the sign bit to the left
- unsigned values: extend with 0s

Examples: 8-bit to 16-bit

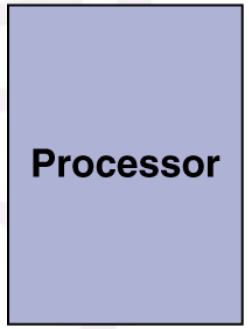
- +2: 0000 0010 => 0000 0000 0000 0010
- –2: 1111 1110 => 1111 1111 1111 1110

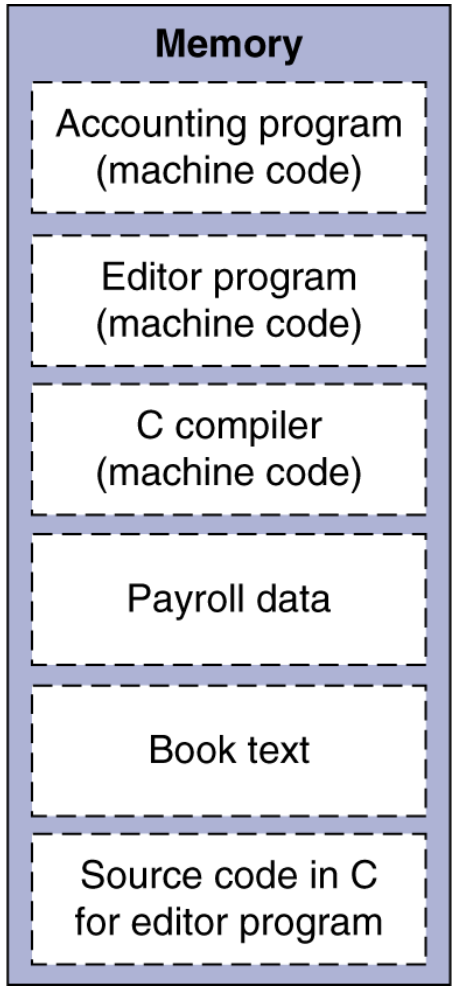
冯氏计算机

R2-08 MIPS指令在二进制下是如何表示的?

- Instructions are encoded in binary (32-bit, regularity), just like data
  - stored in memory
  - Called machine code
  - Stored-program concept
- Programs can operate on programs
  - e.g., compilers, linkers, …
- Binary compatibility allows compiled programs to work on different computers
  - Standardized ISAs

Simplicity favors regularity





R2-9 不同的MIPS指令在二进制下有什么区别?

Good design demands good compromises

R-format:

add $t0, $t1, $t2 or $t0, $t1, $t2 and $t0, $t1, $t2 slt $t0, $t1, $t2

sub $t0, $t1, $t2 nor $t0, $t1, $t2

I-format:

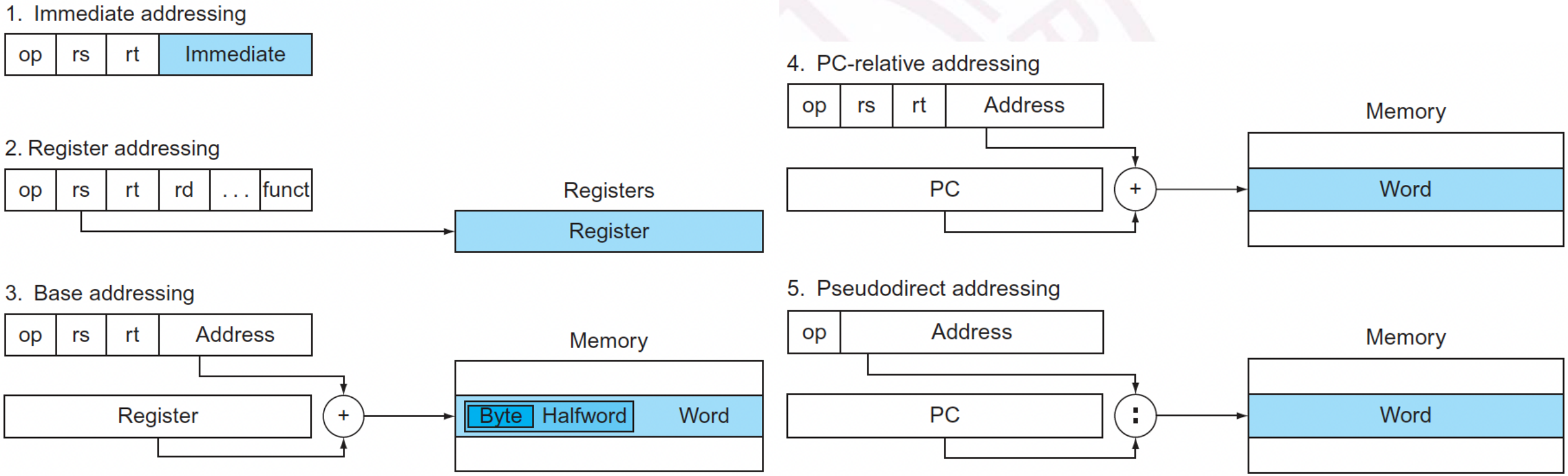
addi $s2, $s1, -1 ori $t0, $t1, 10 sll $t2, $s0, 4 lw $t0, 32($s3) beq rs, rt, L1

slti $s2, $s1, -1 andi $t0, $t1, 10 srl $t2, $s0 sw $t0, 48($s3) bne rs, rt, L1

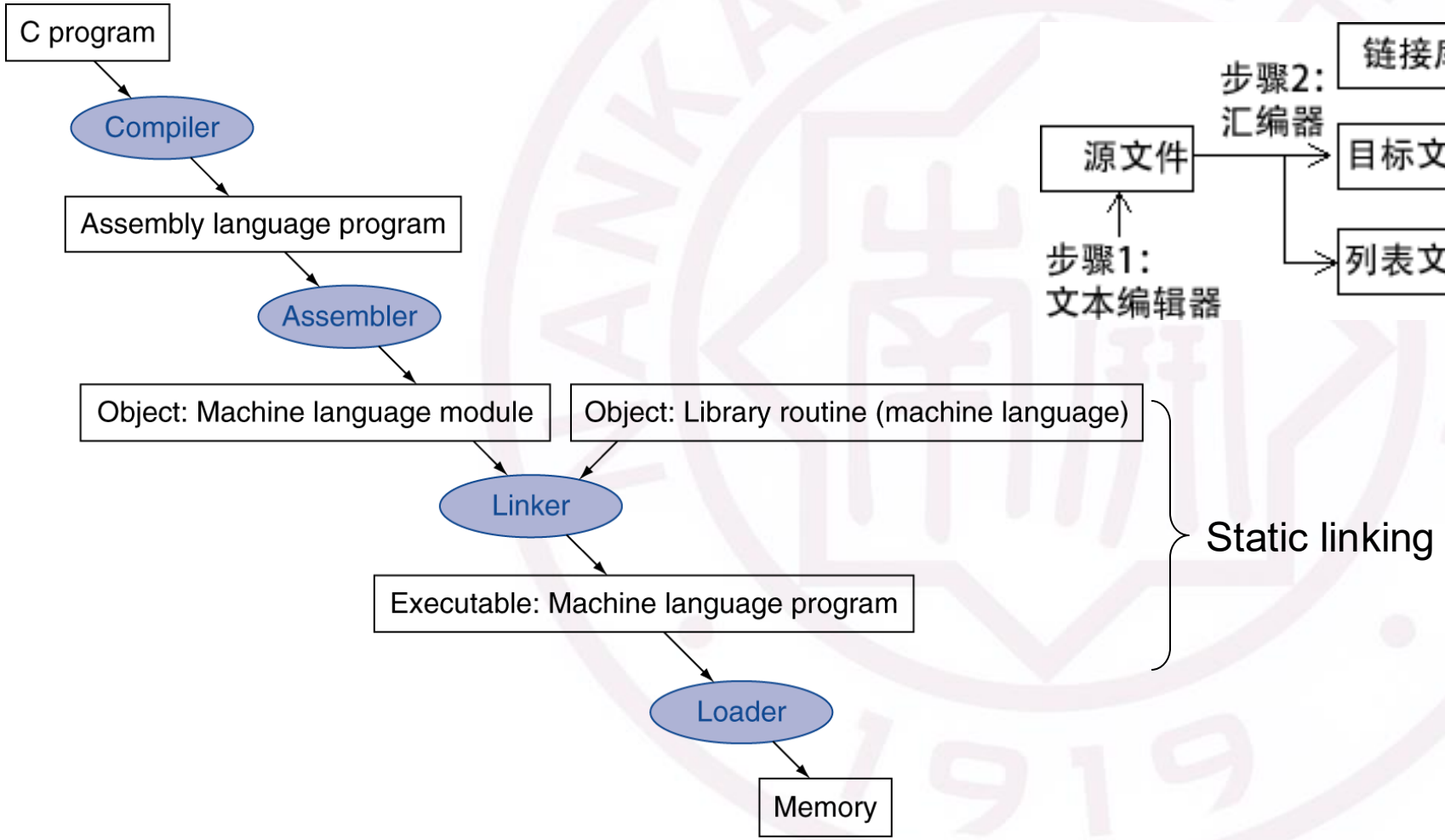
J-format:

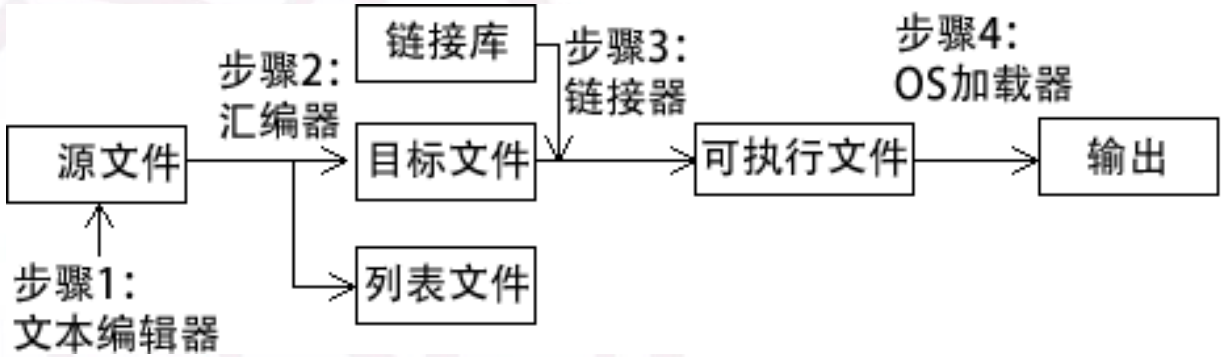
j 10000 jal 2500

R2-10 不同的MIPS指令是如何寻址的?



R2-11 存储的高级语言是如何被翻译并执行的?





对比：Dynamic linked library
程序执行过程中才被链接加载：

- 函数库更新
- 程序只用到部分库函数
- 运行比内存大的程序

R3-01

算术运算在计算机中是如何被执行的?

1. 存储部件的构建
2. 运算部件的构建
3. 如何用更少的位宽实现更大的数据表示范围?

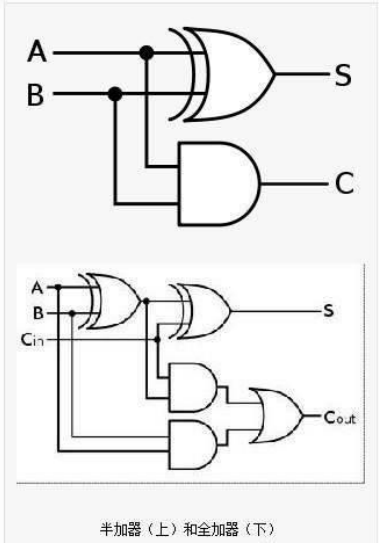
- Operations on integers
  - Addition and subtraction
  - Multiplication and division
  - Dealing with overflow
- Floating-point real numbers
  - Representation and operations

R3-02

整数加法器的设计?

- Example: 7 + 6

- Overflow if result out of range
  - Adding +ve and –ve operands, no overflow
  - Adding two +ve operands
    - Overflow if result sign is 1
  - Adding two –ve operands
    - Overflow if result sign is 0



半加器（上）和全加器（下）

- Add negation of second operand

7 – 6 = 7 + (–6)

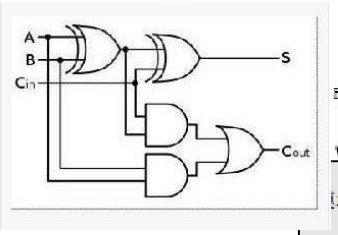
R3-03

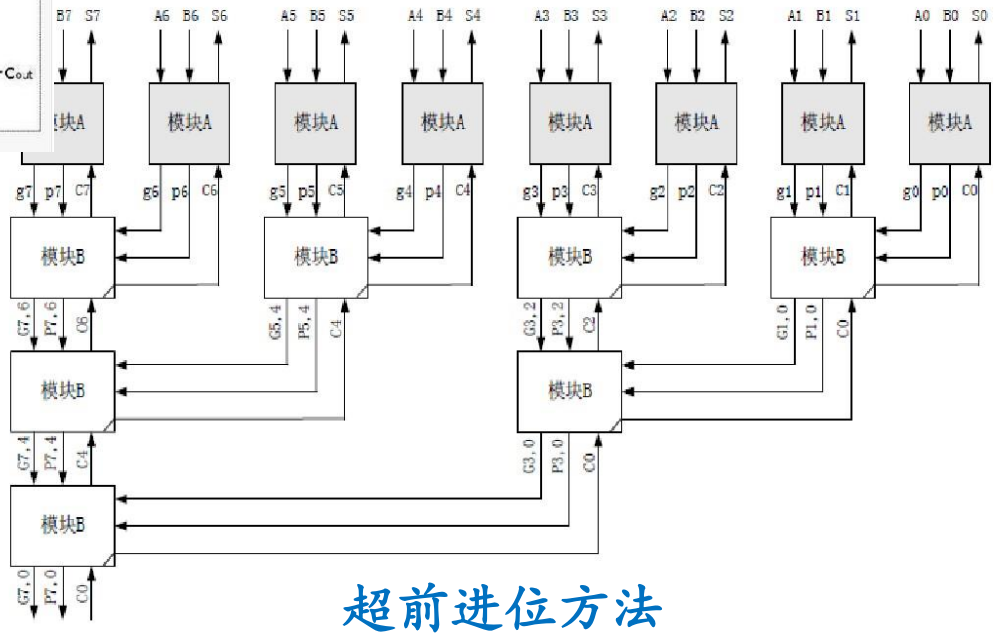
整数加法器的优化?

- 对加法器进位的逻辑表达式做进一步的推导:

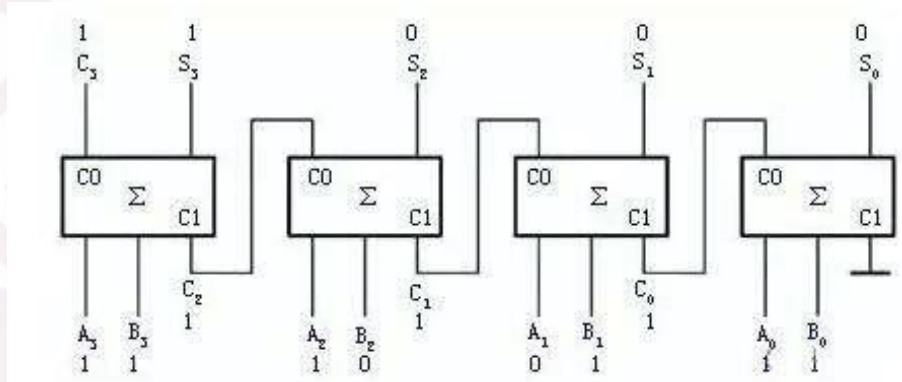
$$C_0 = 0$$

$$C_{i+1} = A_iB_i + A_iC_i + B_iC_i = A_iB_i + (A_i + B_i)C_i$$

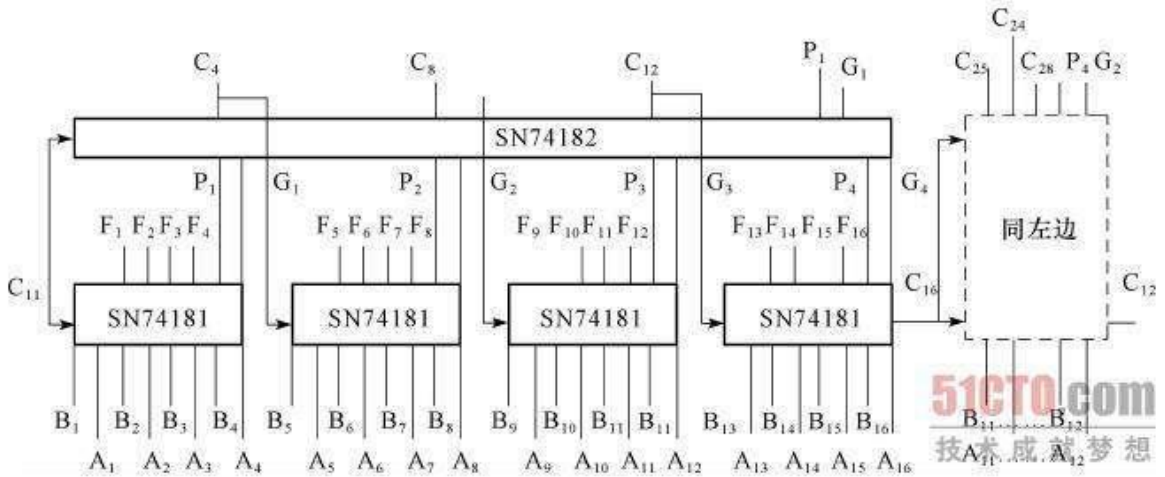




超前进位方法



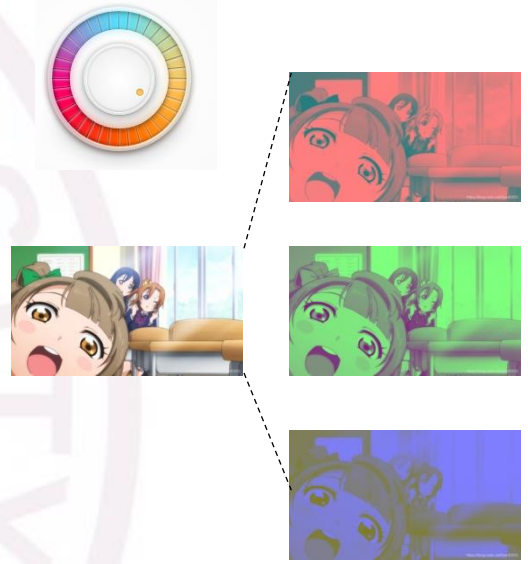
基础整数加法器

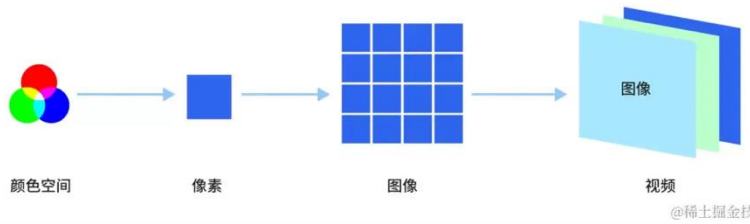


分组进位方法

R3-04 整数加法器的设计？—如何避免只用部分数据导致的浪费

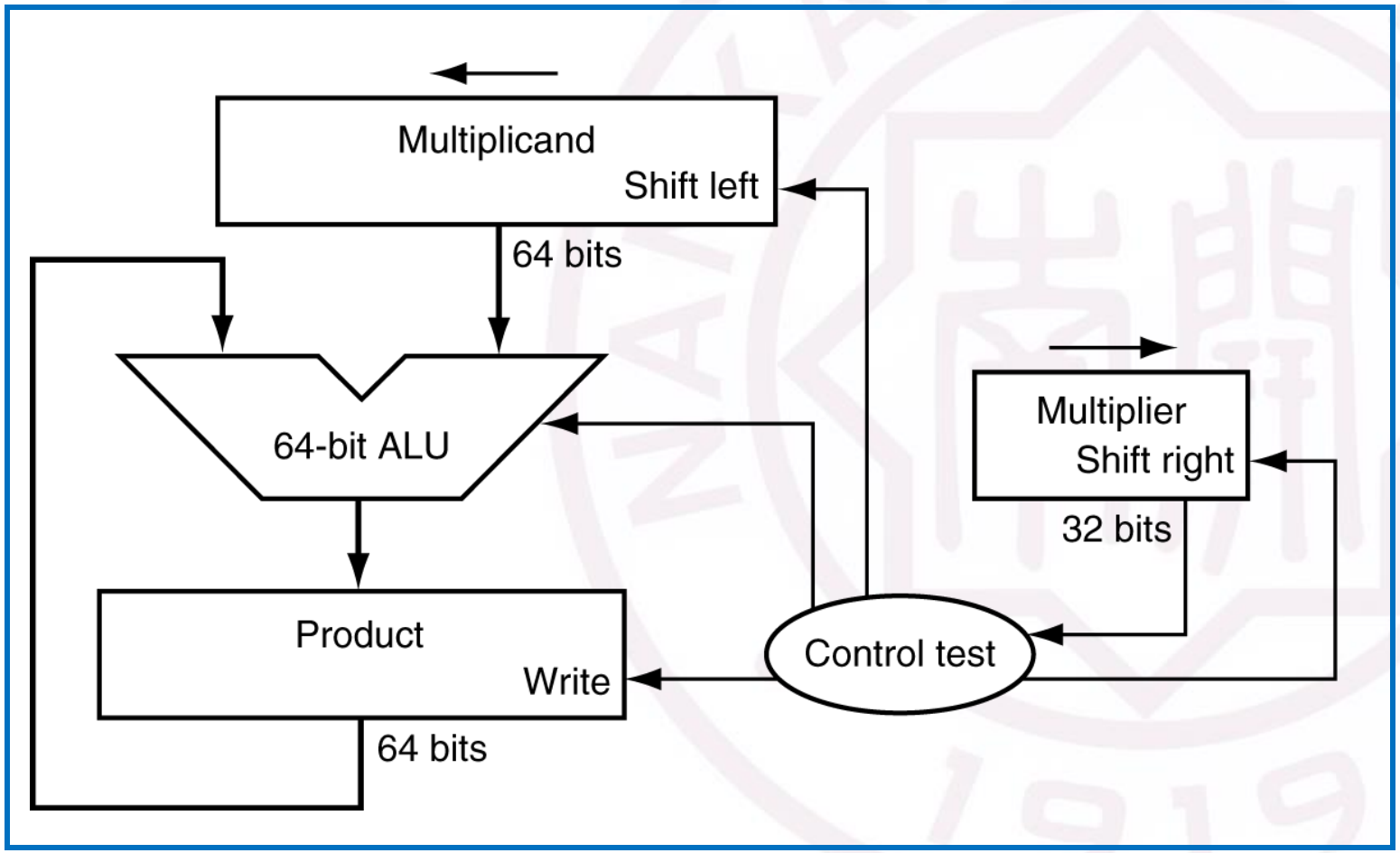
- Graphics and media processing operates on vectors of 8-bit and 16-bit data
  - Use 64-bit adder, with partitioned carry chain
    - Operate on 8×8-bit, 4×16-bit, or 2×32-bit vectors
  - SIMD (single-instruction, multiple-data)
- Saturating operations
  - On overflow, result is largest representable value
    - c.f. 2s-complement modulo arithmetic
  - E.g., clipping in audio, saturation in video





R3-05

整数乘法器的设计?



multiplicand

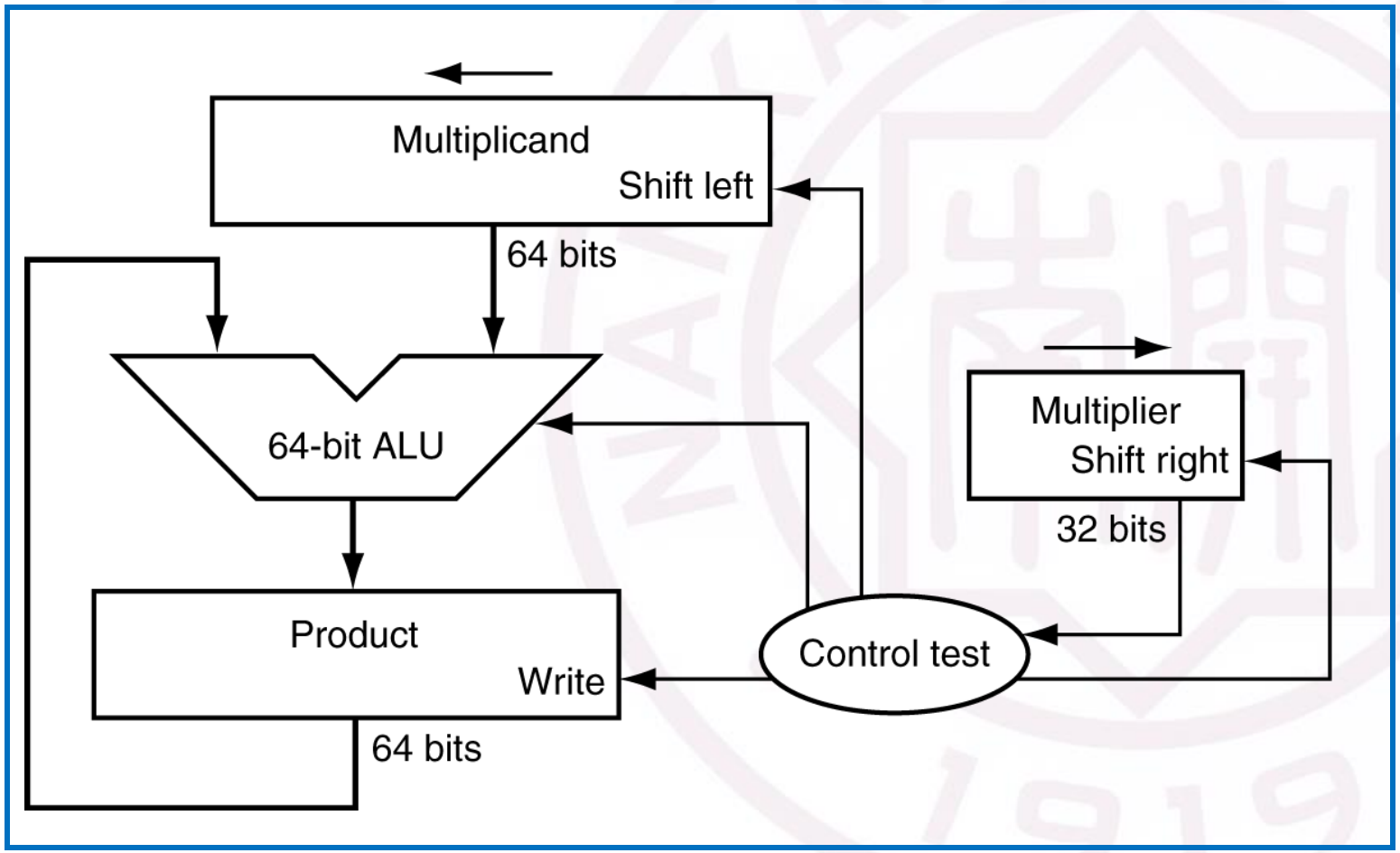
multiplier

```
  1000
× 1001
------
  1000
 0000
0000
1000
-------
1001000
```

product

Length of product is the sum of operand lengths

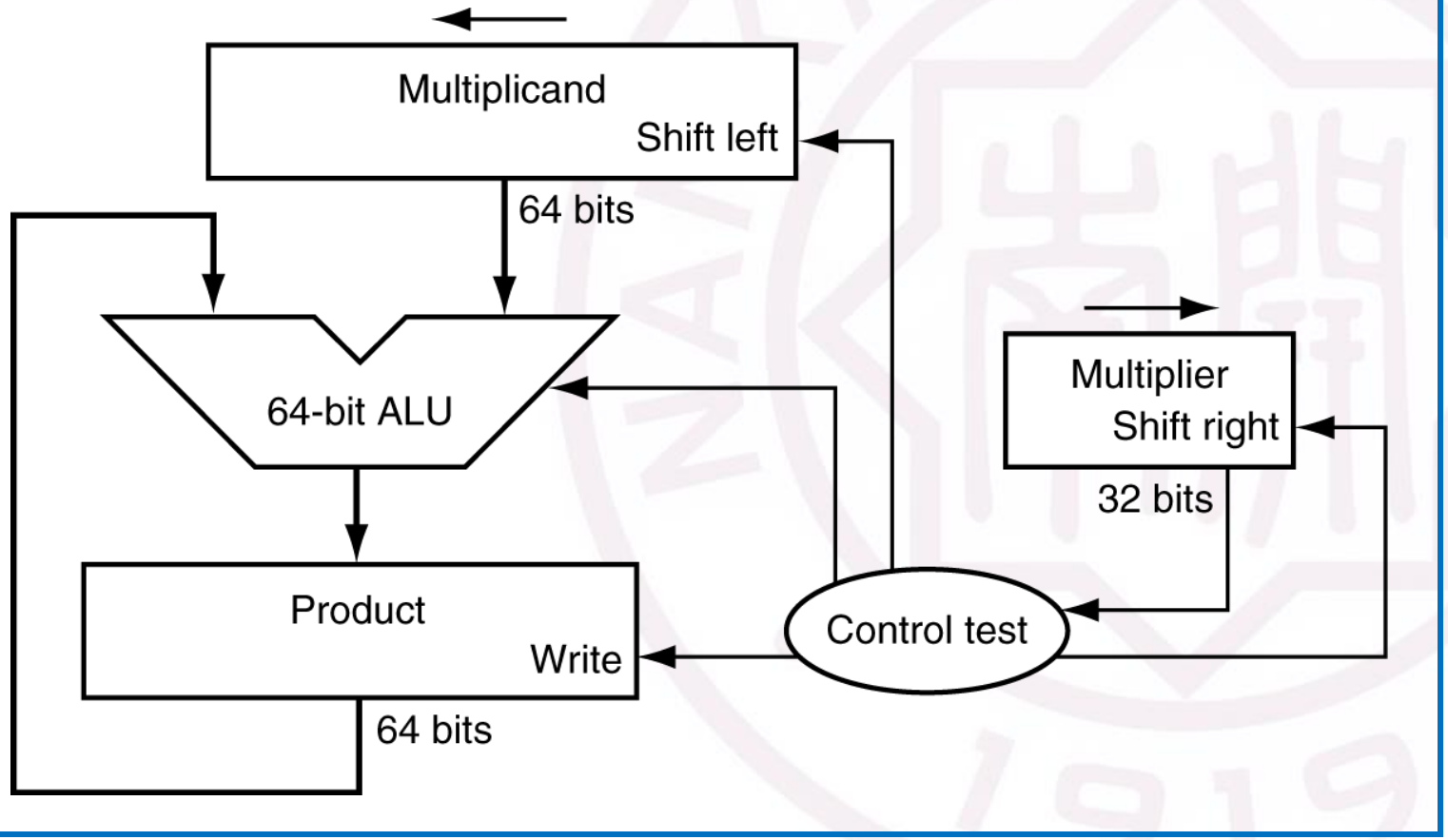
R3-05 整数乘法器的设计?

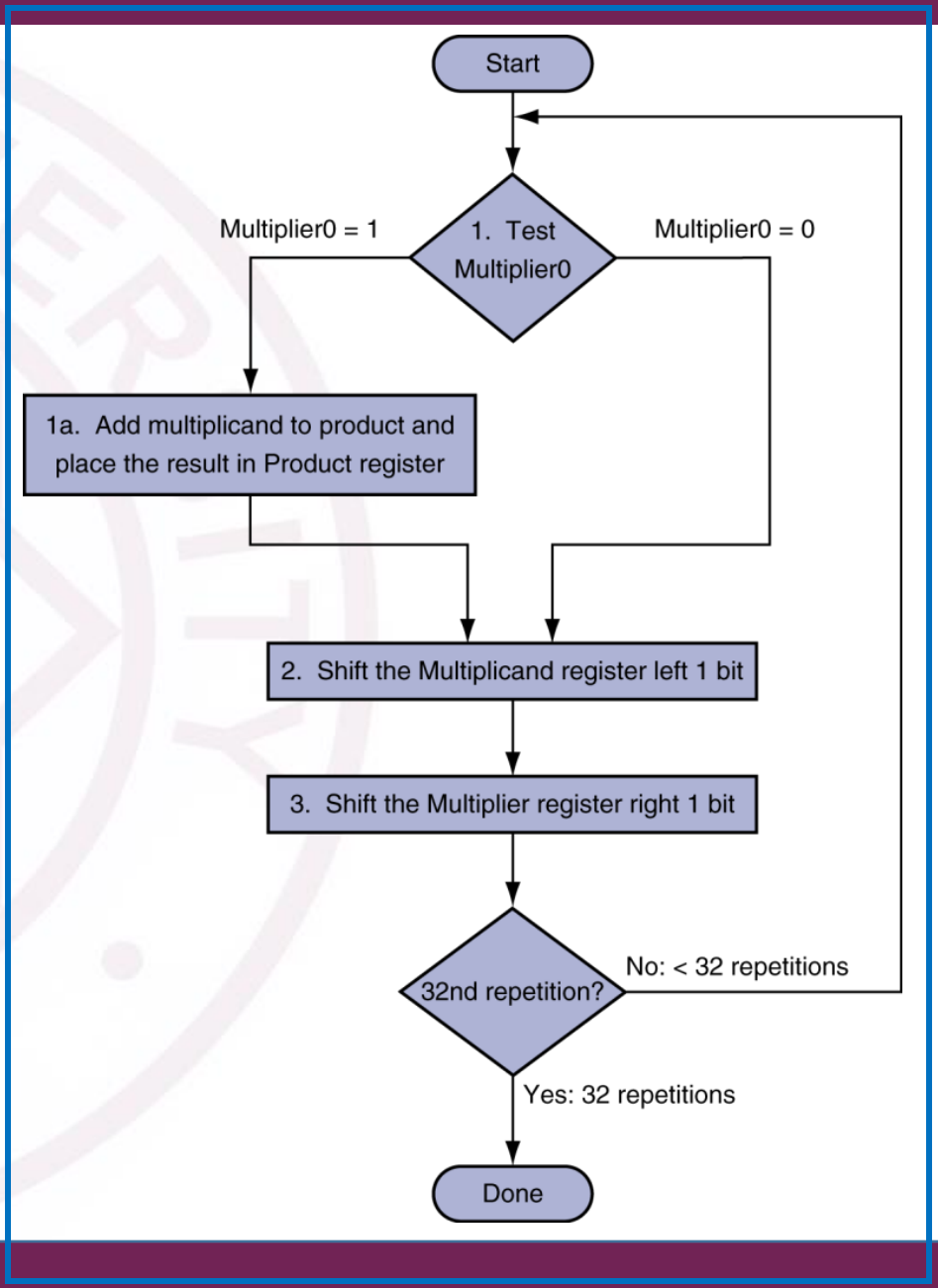


00001000
× 1001
00001000
00000000
00000000
01000000
01001000

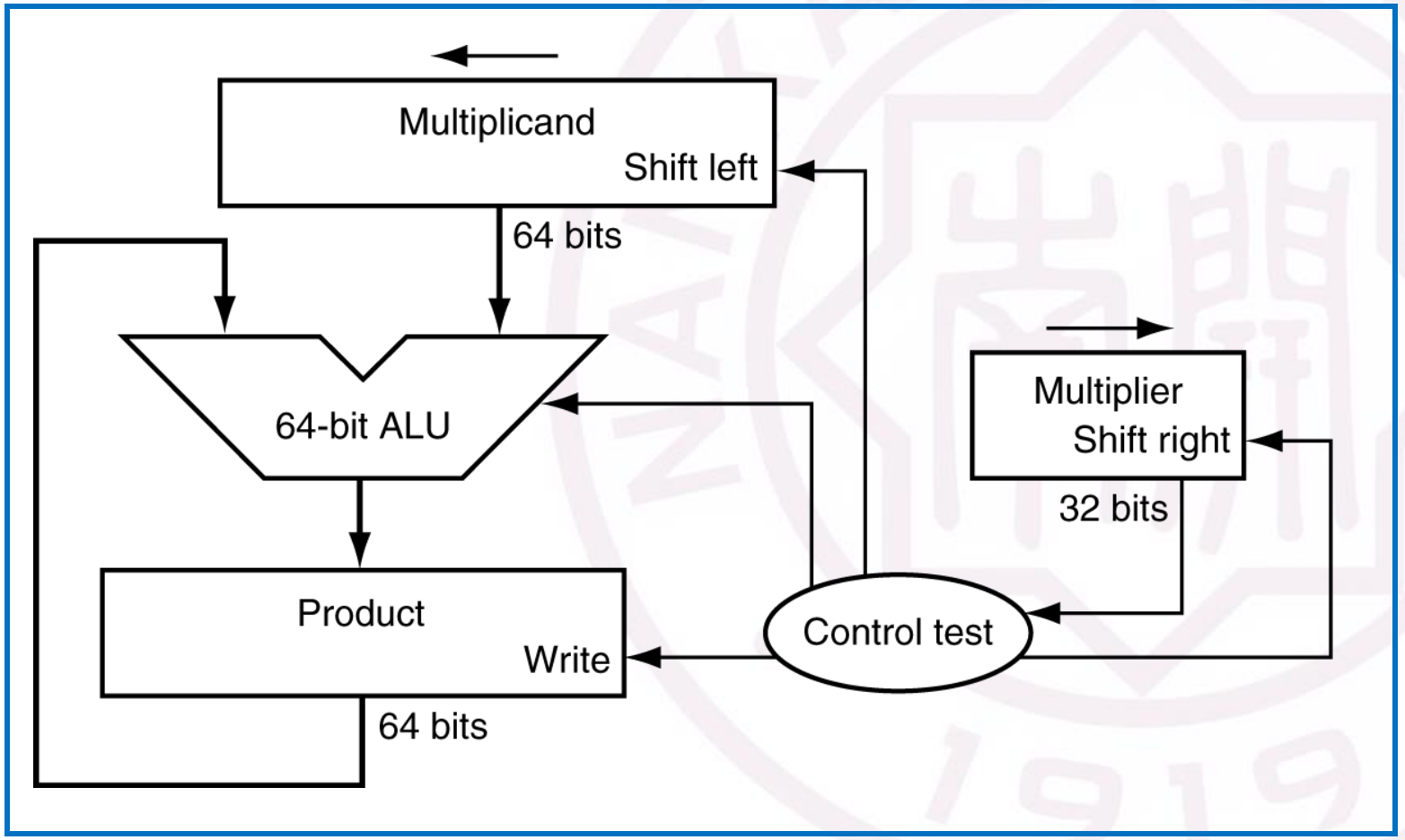
R3-05

整数乘法器的设计?





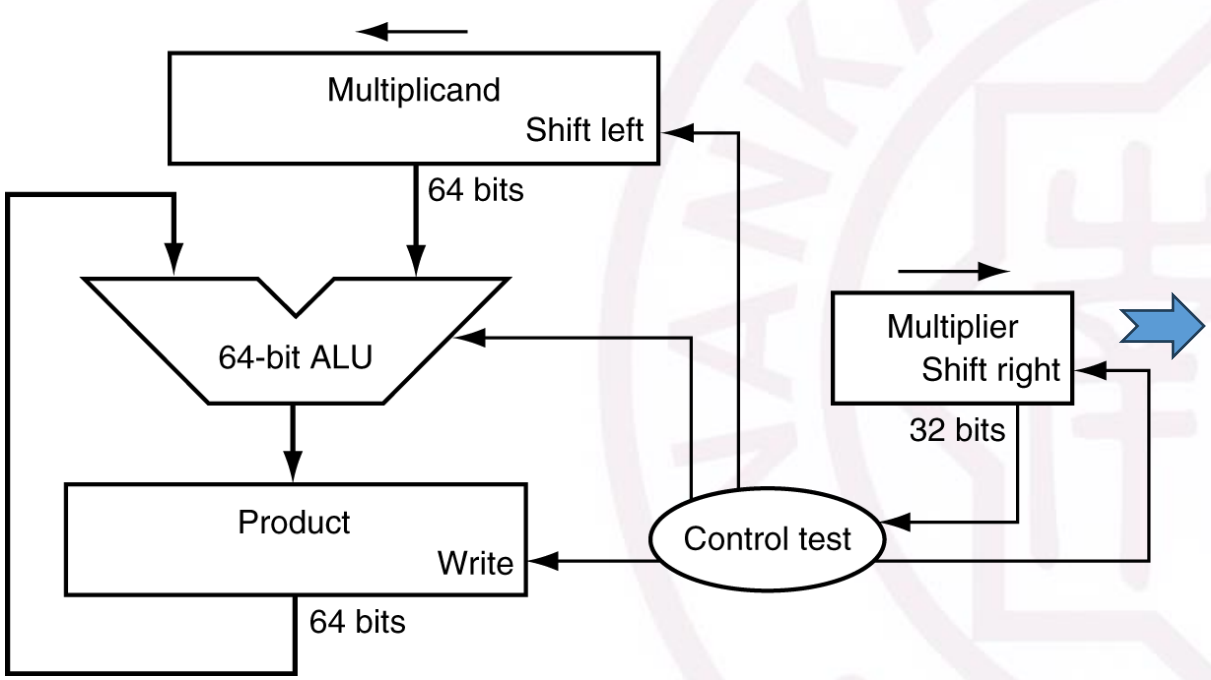
R3-06 整数乘法器的优化?

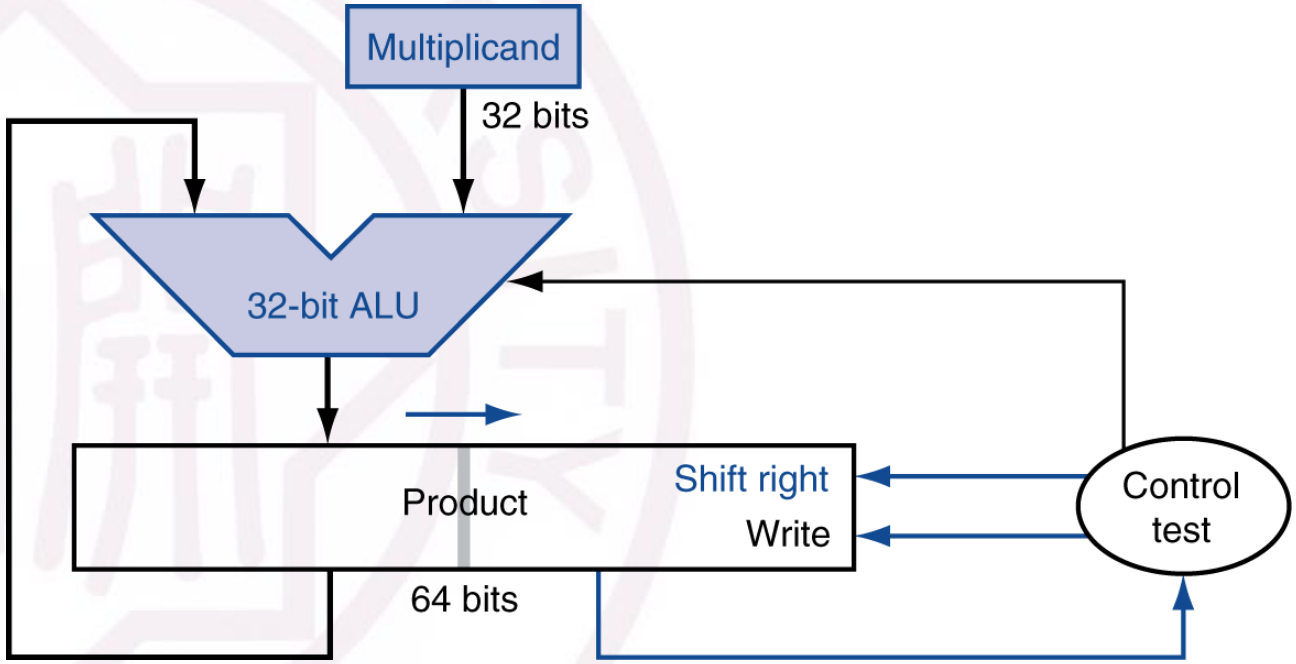


1. 空间角度的优化
   a. 压缩寄存器
   b. 复用寄存器
   c. 减少位宽
2. 时间角度的优化
   a. 左移位
   b. 多个乘法并行执行

R3-06

整数乘法器的优化？——空间角度

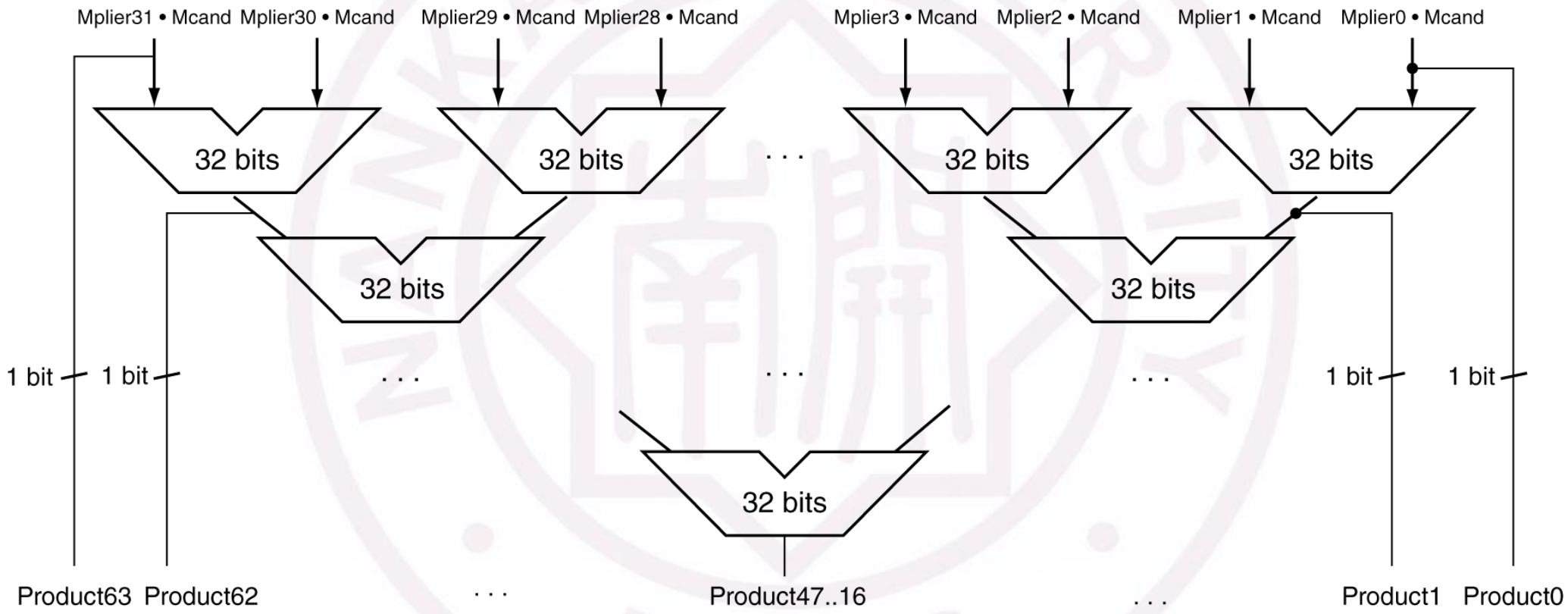




a. 压缩寄存器　　b. 复用寄存器　　c. 减少位宽

R3-06

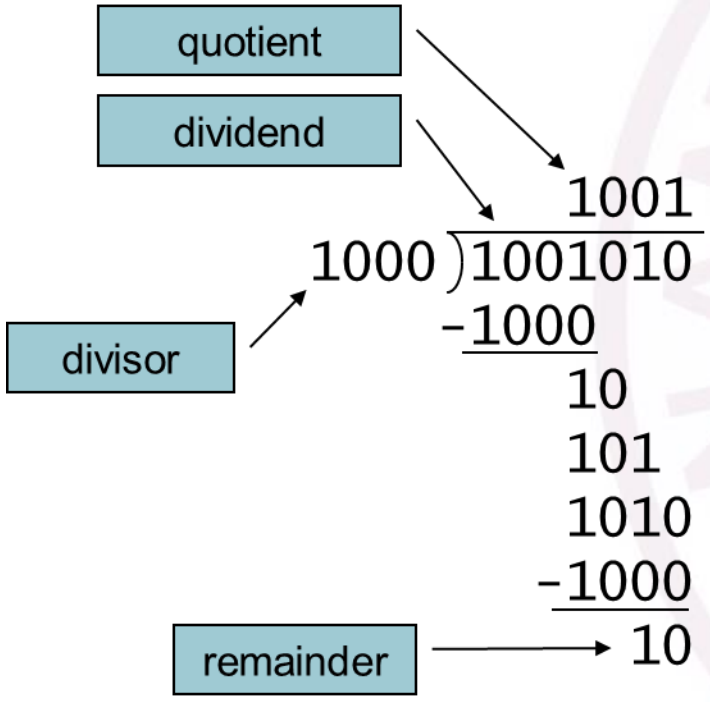
整数乘法器的优化？——时间角度



b. 多个乘法并行执行

R3-07

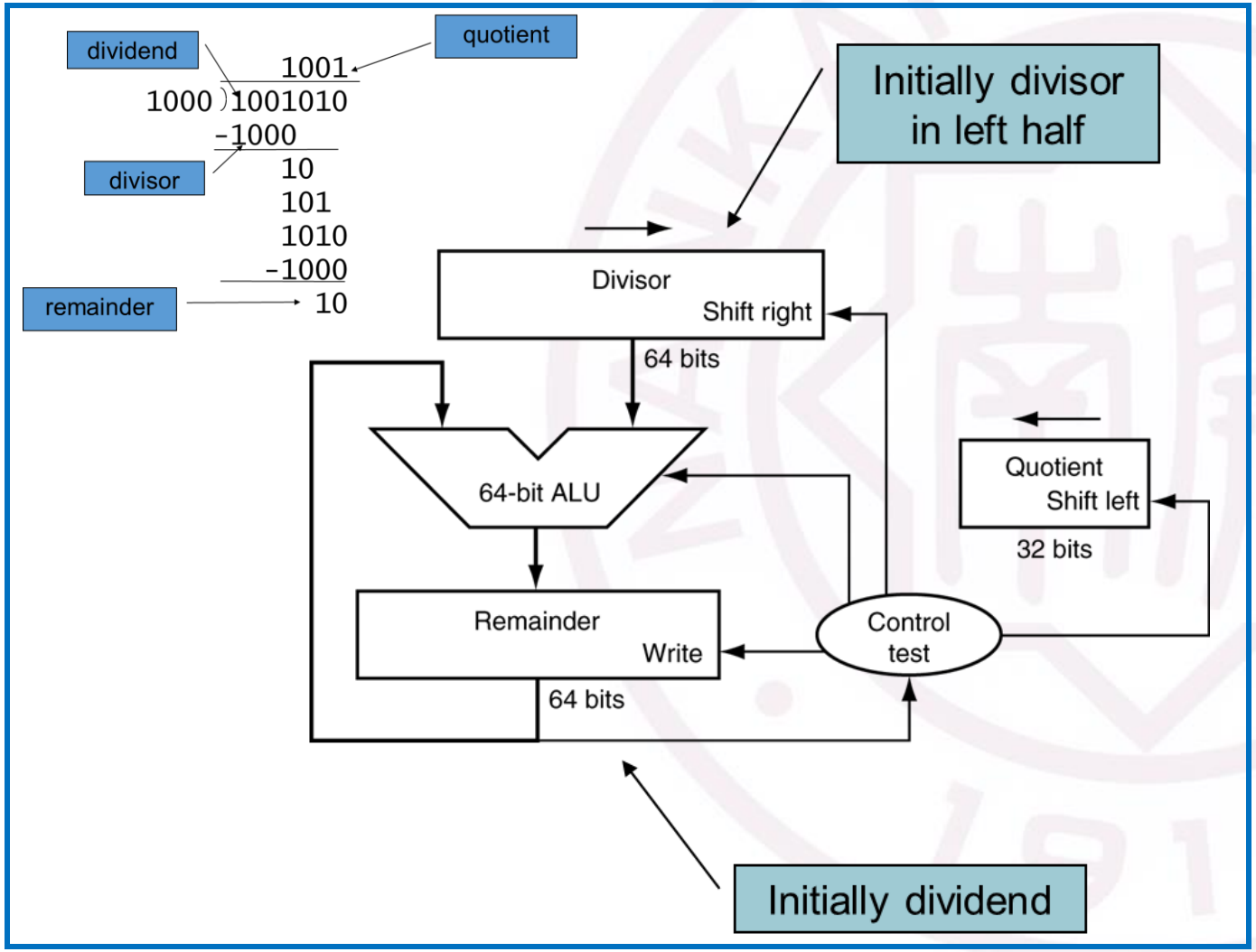
整数除法器的设计?

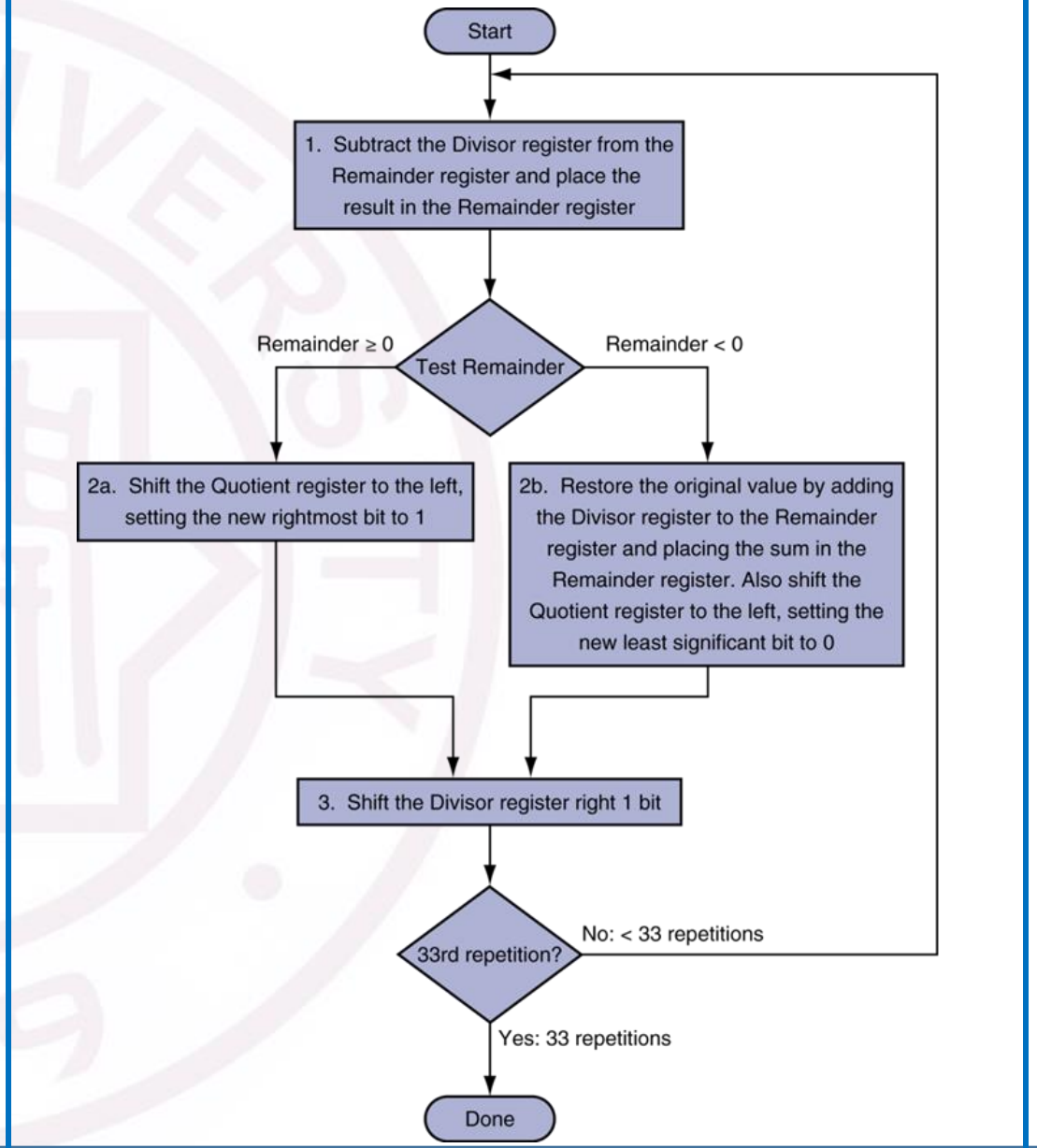


- Check for 0 divisor
- Long division approach
  - If divisor ≤ dividend bits
    1 bit in quotient, subtract
  - Otherwise
    0 bit in quotient, bring down next dividend bit
- Restoring division
  - subtract, and if remainder goes < 0, add divisor back
- Signed division
  - Divide using absolute values
  - Adjust sign of quotient and remainder as required

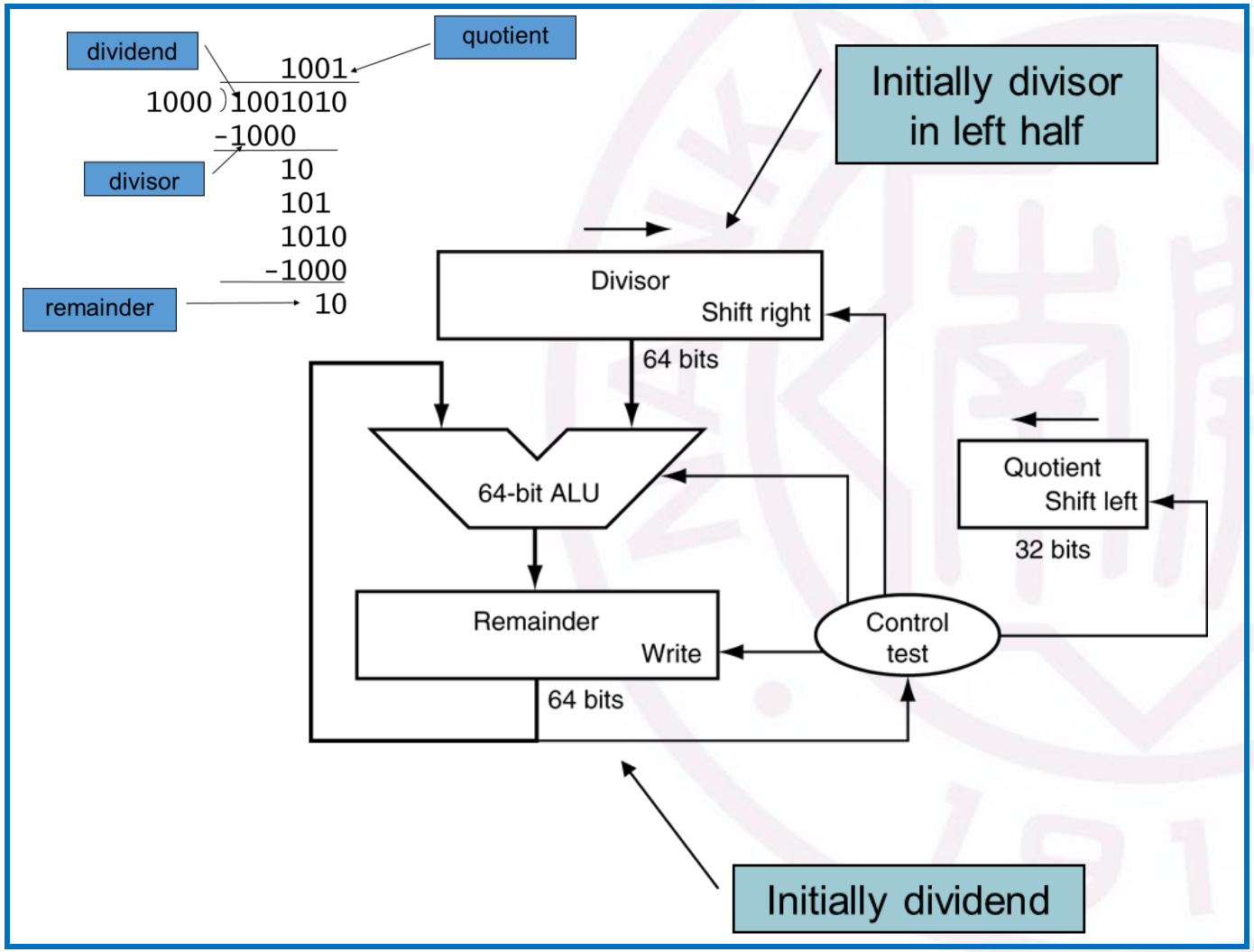
R3-07

整数除法器的设计?



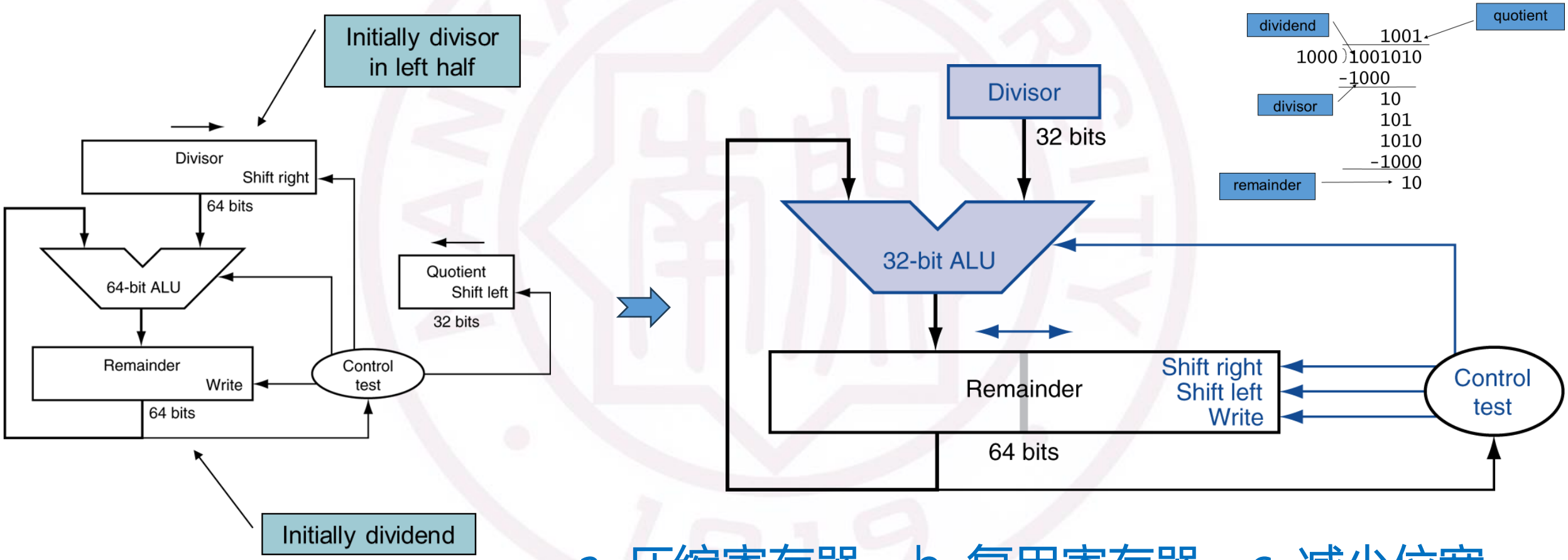


R3-08 整数除法器的优化?



1. 空间角度的优化
   a. 压缩寄存器
   b. 复用寄存器
   c. 减少位宽
2. 时间角度的优化
   a. 右移位?
   b. 多个除法器并行执行?

R3-08 整数除法器的优化？——空间角度



a. 压缩寄存器　b. 复用寄存器　c. 减少位宽

R3-08 整数除法器的优化？——时间角度

- For signed numbers, can't express division by right shift
- Can't use parallel hardware as in multiplier
  - Subtraction is conditional on sign of remainder
- Faster dividers (e.g. SRT devision) generate multiple quotient bits per step
  - Still require multiple steps

R3-09 如何用更少的位宽实现更大的数据表示范围?

浮点数

- Defined by IEEE Std 754-1985 （2019）
  - Including very small and very large numbers
- Like scientific notation
  - $-2.34_{10} \times 10\text{^}56$
  - $+0.002_{10} \times 10\text{^}{-4}$
  - $+987.02_{10} \times 10\text{^}9$
- In binary
  - $\pm 1.xxxxxxx_2 \times 2\text{^}(yyyy_2)$
- Single precision (32-bit) Double precision (64-bit)

R3-09 如何用更少的位宽实现更大的数据表示范围?

single: 8 bits　　single: 23 bits
double: 11 bits　　double: 52 bits

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

- S: sign bit (0 ⇒ non-negative, 1 ⇒ negative)
- Fraction: normalize significand,1.0 ≤ |significand| < 2.0
  - Always has a leading pre-binary-point 1 bit, so no need to represent it explicitly (hidden bit)
  - Significand is Fraction with the "1." --Restored
- Exponent: excess representation: actual exponent + Bias
  - Ensures exponent is unsigned
  - Single: Bias = 127; Double: Bias = 1203

R3-10

浮点数加法的计算流程?

- Now consider a 4-digit binary example
  - $1.000_2 \times 2^{-1} + -1.110_2 \times 2^{-2}$ (0.5 + –0.4375)
- 1. Align binary points
  - Shift number with smaller exponent
  - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1}$
- 2. Add significands
  - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-}1 = 0.001_2 \times 2^{-1}$
- 3. Normalize result & check for over/underflow
  - $1.000_2 \times 2^{-4}$, with no over/underflow
- 4. Round and renormalize if necessary
  - $1.000_2 \times 2^{-4}$ (no change) = 0.0625

R3-11

浮点数乘法的计算流程?

- Now consider a 4-digit binary example
  - $1.000_2 \times 2^{-1} \times -1.110_2 \times 2^{-2}$ (0.5×-0.4375)
- 1. Add exponents
  - Unbiased: $-1 + -2 = -3$
  - Biased: $(-1 + 127) + (-2 + 127) = -3 + 254 - 127 = -3 + 127$
- 2. Multiply significands
  - $1.000_2 \times 1.110_2 = 1.1102 \Rightarrow 1.110_2 \times 2^{-3}$
- 3. Normalize result & check for over/underflow
  - $1.110_2 \times 2^{-3}$ (no change) with no over/underflow
- 4. Round and renormalize if necessary
  - $1.110_2 \times 2^{-3}$ (no change)
- 5. Determine sign: +ve × –ve ⇒ –ve
  - $-1.110_2 \times 2^{-3} = -0.21875$

R3-12

浮点数的精度处理对计算的印象?

- Floating-point addition is not associative

		(x+y)+z	x+(y+z)
x	-1.50E+38		-1.50E+38
y	1.50E+38	0.00E+00	
z	1.0	1.0	1.50E+38
		1.00E+00	0.00E+00

- Bits have no inherent meaning
  - Interpretation depends on the instructions applied
- Computer representations of numbers
  - Finite range and precision
  - Need to account for this in programs

R4-01

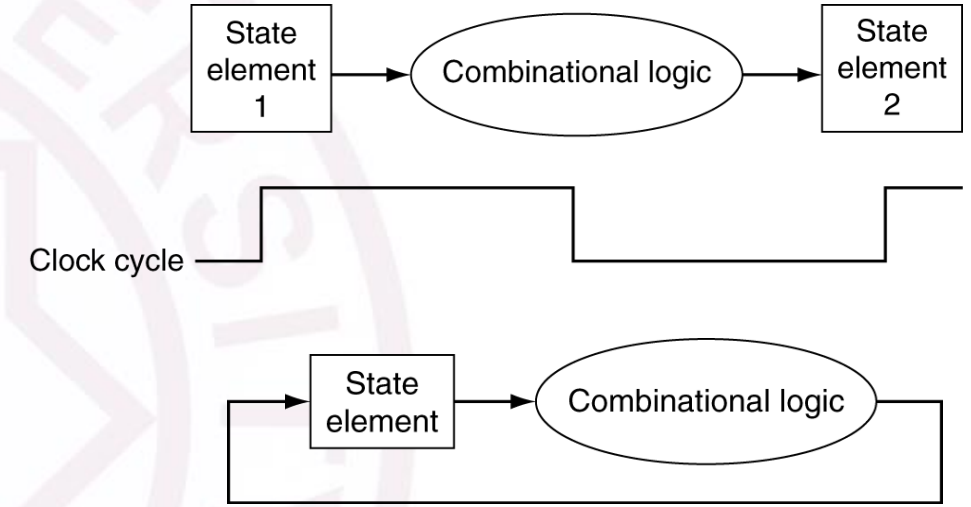
如何设计一个计算机的处理器?

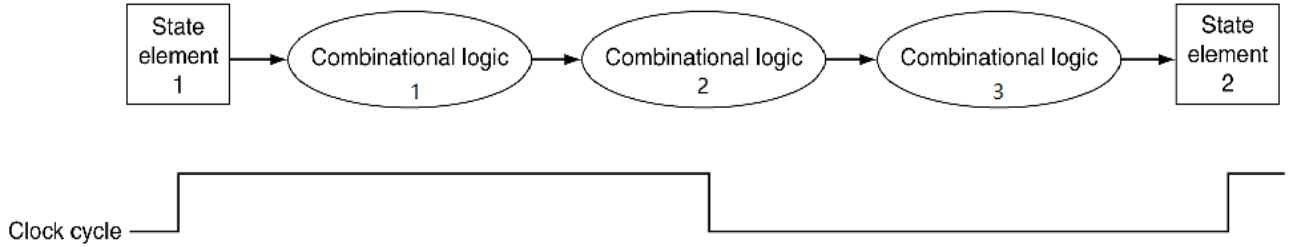
- PC → instruction memory, fetch instruction
- Register numbers → register file, read registers
- Depending on instruction class →
  - Use ALU to calculate
    - Arithmetic result
    - Memory address for load/store
    - Branch target address
  - Access data memory for load/store
  - PC ← target address or PC + 4

- PC
- Register file
- ALU
- Data Memory
- Instruction Memory

R4-01 如何设计一个计算机的处理器?

- Information encoded in binary
  - Low voltage = 0, High voltage = 1
  - One wire per bit
  - Multi-bit data encoded on multi-wire buses
- Combinational element
  - **Operate on data**
  - Output is a function of input
- State (sequential) elements
  - **Store information**

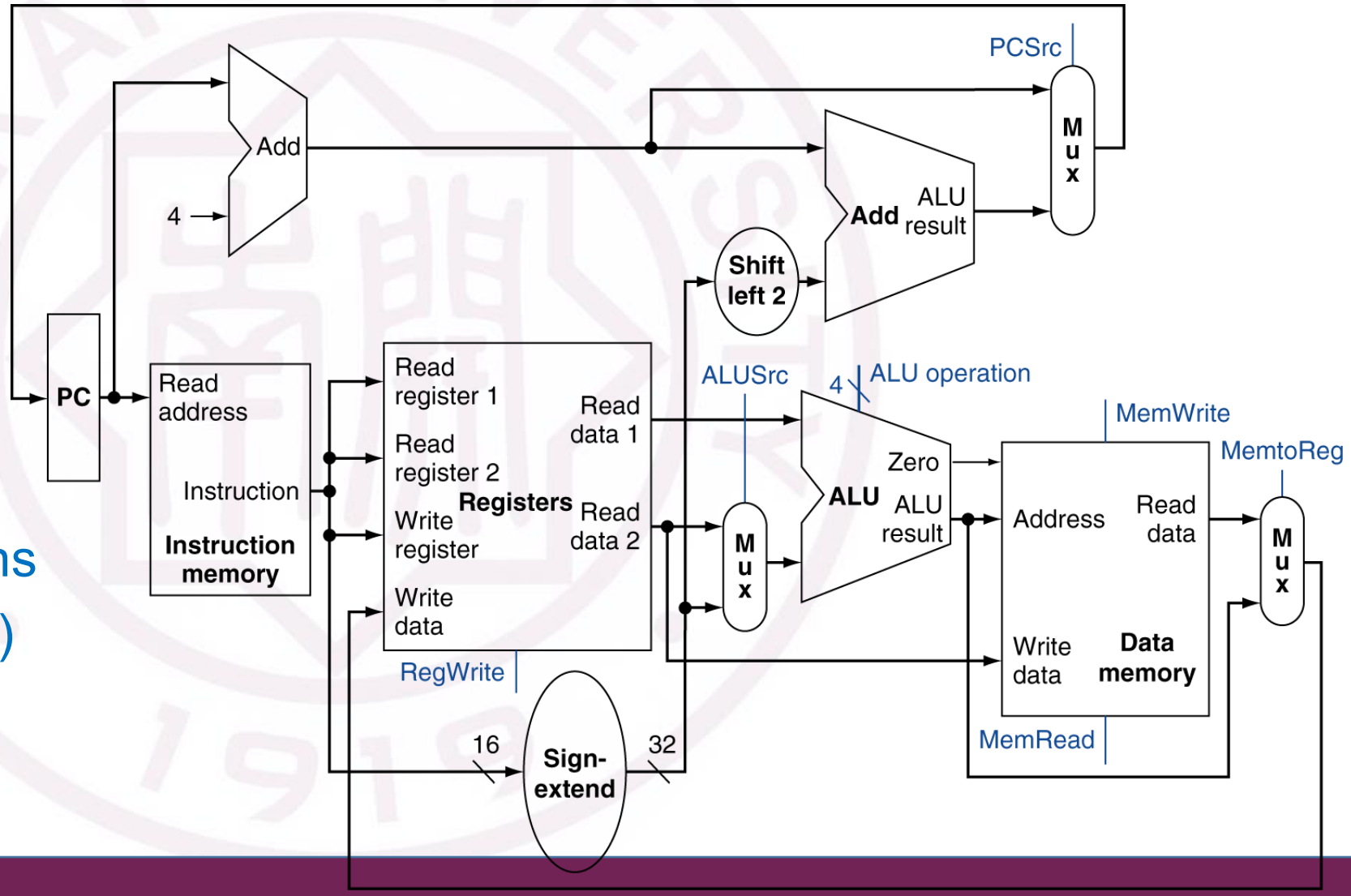




R4-02 如何建立处理器的数据通路?

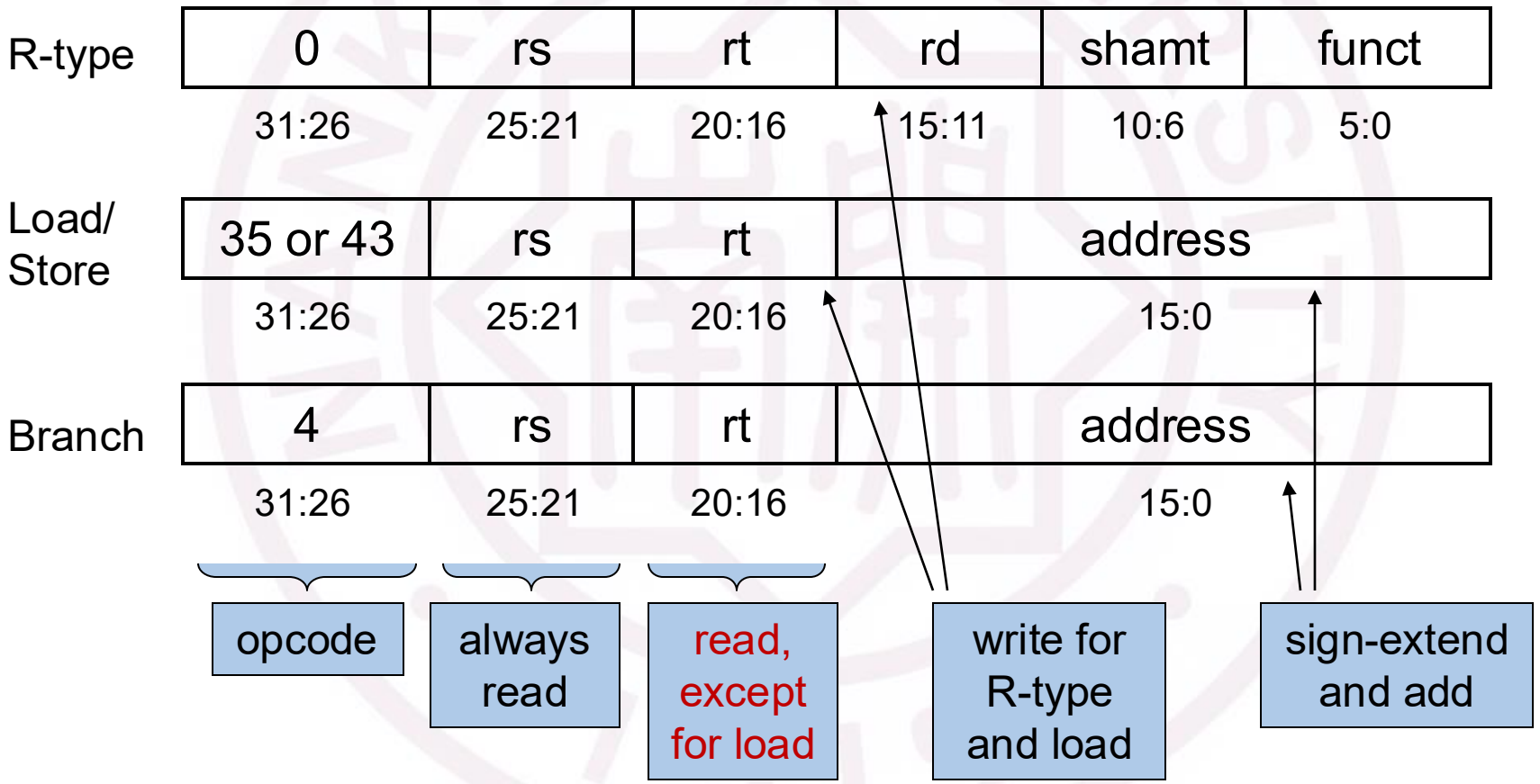
- Subset (most aspects)
  - Memory: `lw`, `sw`
  - Arithmetic/logical: `add`, `sub`, `and`, `or`, `slt`
  - Control transfer: `beq`, `j`
- Addressing Modes
- Adapt to different instructions

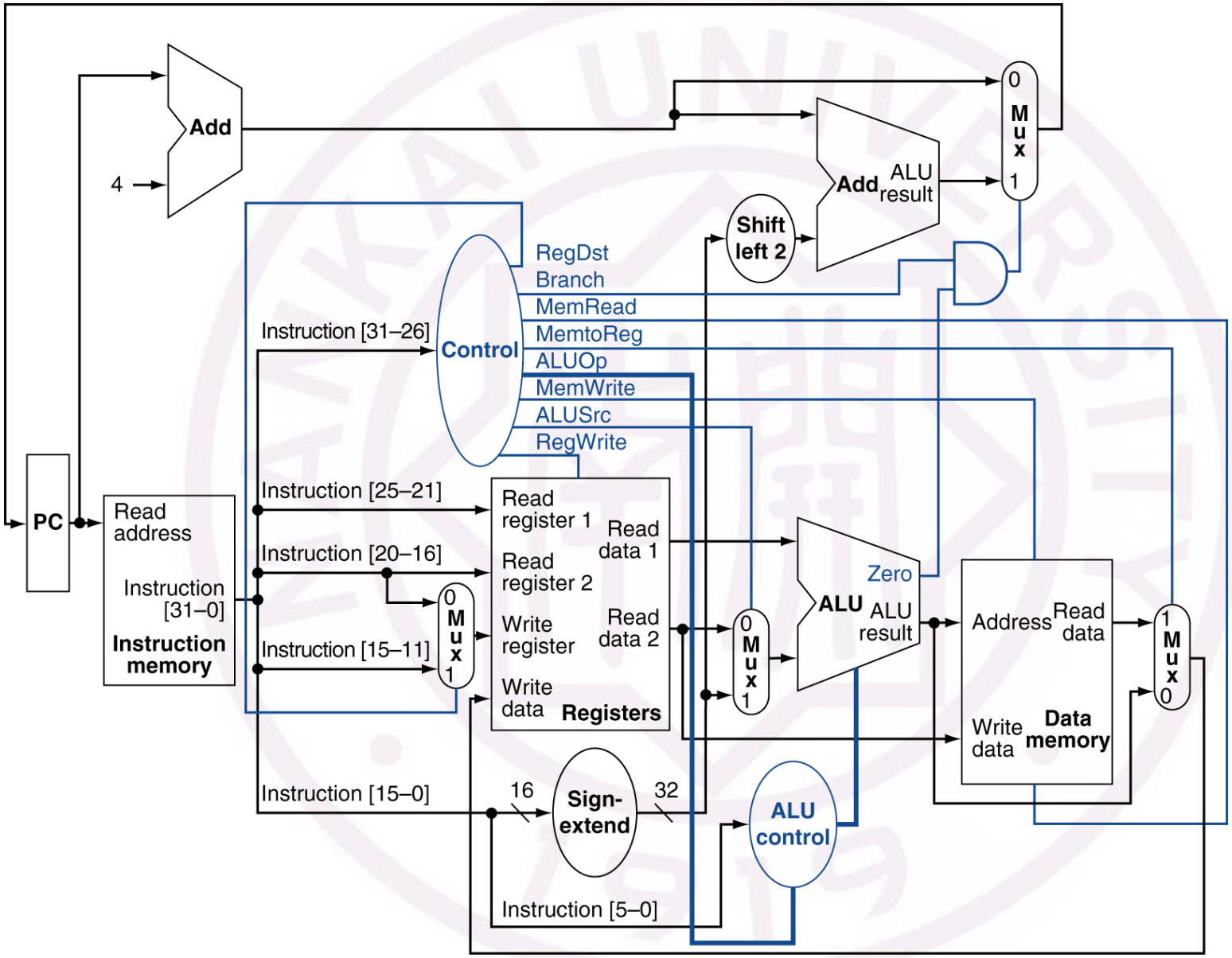
(R-format, I-format, J-format)

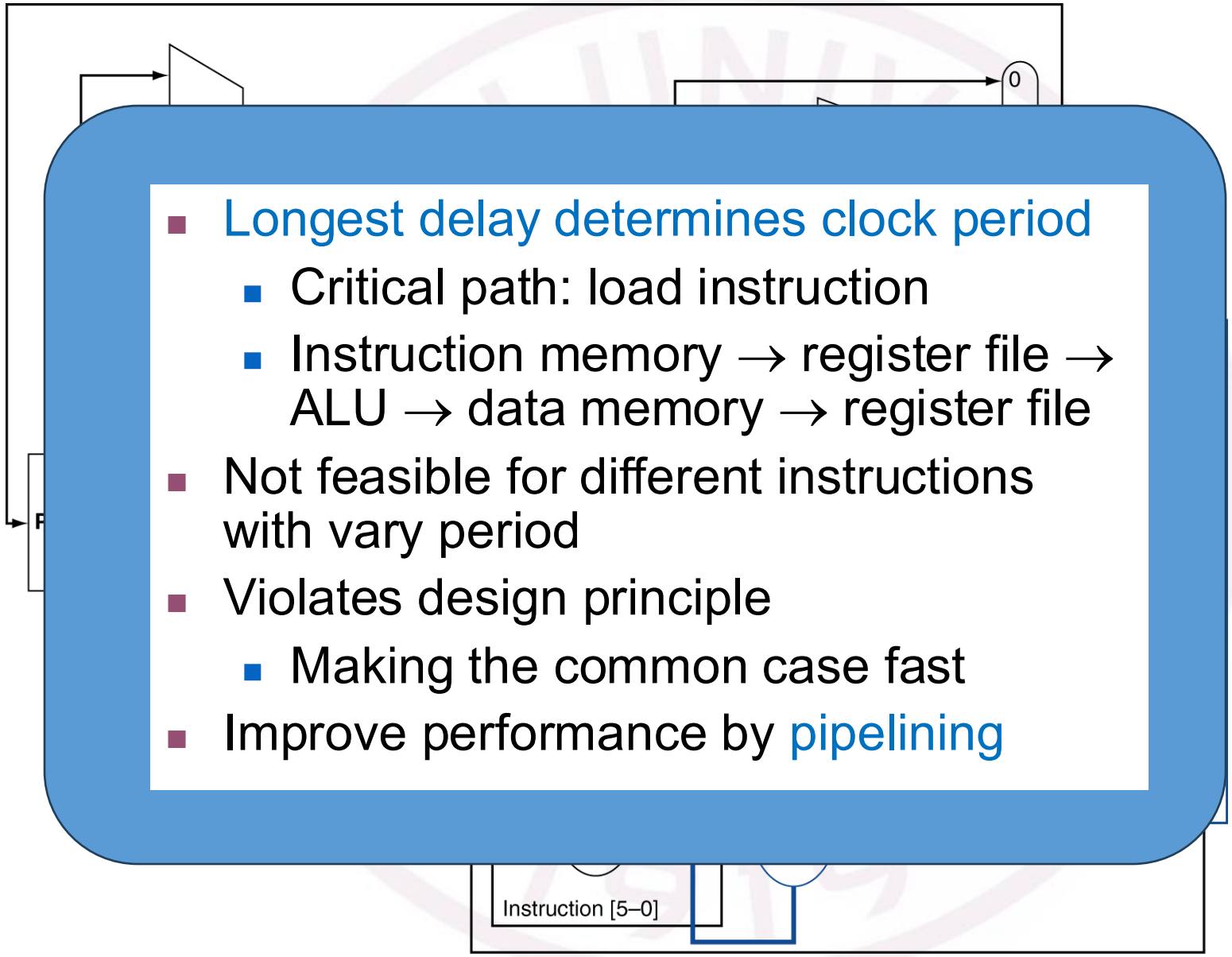


R4-03

如何生成数据通路上控制信号?



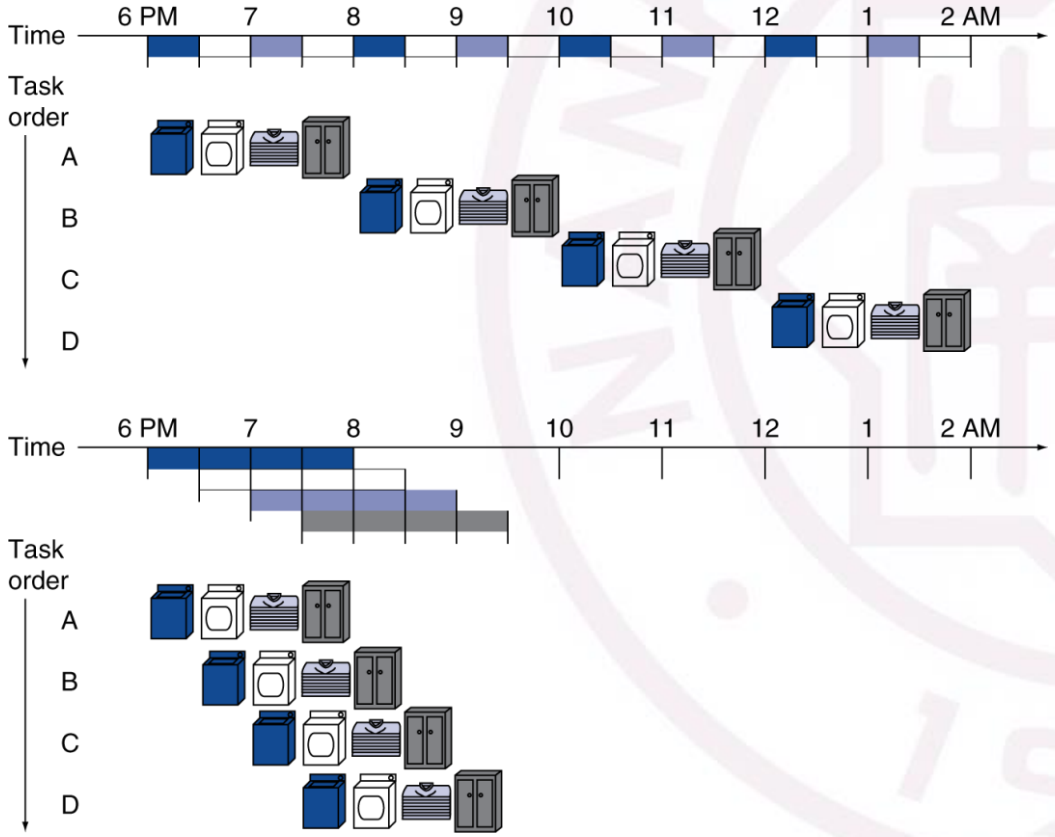

Add
4
Add
ALU result
Shift left 2
0
M
u
x
1
RegDst
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Control
Instruction [31–26]
PC
Read address
Instruction [31–0]
Instruction memory
Instruction [25–21]
Instruction [20–16]
Instruction [15–11]
Instruction [15–0]
Instruction [5–0]
Read register 1
Read register 2
Write register
Write data
Read data 1
Read data 2
Registers
16
Sign-extend
32
ALU
Zero
ALU result
ALU control
Address
Read data
Write data
Data memory



- Longest delay determines clock period
  - Critical path: load instruction
  - Instruction memory → register file → ALU → data memory → register file
- Not feasible for different instructions with vary period
- Violates design principle
  - Making the common case fast
- Improve performance by pipelining

R4-04 流水线是如何设计的?

- Pipelined laundry: overlapping execution



- Four loads:
  - Speedup = 8/3.5 = 2.3
- Non-stop:
  - Speedup = 2n / (0.5n + 1.5) ≈ 4 = number of stages

R4-04 流水线是如何设计的?

单元分割、异常处理、局部优化

- Five stages, one step per stage
  - IF: Instruction fetch from memory
  - ID: Instruction decode & register read
  - EX: Execute operation or calculate address
  - MEM: Access memory operand
  - WB: Write result back to register

等长—多周期

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

R4-04

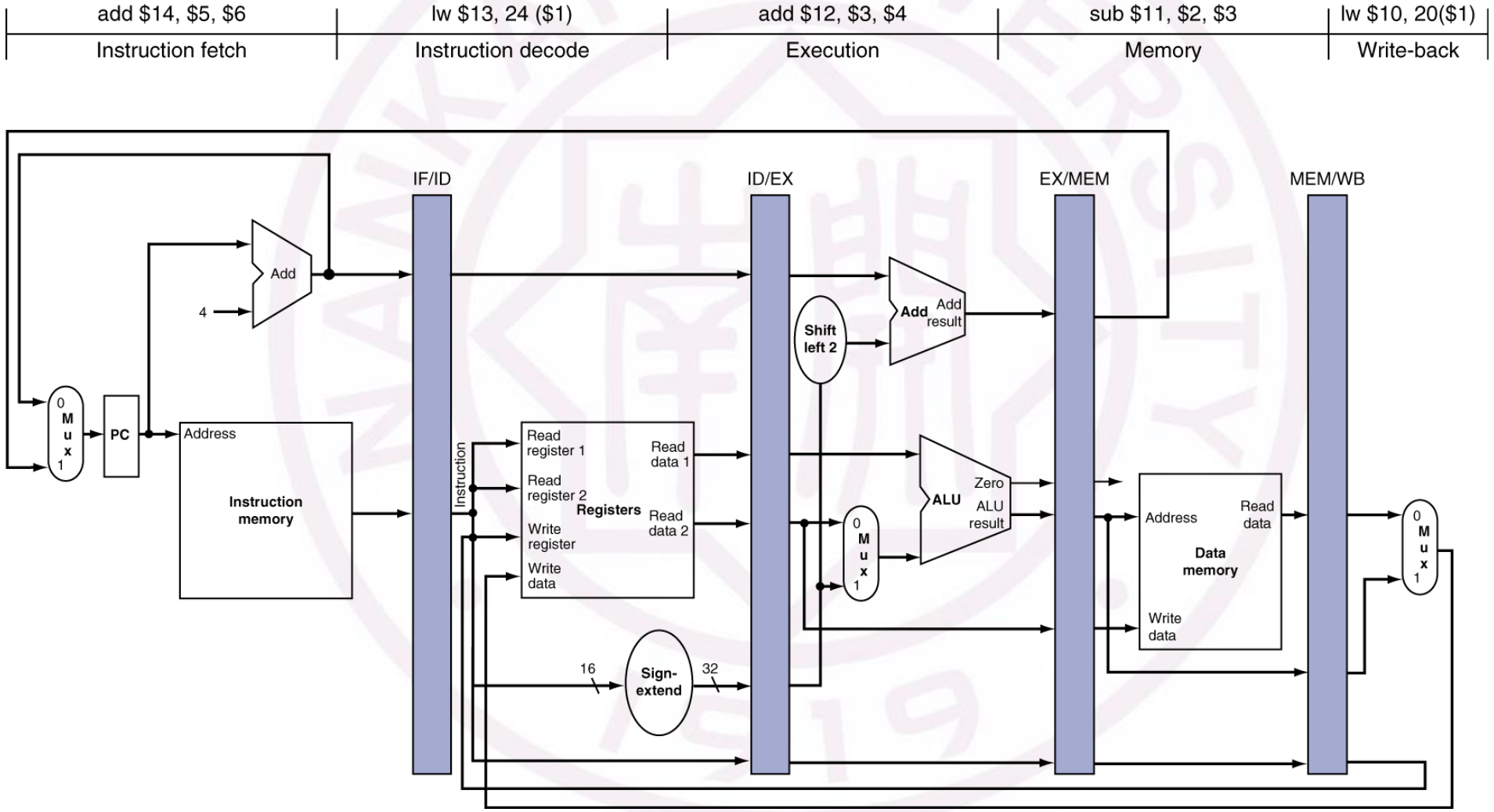
流水线是如何设计的?

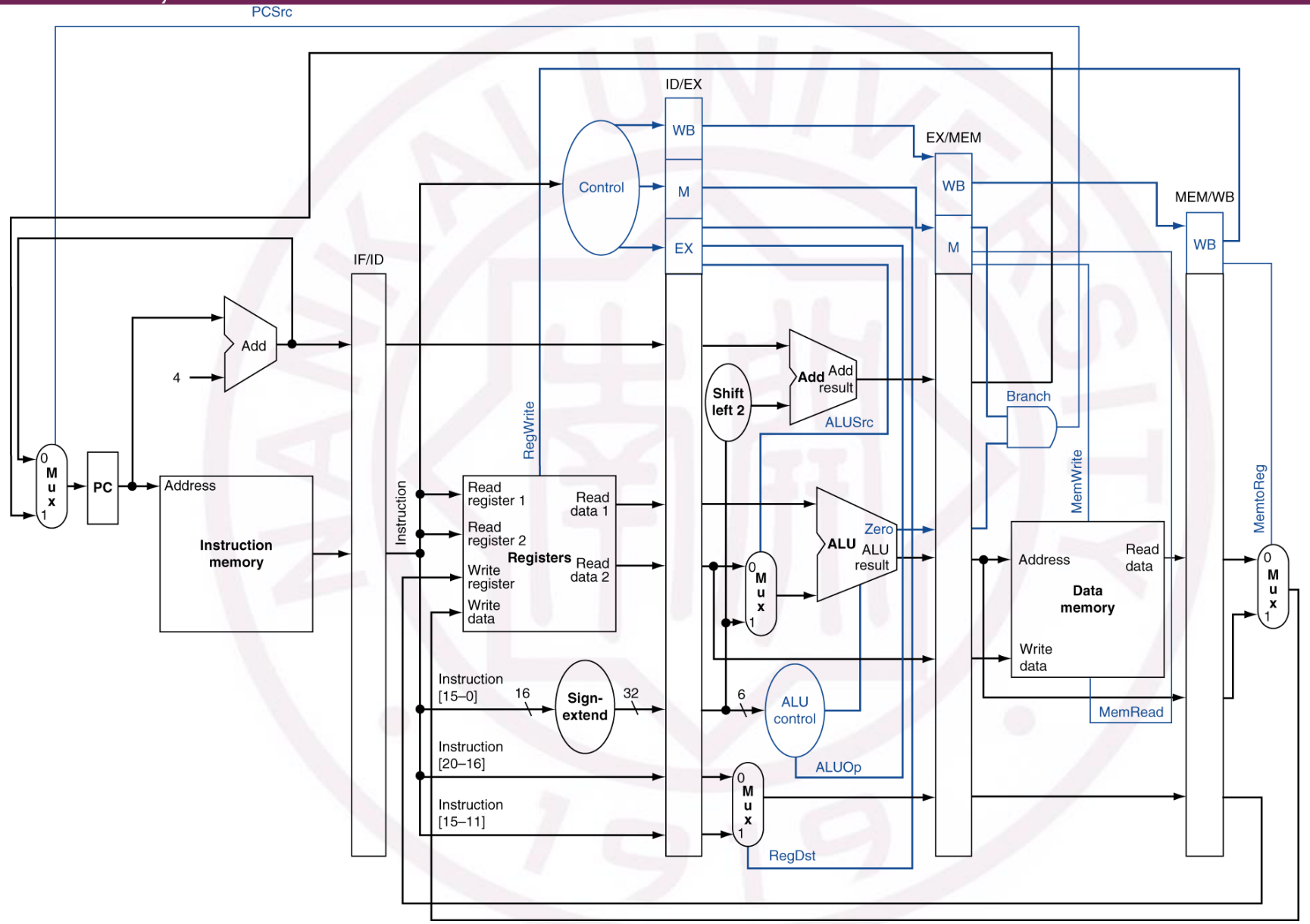
- If all stages are balanced
  - i.e., all take the same time
  - Time between $\text{instructions}_{\text{pipelined}}$ $= \dfrac{\text{Time between instructions}_{\text{nonpipelined}}}{\text{Number of stages}}$
- If not balanced, speedup is less
- Speedup due to increased throughput
- Latency (time for each instruction) does not decrease

CPU流水线为什么不是越长越好?

功率墙问题

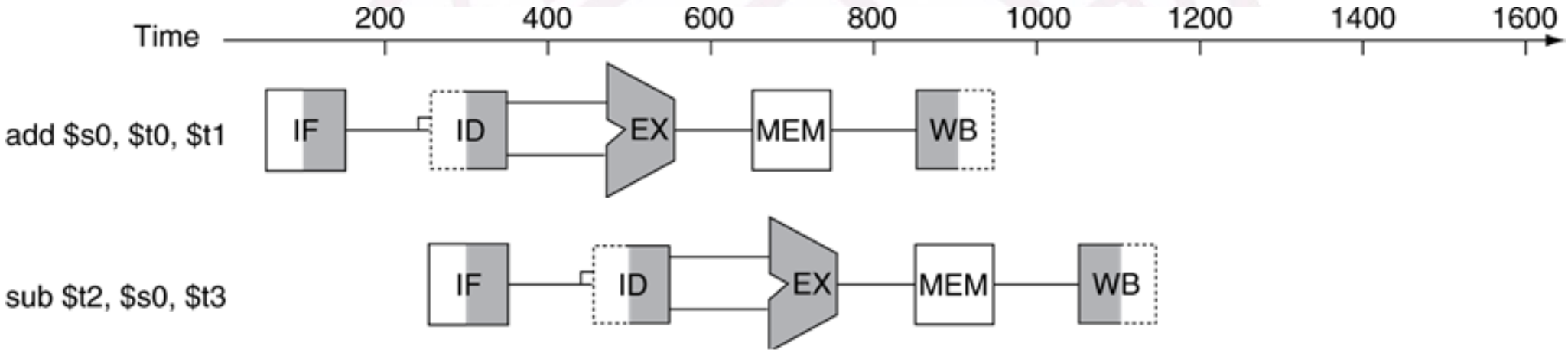
R4-05 流水线的数据通路?




PCSrc
ID/EX
EX/MEM
MEM/WB
IF/ID
WB
M
EX
Control
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
RegWrite
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALUSrc
Branch
MemWrite
MemtoReg
ALU
Zero
ALU result
Address
Data memory
Read data
Write data
MemRead
Instruction [15–0]
16
Sign-extend
32
6
ALU control
ALUOp
Instruction [20–16]
Instruction [15–11]
RegDst

R4-05

流水线的数据通路？——图形表示



R4-06 CPU的流水线中的异常?

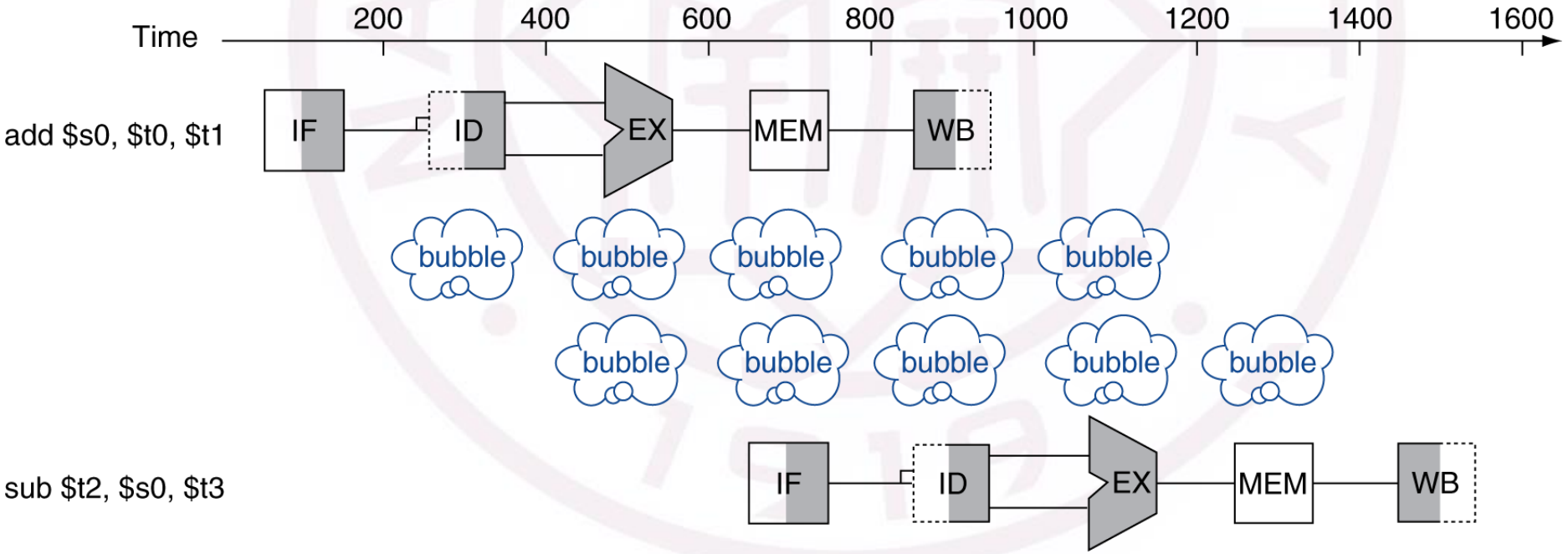
- Situations that prevent starting the next instruction in the next cycle
- Structure hazards
  - A required resource is busy
- Data hazard
  - Need to wait for previous instruction to complete its data read/write
- Control hazard
  - Deciding on control action depends on previous instruction

R4-06 CPU的流水线中的异常？——结构阻塞/冒险

- Conflict for use of a resource
- In MIPS pipeline with a single memory
  - Load/store requires data access
  - Instruction fetch would have to *stall* for that cycle
    - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require separate instruction/data memories
  - Or separate instruction/data caches

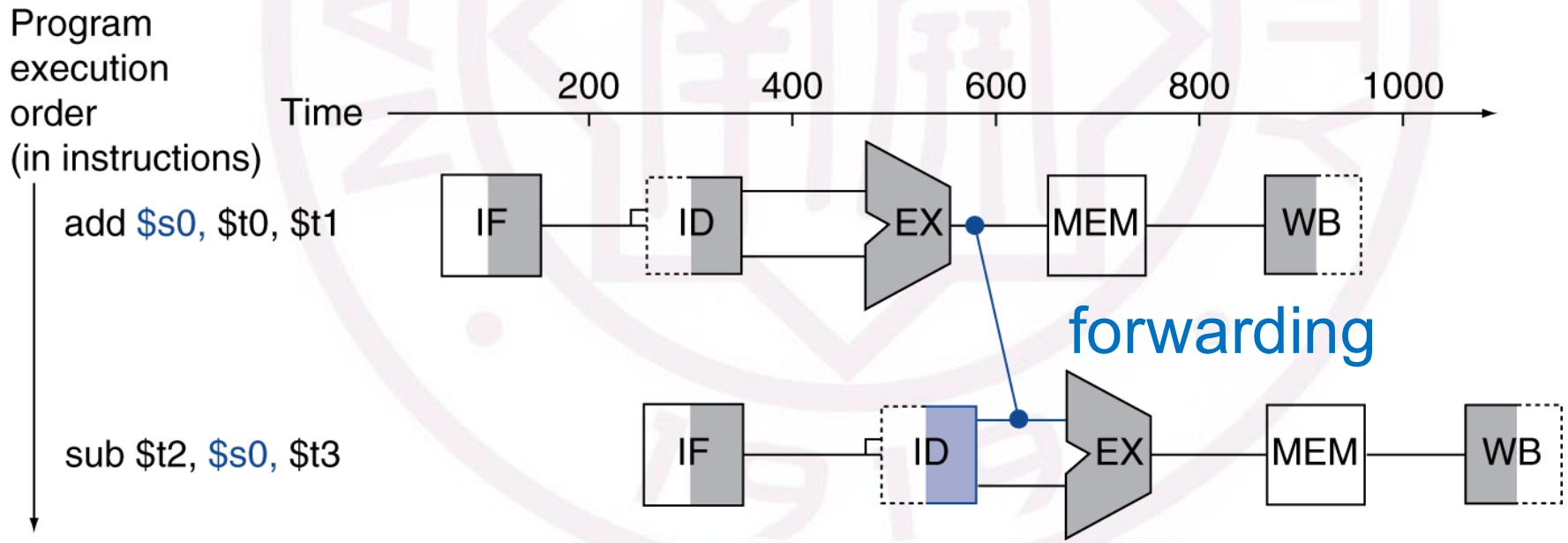
R4-06 CPU的流水线中的异常？——数据阻塞/冒险

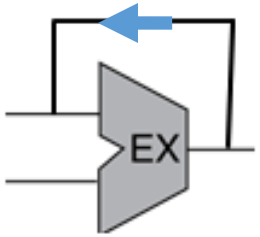
- An instruction depends on completion of data access by a previous instruction
  - `add $s0, $t0, $t1`
    `sub $t2, $s0, $t3`



R4-06 CPU的流水线中的异常？——数据阻塞/冒险

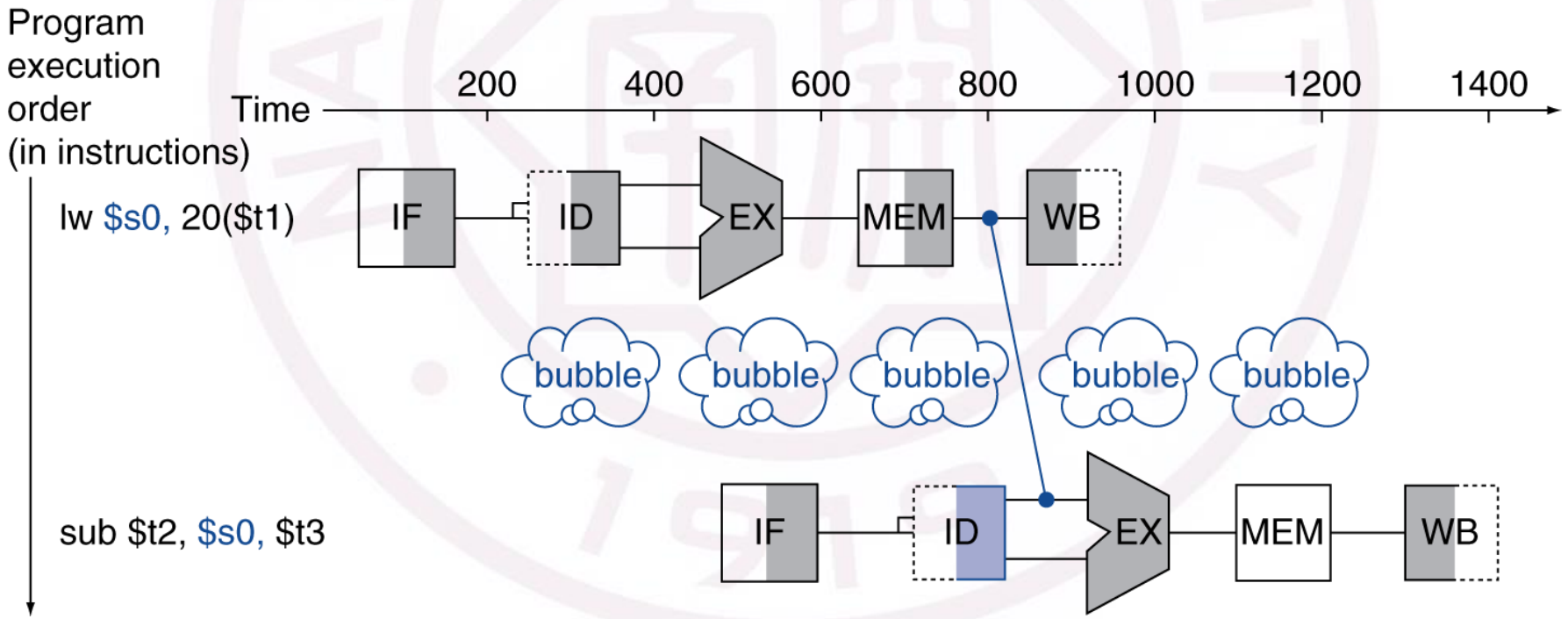
- An instruction depends on completion of data access by a previous instruction
  - `add $s0, $t0, $t1`
    `sub $t2, $s0, $t3`





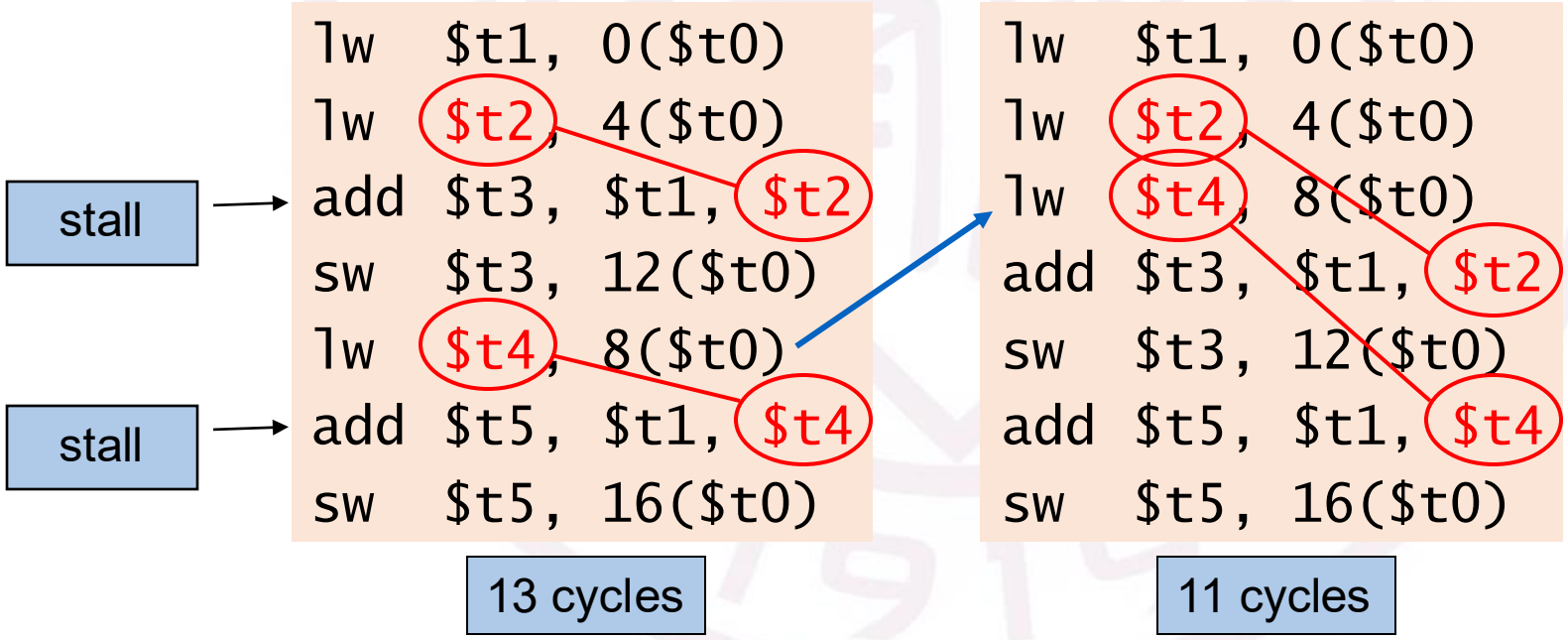
R4-06 CPU的流水线中的异常？——数据阻塞/冒险

- Can't always avoid stalls by forwarding
  - If value not computed when needed
  - Can't forward backward in time!

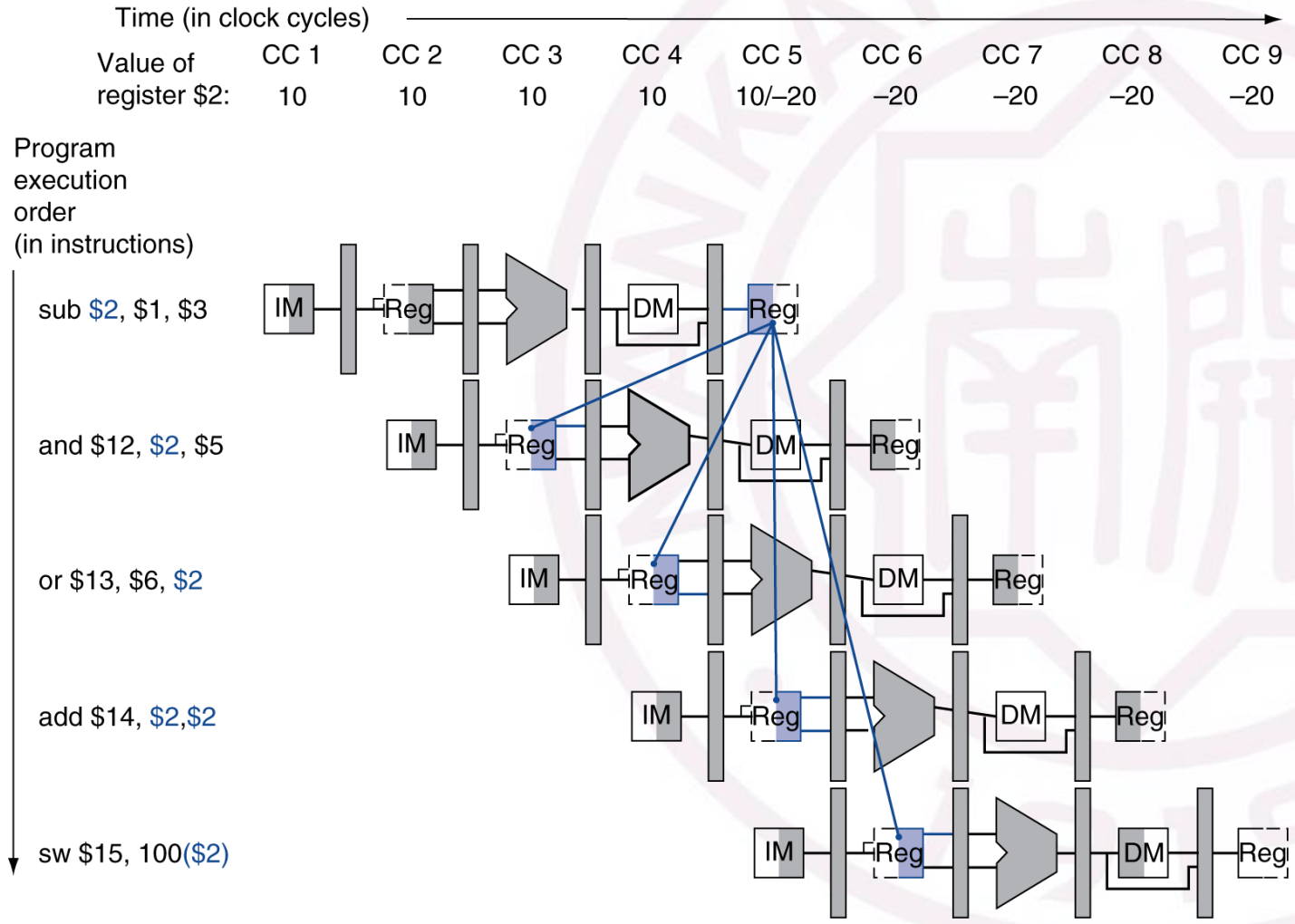


R4-06 CPU的流水线中的异常？——数据阻塞/冒险

- Reorder code (compilers) to avoid use of load result in the next instruction
- C code for `A = B + E; C = B + F;`



R4-06 CPU的流水线中的异常？——数据阻塞/冒险——判断情况1



如何检测到数据相关的发生？

- Pass register numbers along pipeline
  - e.g., ID/EX.RegisterRs = register number for Rs sitting in ID/EX pipeline register
- ALU operand register numbers in EX stage are given by
  - ID/EX.RegisterRs
    ID/EX.RegisterRt

R4-06 CPU的流水线中的异常？——数据阻塞/冒险——判断情况1

Data hazards when

1a. EX/MEM.RegisterRd = ID/EX.RegisterRs
1b. EX/MEM.RegisterRd = ID/EX.RegisterRt

} Fwd from EX/MEM pipeline reg

2a. MEM/WB.RegisterRd = ID/EX.RegisterRs
2b. MEM/WB.RegisterRd = ID/EX.RegisterRt

} Fwd from MEM/WB pipeline reg

Q1: 有了Rd，但确不会回写?

EX/MEM.RegWrite

MEM/WB.RegWrite

Q2: Rd 为$zero时?

EX/MEM.RegisterRd ≠ 0

MEM/WB.RegisterRd ≠ 0

R4-06 CPU的流水线中的异常？——数据阻塞/冒险——判断情况1

- **EX hazard**
  - if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
    ForwardA = 10
  - if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
    ForwardB = 10
- ***MEM hazard***
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs))
    ForwardA = 01
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and (MEM/WB.RegisterRd = ID/EX.RegisterRt))
    ForwardB = 01

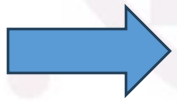
R4-06 CPU的流水线中的异常？——数据阻塞/冒险——判断情况1

- Consider the sequence:

```
add $1,$1,$2
add $1,$1,$3
add $1,$1,$4
```

vs.

```
add $1,$1,$2
add $5,$1,$3
add $1,$1,$4
```

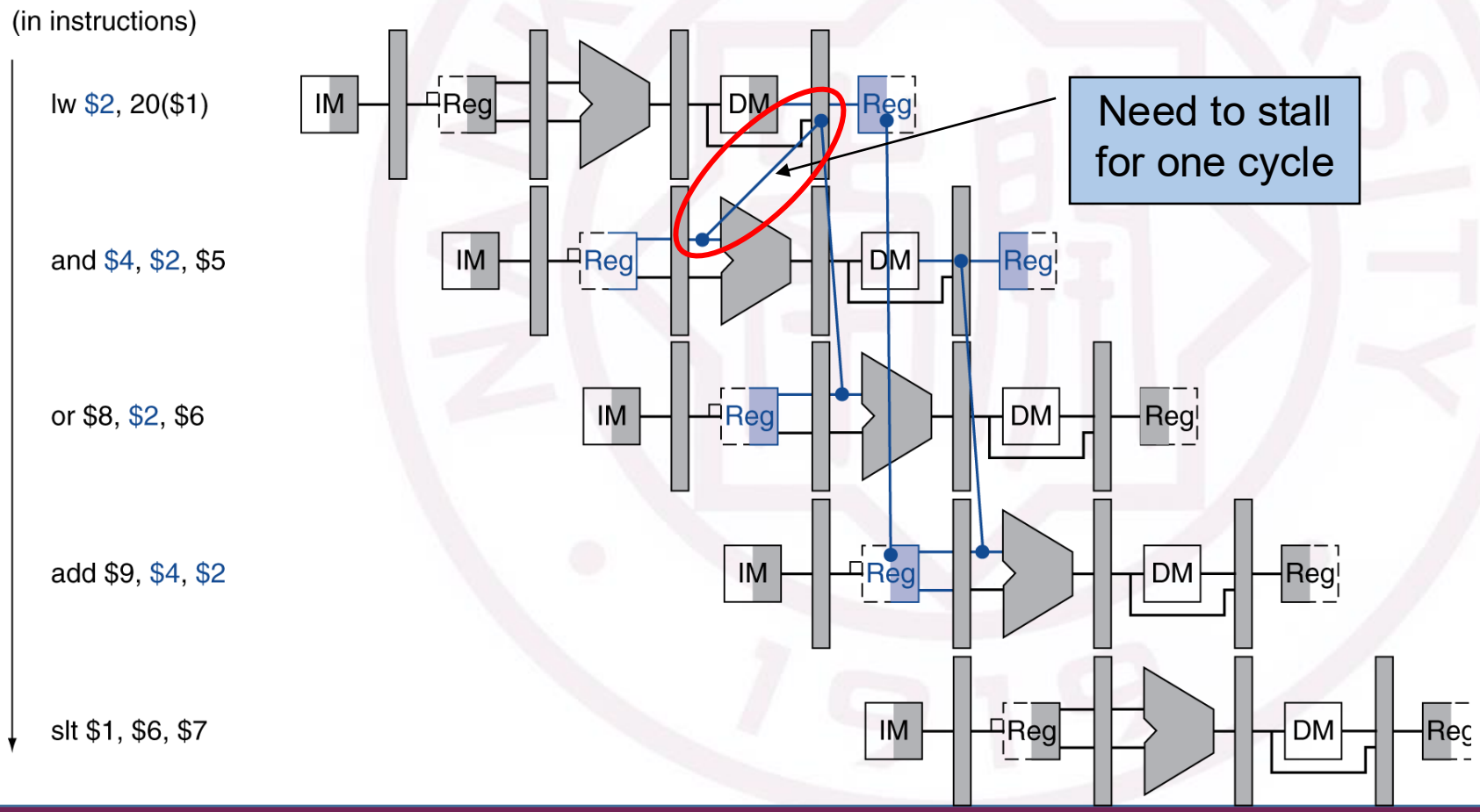


Revise MEM hazard

- if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
  and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
  and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
  and (MEM/WB.RegisterRd = ID/EX.RegisterRs))
  ForwardA = 01
- if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
  and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
  and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
  and (MEM/WB.RegisterRd = ID/EX.RegisterRt))
  ForwardB = 01

R4-06 CPU的流水线中的异常？——数据阻塞/冒险——判断情况2

Load—Use又该如何检测?



Load—Use
又该如何检测?

R4-06 CPU的流水线中的异常？——数据阻塞/冒险——判断情况2

- Check when using instruction is decoded in ID stage
- ALU operand register numbers in ID stage are given by
  - IF/ID.RegisterRs, IF/ID.RegisterRt
- Load-use hazard when
  - ID/EX.MemRead and
    ((ID/EX.Register**Rt** = IF/ID.RegisterRs) or
    (ID/EX.Register**Rt** = IF/ID.RegisterRt))
- **If detected, stall and insert bubble**

Load—Use
又该如何检测?

R-型

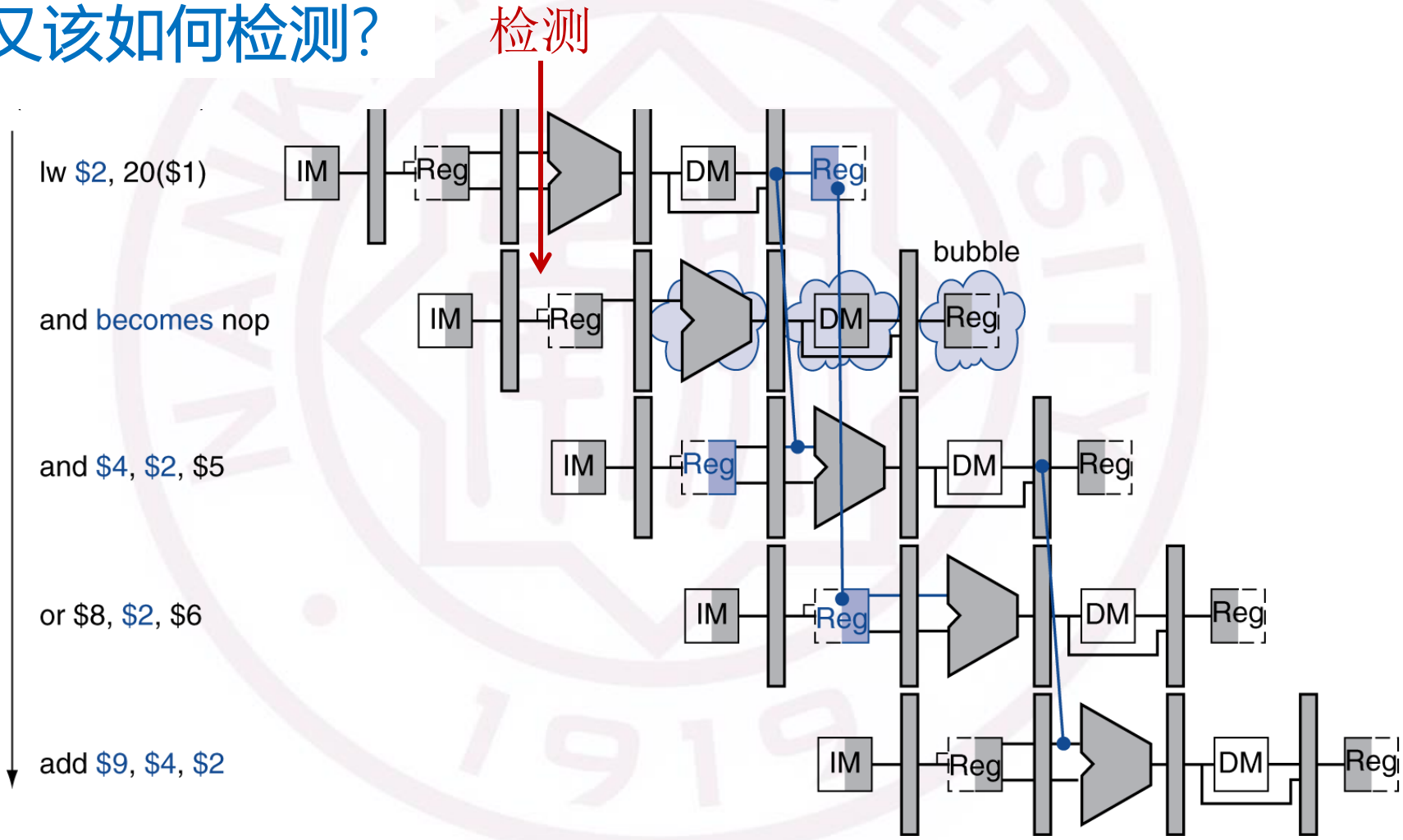
op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

I-型

op	rs	rt	constant or address
6 bits	5 bits	5 bits	16 bits

R4-06 CPU的流水线中的异常？——数据阻塞/冒险——判断情况2

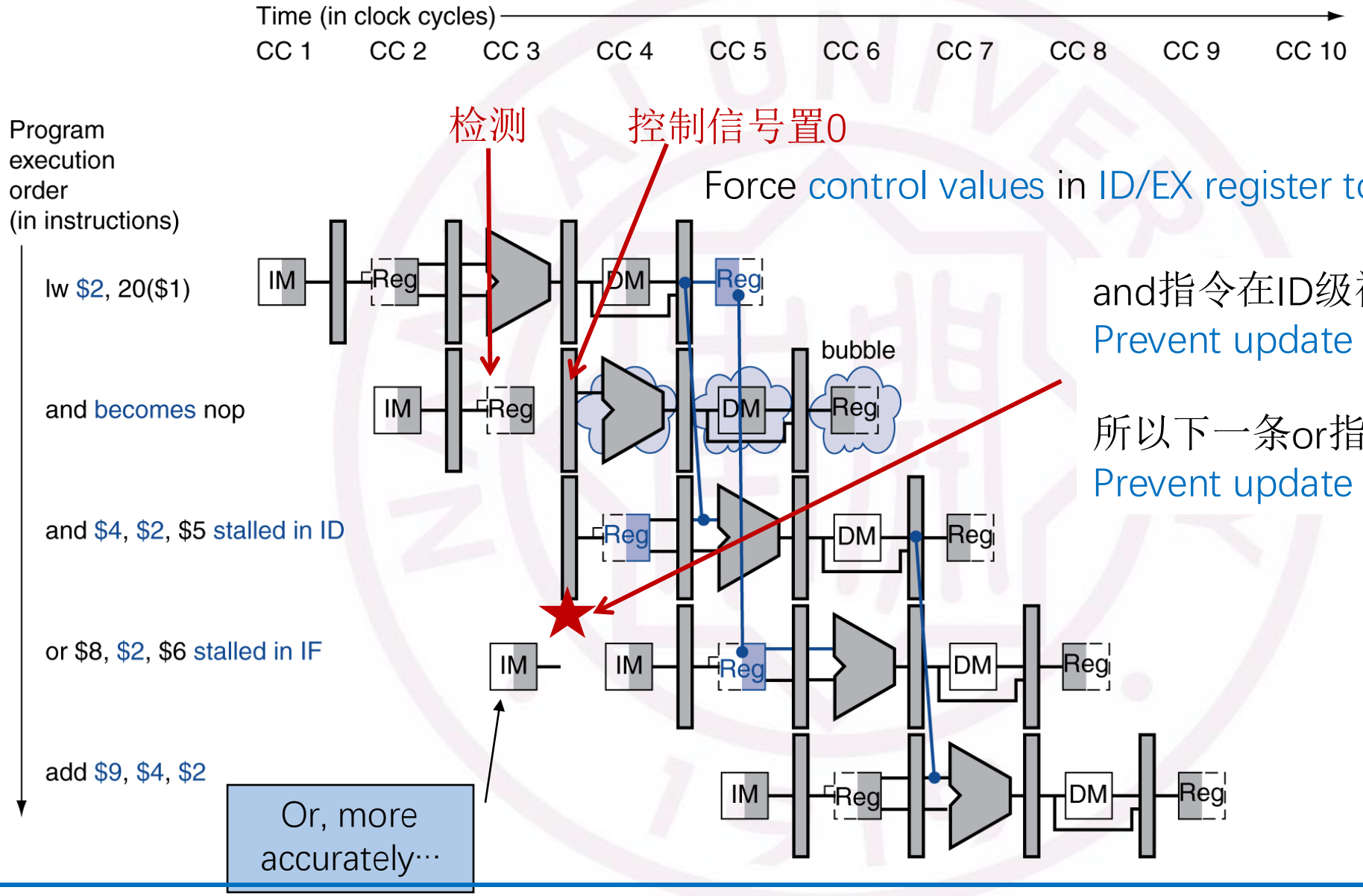
Load—Use又该如何检测?



R4-06 CPU的流水线中的异常？——数据阻塞/冒险——判断情况2

Load—Use又该如何处理?

- Force control values in ID/EX register to 0
  - EX, MEM and WB do **nop** (no-operation)
- Prevent update of PC and IF/ID register
  - Using instruction is decoded again
  - 1-cycle stall allows MEM to read data for `lw`
    - Can subsequently forward to EX stage

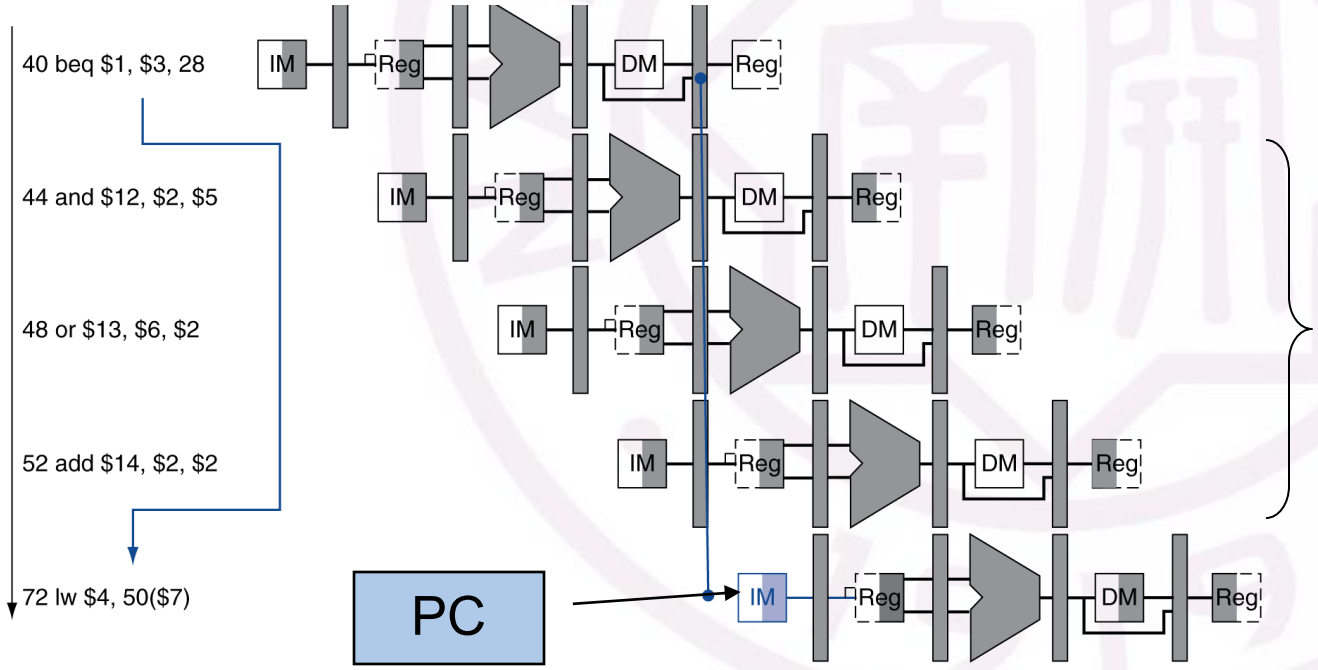


and指令在ID级被阻塞，
Prevent update of IF/ID register

所以下一条or指令需要在IF级被阻塞
Prevent update of PC

R4-06 CPU的流水线中的异常？——控制阻塞/冒险

- Longer pipelines can't readily determine branch outcome early
  - Stall penalty becomes unacceptable

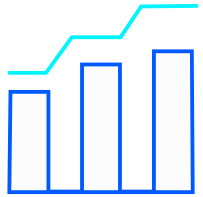


If branch outcome determined in MEM

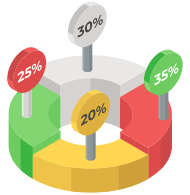
Flush these instructions (Set control values to 0)

R4-06 CPU的流水线中的异常？——控制阻塞/冒险

- Static branch prediction
  - Based on typical branch behavior
  - Example: loop and if-statement branches
    - Predict backward branches taken
    - Predict forward branches not taken
- Dynamic branch prediction
  - Branch prediction buffer (aka branch history table, 1bit)
  - Indexed by recent branch instruction addresses
  - Stores outcome (taken/not taken)
  - Execute a branch: **Check table、 Start fetching、If wrong, flush pipeline** and **flip prediction**

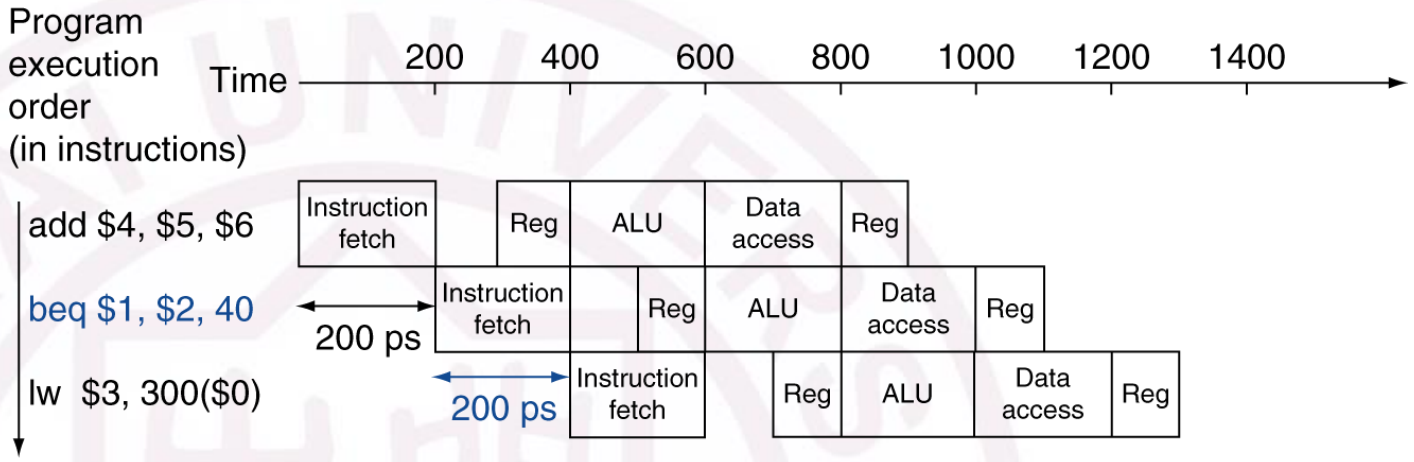


统计分析

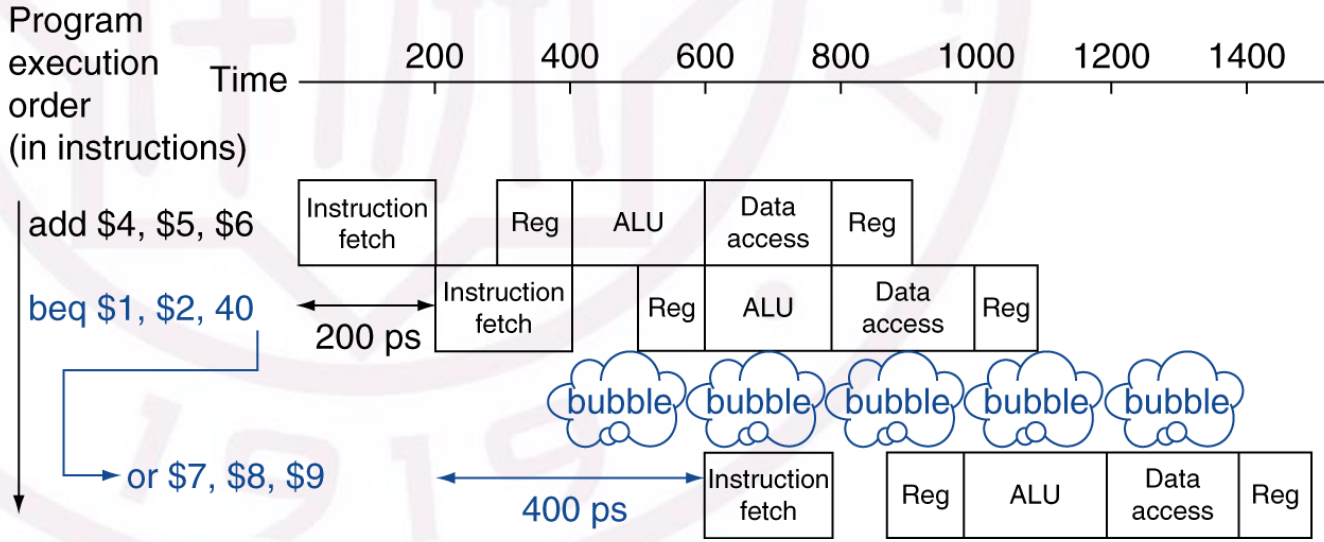


行为记录

Prediction correct

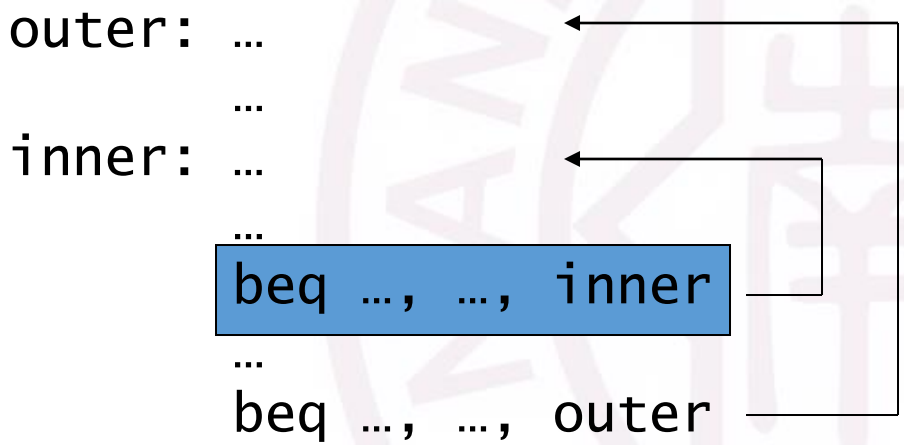


Prediction incorrect



R4-06 CPU的流水线中的异常？——控制阻塞/冒险

- Dynamic branch prediction (1bit)——双重循环问题



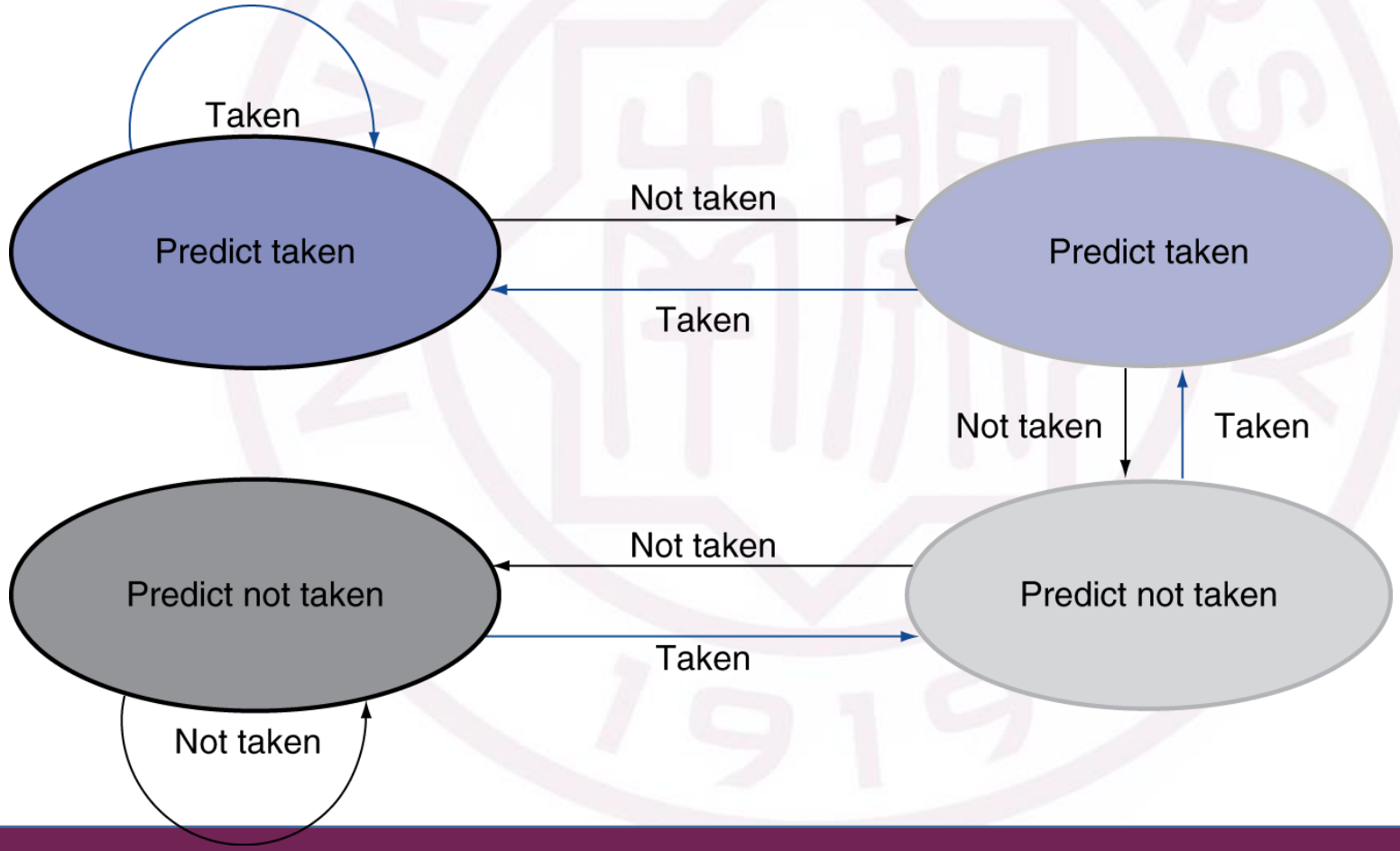
Inner loop branches

mispredicted twice!

- Mispredict as taken on last iteration of inner loop
- Then mispredict as not taken on first iteration of inner loop next time around

R4-06 CPU的流水线中的异常？——控制阻塞/冒险

- Dynamic branch prediction (1bit)——双重循环问题



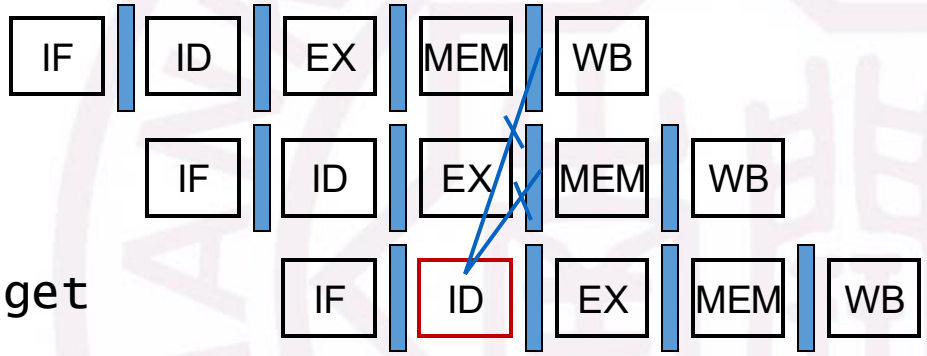
2bit 预测位
有限状态机设计

反复跳转

R4-07 CPU的流水线中的异常？——数据和控制相关同时发生

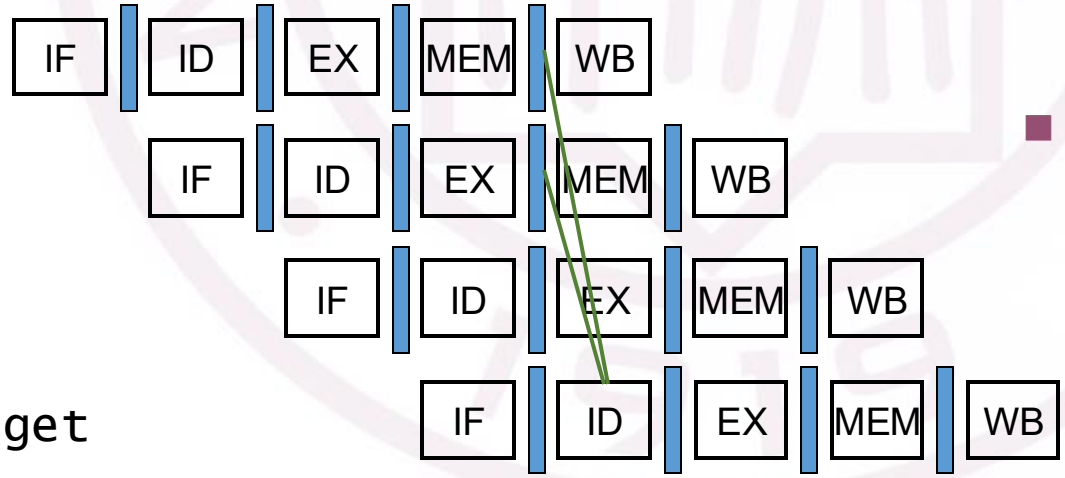
1. 当R型遇上Beq会发生什么？

```
add $1, $2, $3
add $4, $5, $6
beq $1, $4, target
```



- Can’t resolve using forwarding

```
add $1, $2, $3
add $4, $5, $6
…
beq $1, $4, target
```



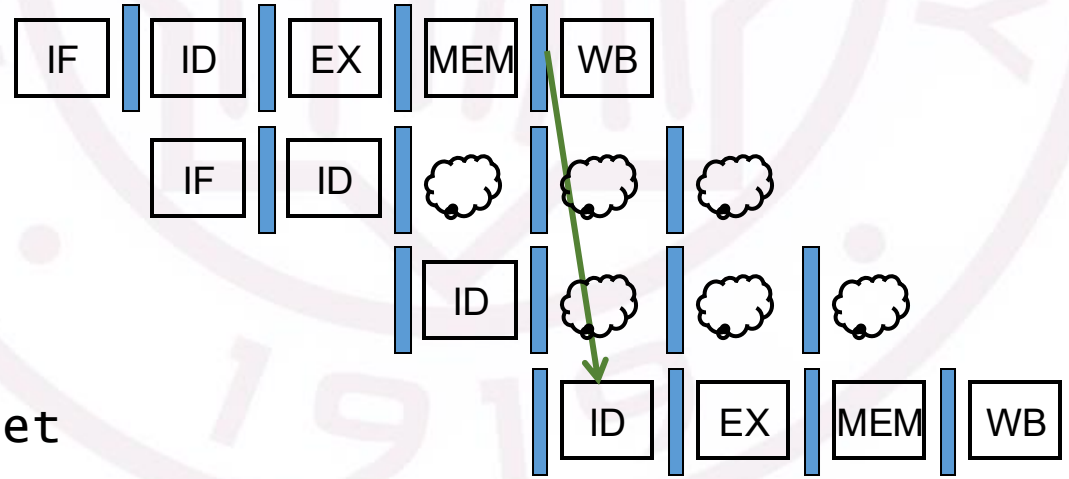
- Can resolve using forwarding

R4-07 CPU的流水线中的异常？——数据和控制相关同时发生

2. 当Load遇上Beq会发生什么？

- If a comparison register is a destination of immediately preceding load instruction
  - Need 2 stall cycles

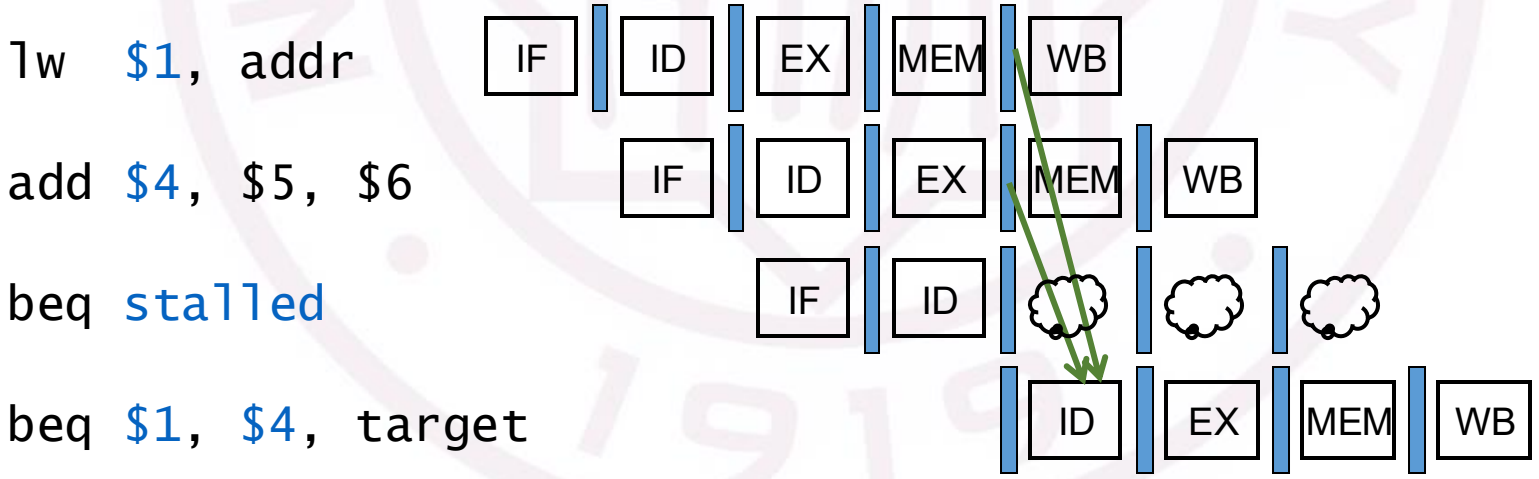
```
lw   $1, addr
beq stalled
beq stalled
beq $1, $0, target
```



R4-07 CPU的流水线中的异常？——数据和控制相关同时发生

3. 当Load+R型遇上Beq会发生什么？

- If a comparison register is a destination of preceding ALU instruction or 2nd preceding load instruction
  - Need 1 stall cycle



R4-08 CPU的流水线中的异常和中断区别?

- **“Unexpected” events** requiring change in flow of control
  - Different ISAs use the terms differently
- Exception
  - Arises within the CPU
    - e.g., undefined opcode, overflow, syscall, …
- Interrupt
  - From an external I/O controller

Dealing with them without sacrificing performance is hard

R4-09 MIPS异常和中断如何处理?

- In MIPS, exceptions managed by a System Control Coprocessor (CP0)
- Save PC of offending (or interrupted) instruction
  - In MIPS: Exception Program Counter (EPC)
- Save indication of the problem
  - In MIPS: Cause register
  - We'll assume 1-bit
    - 0 for undefined opcode, 1 for overflow
- Jump to handler at 8000 00180 (Default of MIPS)

R4-10

流水线的优化?

R4-10

流水线的优化？——更长流水线

型号	内核/线程	基本频率	加速频率	第三级高速缓存	典型设计功耗	微架构代号	内核代号
AMD Ryzen 9 系列 AM4 插座							
Ryzen 9 3900X	12/24	3.8 GHz	4.6 GHz	64 MiB	105 Watt	Zen 2	Matisse
Ryzen 9 3900XT	12/24	3.8 GHz	4.7 GHz	64 MiB	105 Watt	Zen 2	Matisse
Ryzen 9 5900X	12/24	3.7 GHz	4.8 GHz	64 MiB	105 Watt	Zen 3	Vermeer
Ryzen 9 3950X	16/32	3.5 GHz	4.7 GHz	64 MiB	105 Watt	Zen 2	Matisse
Ryzen 9 5950X	16/32	3.4 GHz	4.9 GHz	64 MiB	105 Watt	Zen 3	Vermeer
AMD Ryzen 7/5 系列 AM4 插座							
Ryzen 9 5800X	8/16	3.8 GHz	4.7 GHz	32 MiB	105 Watt	Zen 3	Vermeer
Ryzen 5 5600X	6/12	3.7 GHz	4.6 GHz	32 MiB	65 Watt	Zen 3	Vermeer
Ryzen 7 3800X	8/16	3.9 GHz	4.5 GHz	32 MiB	105 Watt	Zen 2	Matisse
Intel Core i9/i7/i5 K 系列 LGA1200 插座							
Core i9 10900K	10/20	3.7 GHz	5.3 GHz	20 MiB	125 Watt	CometLake	CometLake
Core i7 10700K	8/16	3.8 GHz	5.1 GHz	16 MiB	125 Watt	CometLake	CometLake
Core i5 10600K	6/12	4.1 GHz	4.8 GHz	12 MiB	125 Watt	CometLake	CometLake
Core i9 11900K	8/16	3.5 GHz	5.3 GHz	16 MiB	125 Watt	Cypress Cove	RocketLake
Core i7 11700K	8/16	3.6 GHz	5.0 GHz	16 MiB	125 Watt	Cypress Cove	RocketLake
Core i5 11600K	6/12	3.9 GHz	4.9 GHz	12 MiB	125 Watt	Cypress Cove	RocketLake

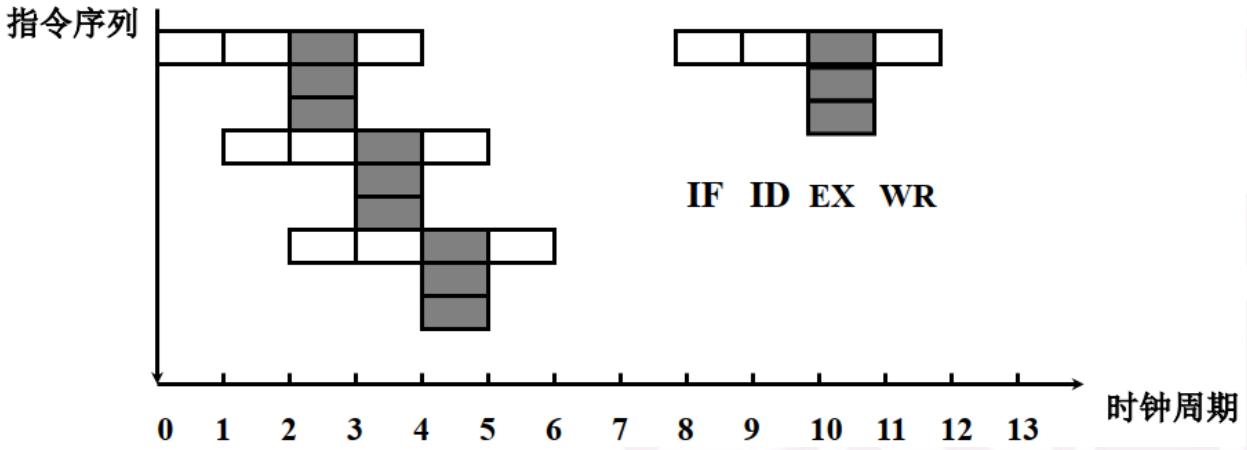
从测试来看，Cypress Cove 的等效流水线深度大约是 14-22 级

奔腾4(Pentium, P4)处理器所使用的核心有三个发展阶段：Willamette、Northwood、Prescott

第一代P4（Willamette）设计了13级流水线；
第二代P4（Northwood）设计了20级流水线，大获成功；
第三代P4（Prescott）把流水线长度增加到了31级，缓存也加大了，相应也把主频拉的很高，结果却并不成功。

R4-10

流水线的优化？——超长指令字/静态多发射处理器



超长指令字（VLIW）（静态多发射处理器）是由美国Yale大学教授Fisher提出的，是一**条指令来实现多个操作的并行执行**，之所以放到一条指令是为了减少内存访问。通常一条指令多达上百位，有若干操作数，每条指令可以做不同的几种运算。那些指令可以并行执行是由编译器来选择的。

- Static multiple issue (静态多发射)
  - Compiler groups instructions into “issue packets” (a single cycle)
  - Packages them into “issue slots”
  - Compiler detects and avoids hazards
  - Determined by pipeline resources required

R4-10 流水线的优化？——动态多发射处理器（超标量处理器）

超级标量是指cpu内一般能有多条流水线，这些流水线能够并行处理。在单流水线结构中，指令虽然能够重叠执行，但仍然是顺序的，每个周期只能发射(issue)或回收(retire)一条指令。超级标量结构的cpu支持指令级并行，每个周期可以发射多条指令(2-4条居多).这样，可以使得cpu的IPC (Instruction Per Clock) > 1， 从而提高cpu处理速度。

超标量处理器的指令执行和发射不一定是顺序的。

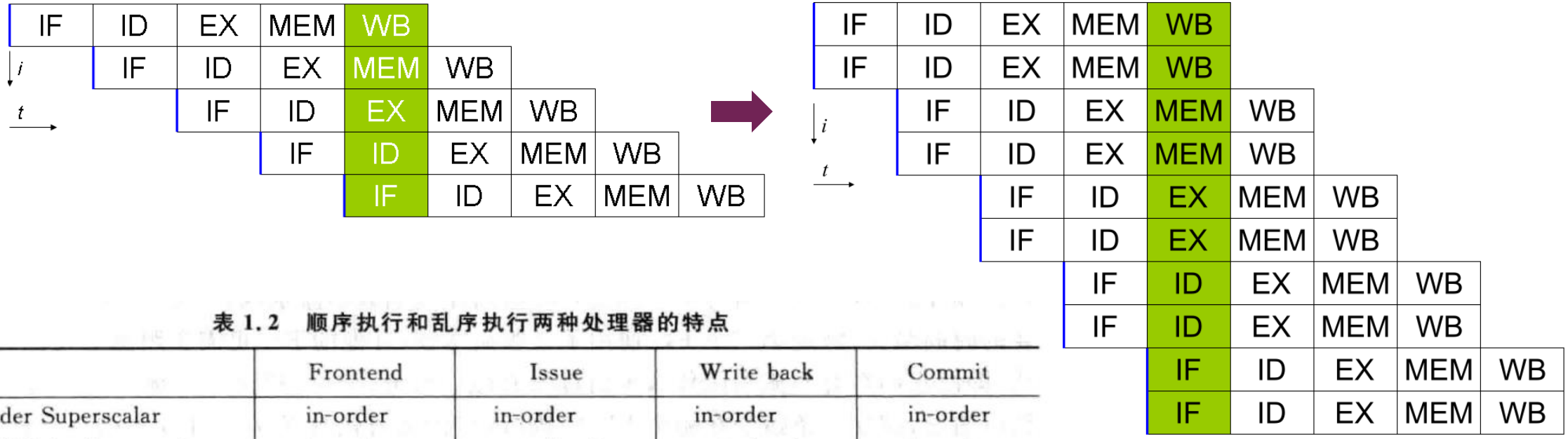


表 1.2 顺序执行和乱序执行两种处理器的特点

	Frontend	Issue	Write back	Commit
In-Order Superscalar	in-order	in-order	in-order	in-order
Out-of-Order Superscalar	in-order	out-of-order	out-of-order	in-order

R4-10

流水线的优化？——动态态多发射处理器（超标量处理器）

- Dynamic multiple issue
  - CPU examines instruction stream and chooses instructions to issue each cycle
  - Compiler can help by reordering instructions
    - **Compiler** can reorder instructions
      - e.g., move load before branch
      - Can include “fix-up” instructions to recover from incorrect guess
  - CPU resolves hazards using advanced techniques at runtime
    - **Hardware** can look ahead for instructions to execute
      - Buffer results until it determines they are actually needed
      - Flush buffers on incorrect speculation

R4-10

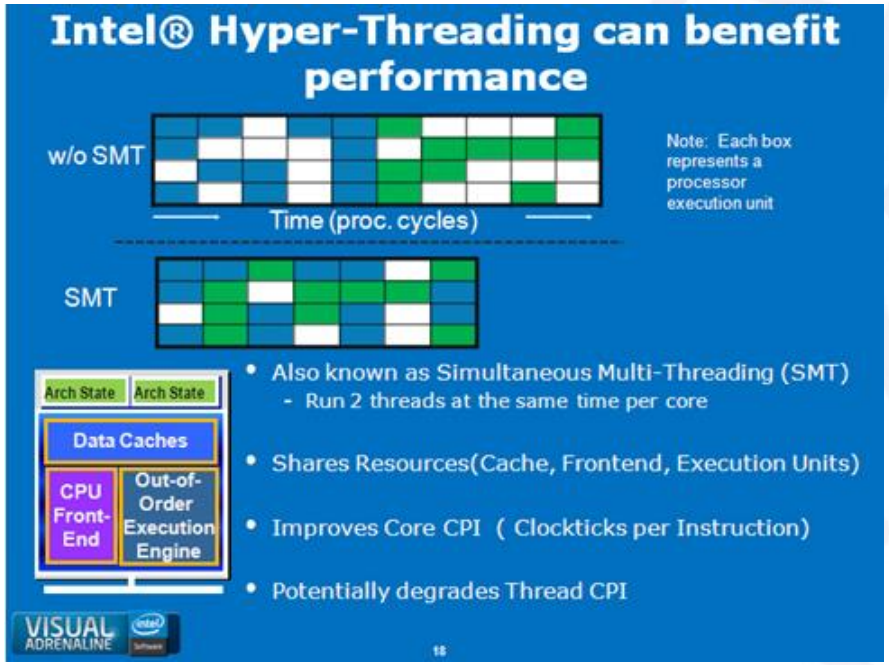
流水线的优化？——数据角度的优化

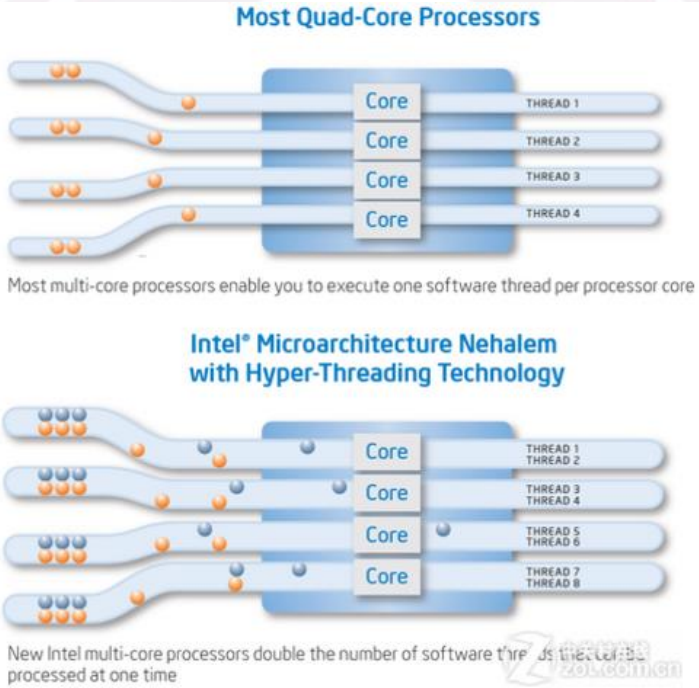
向量计算机以向量作为基本操作单位，操作数和结果都以向量的形式存在。

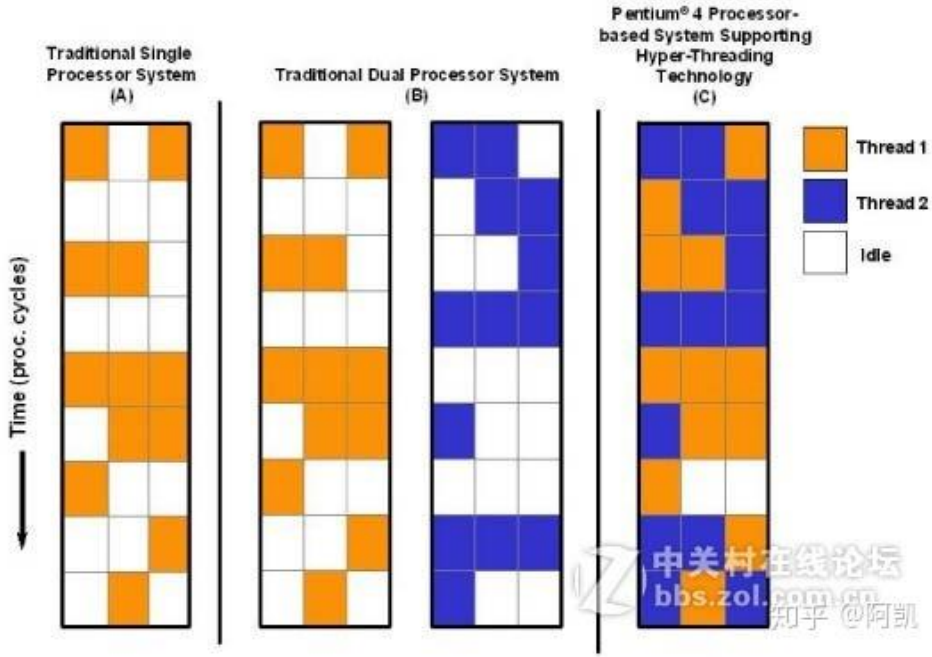
向量机适用于线性规划、傅里叶变换、滤波计算以及矩阵、线性代数、偏微分方程、积分等数学问题的求解，主要解决气象研究与天气预报、航空航天飞行器设计、原子能与核反应研究、地球物理研究、地震分析、大型工程设计，以及社会和经济现象大规模模拟等领域的大型计算问题。

- **向量计算机是一种特定类型的硬件架构**，专门针对向量操作设计；
- **单质量多数据流（SIMD）是一种通用的并行计算模型**，它可以被向量机、现代CPU或GPU采用

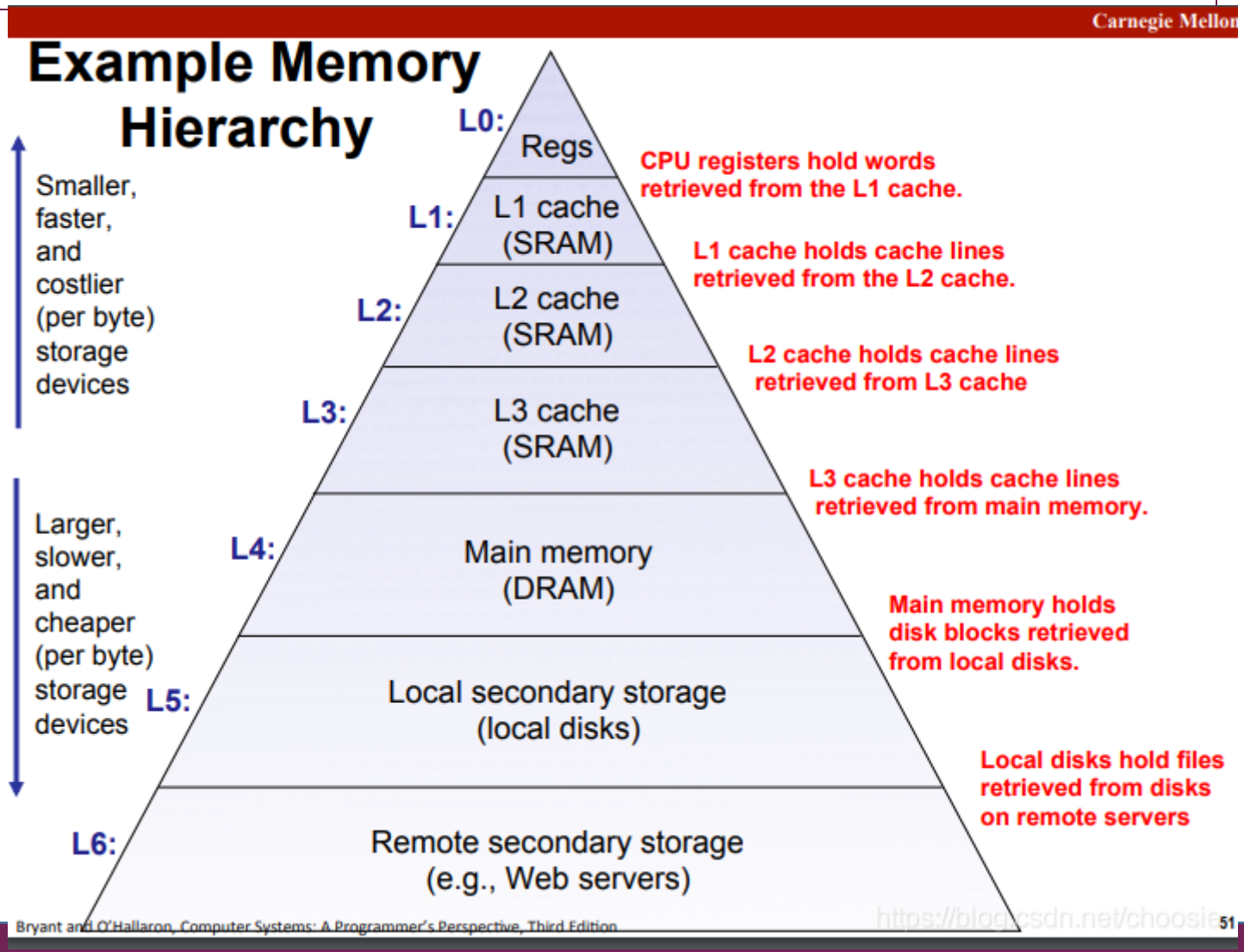
R4-10 流水线的优化？——软件角度的优化（超线程）







R5-01 如何设计容量又大速度又快的计算机存储?



R5-02 不同的存储器之间的区别？——SRAM（Cache）

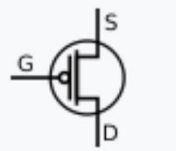
SRAM（静态随机访问存储）

基本单元

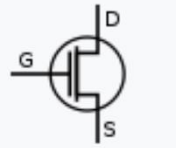
6个晶体管: 4个用于存储, 2个用于控制

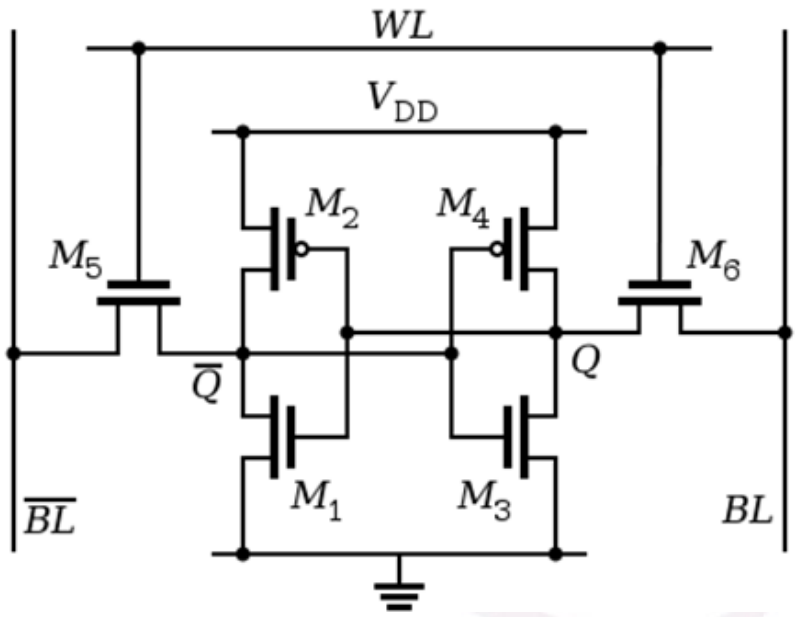
MOS管

N型

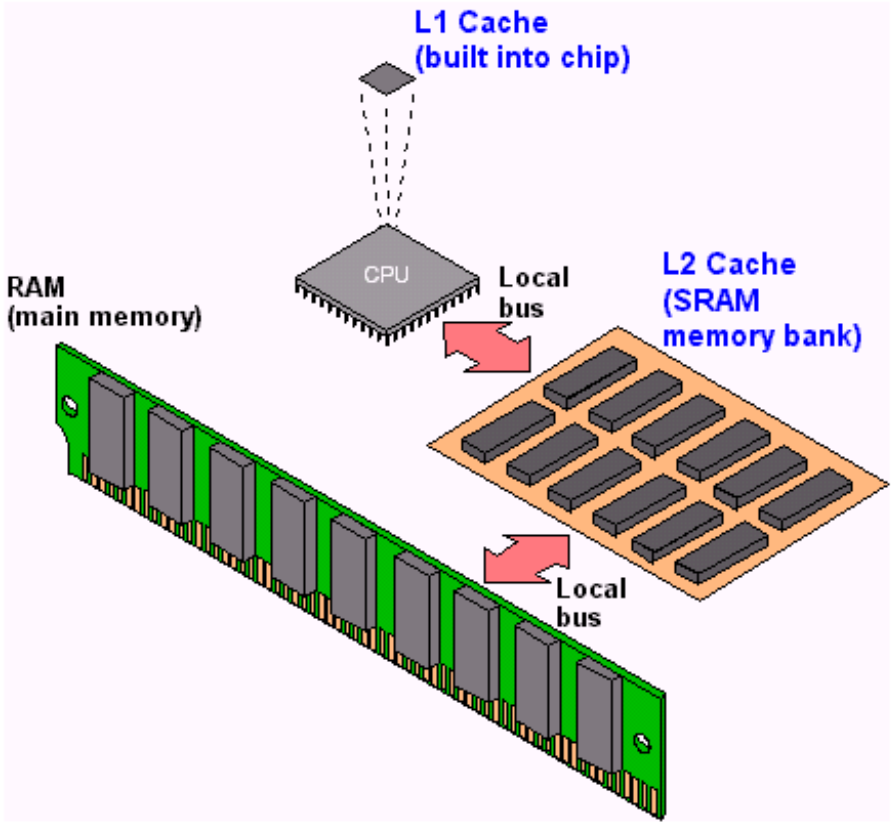


P型





From Computer Desktop Encyclopedia
© 1999 The Computer Language Co. Inc.

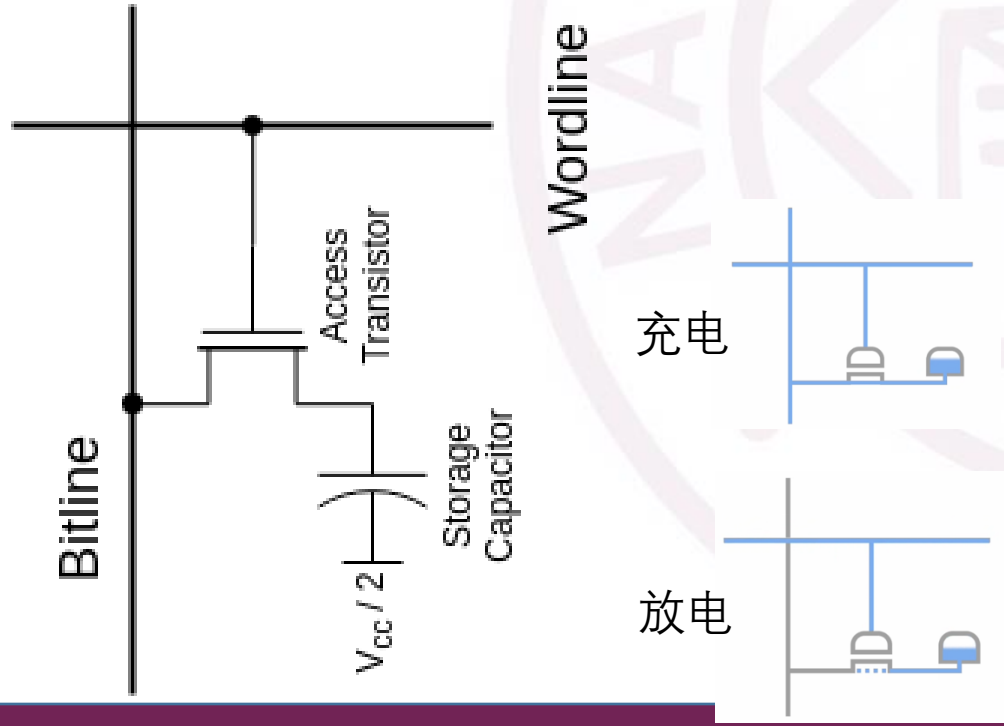


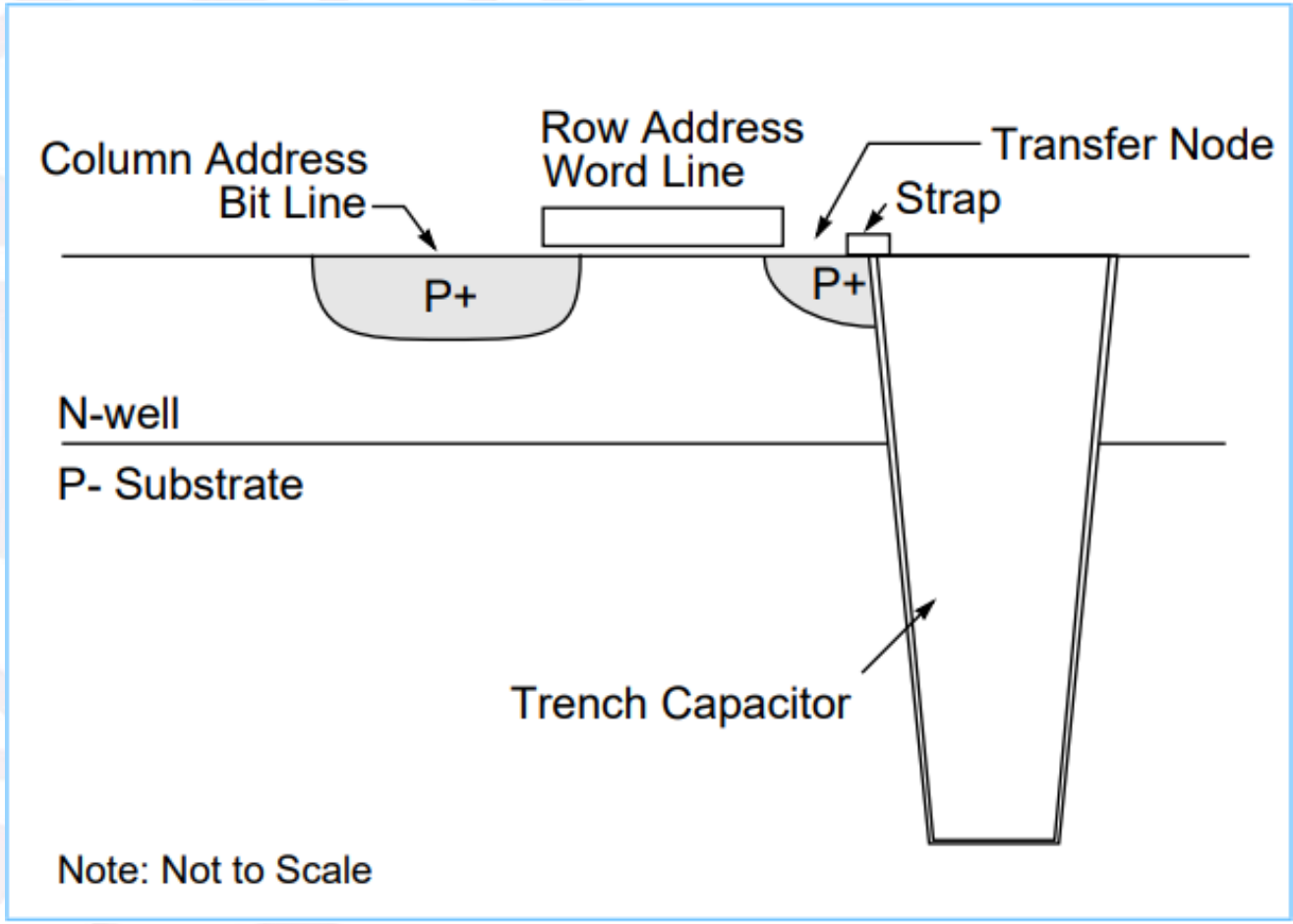
R5-02 不同的存储器之间的区别？—— DRAM（内存/Memory）

DRAM（动态随机访问存储）

晶体管+电容

行、列地址分别传送



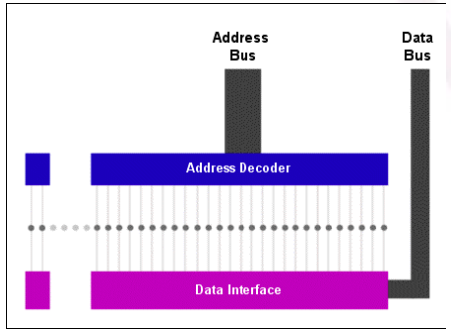


R5-02

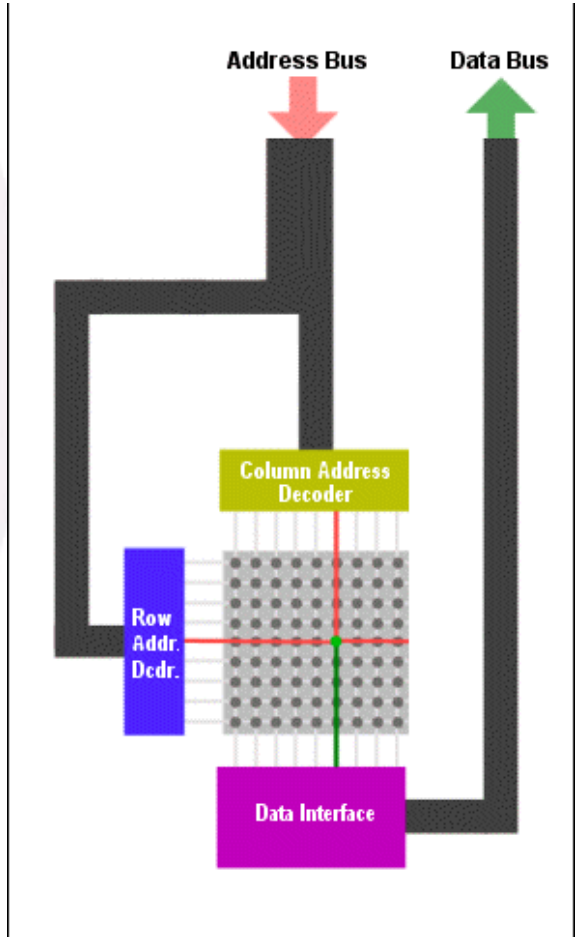
不同的存储器之间的区别?

SRAM

- <10ns
- 单位面积存储单元更少
- 利用反相器实现
- 不需刷新
- 价格更高
- 用于Cache
- 行列地址独立



- DRAM
  - 60ns – 80ns
  - 单位面积存储单元更多
  - 利用电容实现
  - 需要刷新
  - 价格更低
  - 用于主存
  - 行列地址复用



R5-02 不同的存储器之间的区别？——闪存（非易失性存储）

闪速存储器 (Flash ROM)

Flash是20世纪80年代中期出现的一种块擦写型存储器，是一种高密度、非易失性的读/写半导体存储器，突破了传统的存储器体系，改善了存储器产品的特性。由于单片存储容量大，易于修改，Flash ROM也常用于数码相机和U盘中，因其具有低功耗、高密度等特点，且没有机电移动装置，特别适合于便携式设备。

- NOR flash: bit cell like a NOR gate
- NAND flash: bit cell like a NAND gate
- Flash bits wears out after 1000's of accesses
  - Not suitable for direct RAM or disk replacement
  - Wear leveling: remap data to less used blocks

根据架构的不同，NAND闪存又分为：

SLC：单阶存储单元闪存

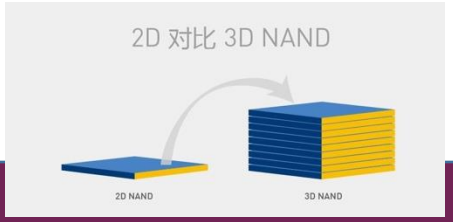
每个存储单元内存储1个信息比特

MLC：多阶存储单元闪存

每个存储单元内存储2个以上的信息比特

TLC：三阶存储单元闪存

每个储存单元内储存3个信息比特



R5-02 不同的存储器之间的区别？——磁盘（非易失性存储）

- 磁表面存储器是利用涂覆在载体表面的磁性材料，两种不同的**磁化状态**来表示二进制信息的"0"和"1"。将磁性材料均匀地涂覆在圆形的铝合金或塑料的载体上就成为**磁盘**，涂覆在聚酯塑料带上就成为**磁带**。

- 记录密度：单位长度内所存储的二进制信息量
  - 磁盘位密度 $D_b = f_t / \pi d_{min}$
  - f_t 每道总位数；d_{min} 同心圆最小直径
- 存储容量
  - 磁盘存储器容量 C = n × k × s
  - n代表存放信息的盘面数
  - k为每个盘面的磁道数
  - s为每条磁道记录的二进制代码数

R5-02 不同的存储器之间的区别？——磁盘（非易失性存储）

- 平均寻址时间
  - 寻址时间 = 寻道时间 + 等待时间(旋转延迟)
  - 平均寻址时间 = 平均寻道时间 + 平均等待时间
  - 磁带存储器的寻址时间是磁带空转到磁头应访问的记录区段所在位置的区间

- 数据传输率
  - 单位时间内存储器向主机传送数据量
  - $D_r = D_b \times V$
  - D_b：记录密度
  - V：记录介质的运动速度

R5-02 不同的存储器之间的区别？——磁带（非易失性存储）

- 磁带存储器属于磁表面存储器。
  - 特点：脱机保存、互换读出、存储容量大(TB)、价格低廉、携带方便、能耗低
  - 介质比磁盘稳定，保存期更长，磁盘保固期最多3-5年，磁带最长可达50年。
  - 尽管磁带机的带速和结构不同，只要是按标准格式记录的磁带，都可在各种磁带机上读出，即具有磁带互换性。

R5-02 不同的存储器之间的区别？——光盘（非易失性存储）

光盘是以光信息做为存储的载体并用来存储数据的一种物品。它利用激光原理进行读、写，是迅速发展的一种辅助存储器，可以存放各种文字、声音、图形、图像和动画等多媒体数字信息。

实验室阶段单碟存储700TB

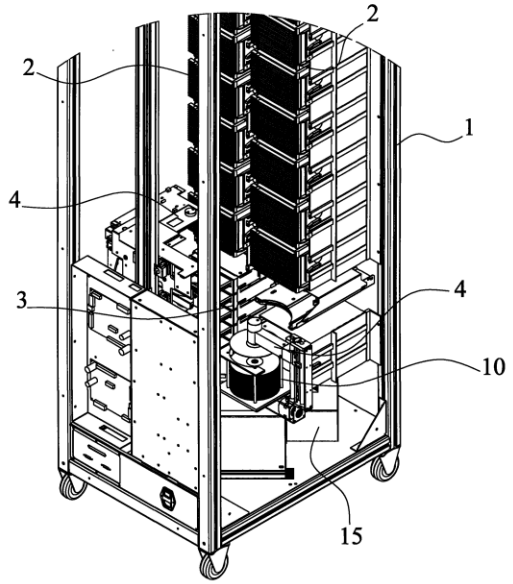
CD 650MB-700MB

=74分钟CD质量的音乐或VHS质量的视频

DVD 单面数据4.3G。

蓝光光碟 单层容量为25或是27GB

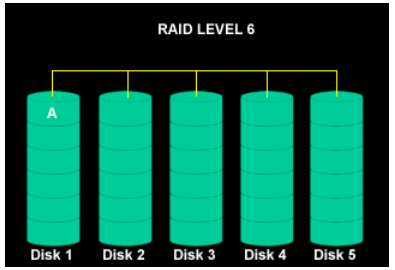
光盘库



R5-03 如何组合多个存储器提供稳定的存储服务？——磁盘阵列

RAID (Redundant Array of Independent Disks) 独立磁盘冗余阵列，简称磁盘阵列

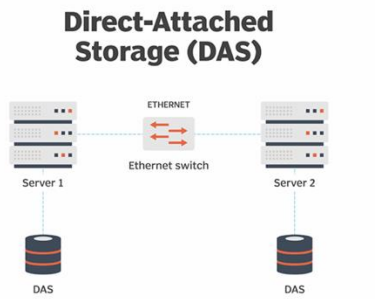
- 大容量
  - 多个磁盘组成的RAID系统具有海量的存储空间。
- 高性能
  - 通过数据条带化，RAID将数据I/O分散到各个成员磁盘上，从而获得比单个磁盘成倍增长的聚合I/O性能。
- 可靠性
  - 冗余技术大幅提升数据可用性和可靠性
- 可管理性
  - RAID内部完成了大量的存储管理工作，管理员只需要管理单个虚拟驱动器，

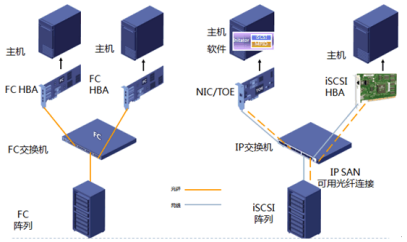


R5-03

如何组合多个存储器提供稳定的存储服务？——服务模式

特性	DAS（直连式存储）	NAS（网络附属存储）	SAN（存储区域网络）
中文名称	直连式存储	网络附属存储	存储区域网络
是否通过网络	❌ 否，直接连接（如SATA/SAS）	✅ 是，走局域网（Ethernet）	✅ 是，走专用网络（光纤/iSCSI）
存储访问粒度	块级	文件级	块级
使用方式	本地磁盘一样	文件共享方式	远程磁盘映射，像本地盘一样
常见协议	SATA / SAS / NVMe	NFS / SMB / CIFS	FC / iSCSI / FCoE
多台主机共享	❌ 通常不支持共享	✅ 支持多人共享文件	✅ 支持多主机访问同一存储卷
性能	高（本地总线）	受网络影响	高（专用高速通道）
成本	最低（只买硬盘）	中等（有管理功能）	高（需配光纤通道、交换机等）
管理复杂度	最低	较低	最高
适用场景	单台服务器本地存储	企业共享文件服务器	数据中心、数据库、高性能集群





R5-04 如何防止存储错误？——纠错/检错（汉明编码）

- Hamming distance
  - Number of bits that are different between two bit patterns
  - Minimum distance = 2 provides single bit error detection
    - E.g. parity code
  - Minimum distance = 3 provides single error correction, 2 bit error detection
- To calculate Hamming code:
  - Number bits from 1 on the left
  - All bit positions that are a power 2 are parity bits
  - Each parity bit checks certain data bits

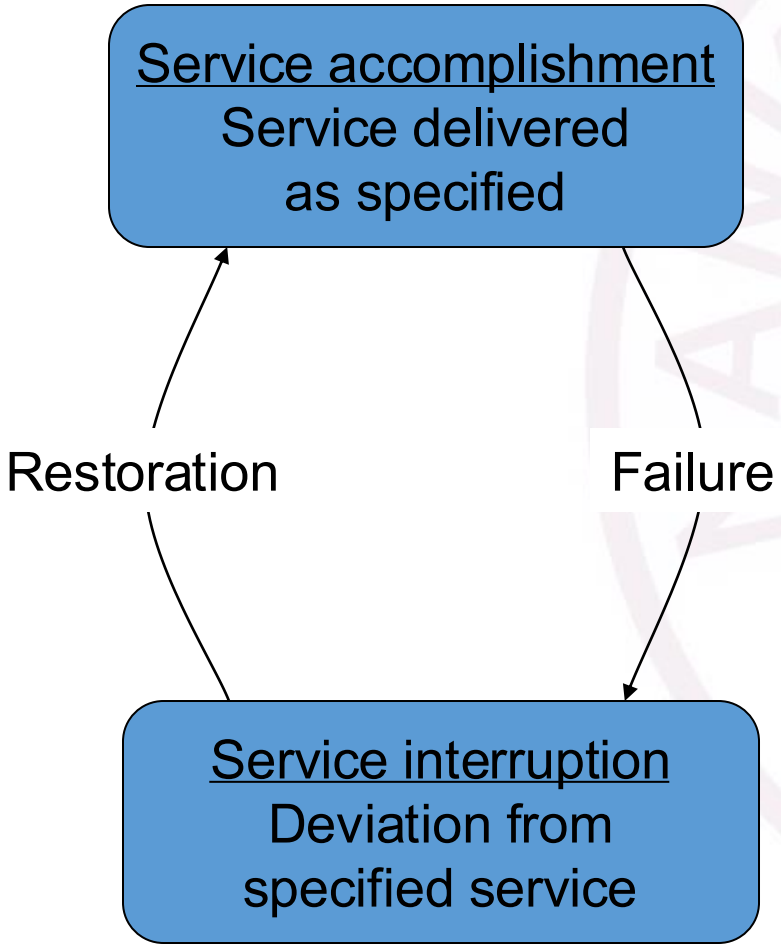
最小汉明距离 $d_{\min}$：

最多错误检测能力：$D = d_{\min} - 1$

错误纠正能力：$C = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor$

R5-04

存储器的可靠性和可用性?



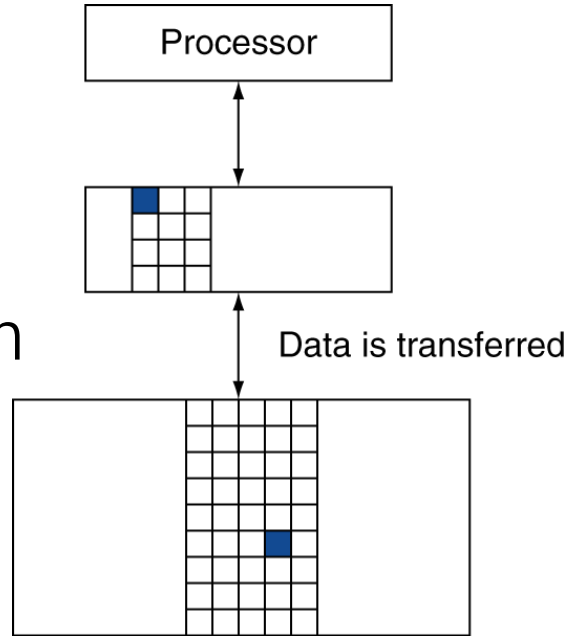
- Fault: failure of a component
  - May or may not lead to system failure
- Reliability: mean time to failure (MTTF)
- Service interruption: mean time to repair (MTTR)
- Mean time between failures
  - MTBF = MTTF + MTTR
- Availability = MTTF / (MTTF + MTTR)
- Improving Availability
  - Increase MTTF: fault avoidance, fault tolerance, fault forecasting
  - Reduce MTTR: improved tools and processes for diagnosis and repair

R5-05 存储层次结构的设计依据?

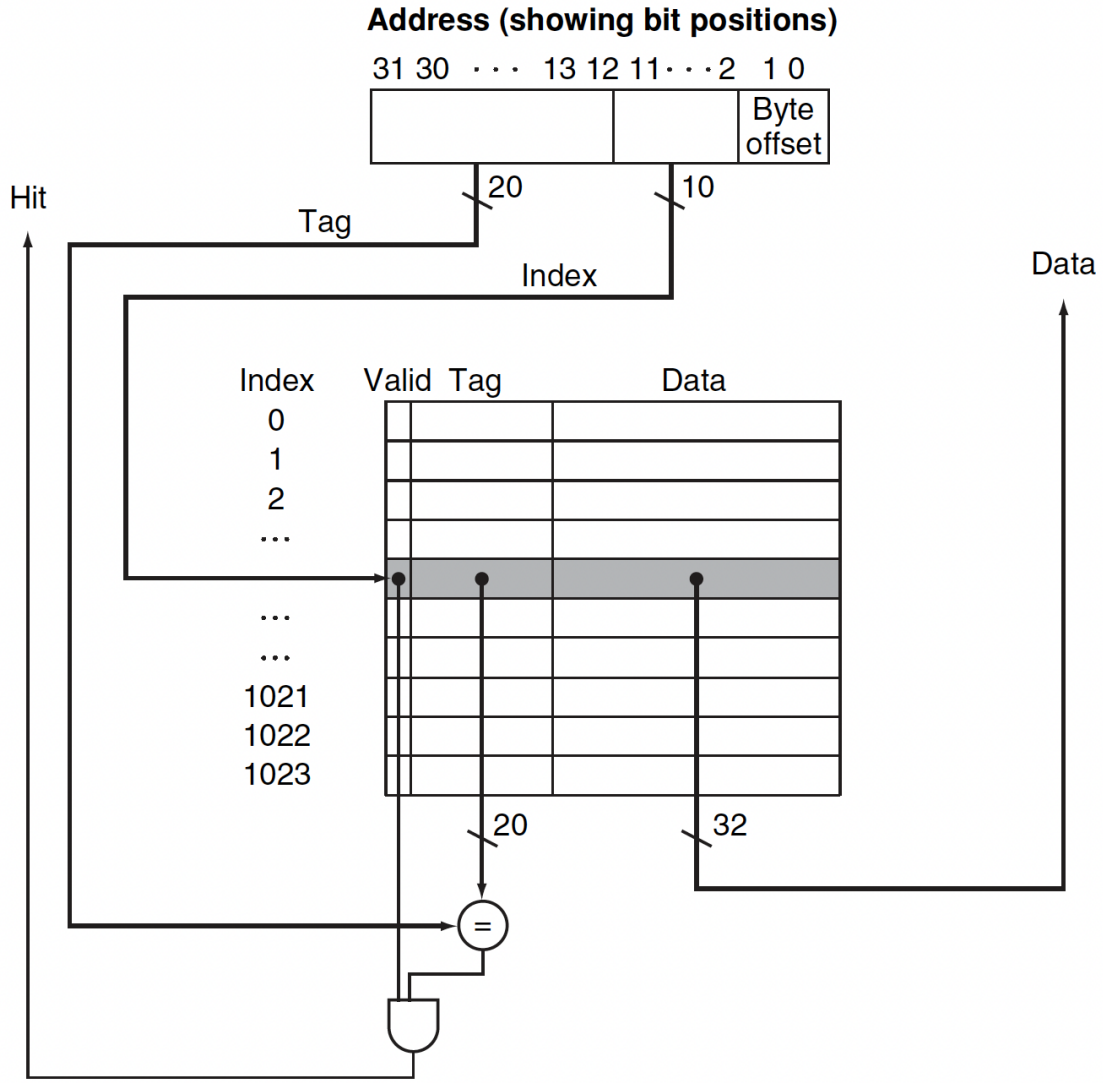
- Programs access a small proportion of their address space at any time
- Temporal locality
  - Items accessed recently are likely to be accessed again soon
  - e.g., instructions in a loop, induction variables
- Spatial locality
  - Items near those accessed recently are likely to be accessed soon
  - E.g., sequential instruction access, array data

R5-06 存储层次结构是如何工作的?

- Memory hierarchy
  - Store everything on disk
  - Copy recently accessed (and nearby) items from disk to smaller DRAM memory
    - Main memory
  - Copy more recently accessed (and nearby) items from DRAM to smaller SRAM memory (Cache)
    - Cache memory attached to CPU



R5-07 存储层次结构中的Cache的原理？——cache读访存



- Block (aka line): unit of copying
  - May be **multiple words**
- If accessed data is present in upper level
  - Hit: access satisfied by upper level
    - Hit ratio: hits/accesses
- If accessed data is absent
  - Miss: block copied from lower level
    - Time taken: miss penalty
    - Miss ratio: misses/accesses
      = 1 – hit ratio
  - Then accessed data supplied from upper level

R5-08 存储层次结构中的Cache的原理？——cache写访存（命中）

- On data-write hit, could just update the block in cache
  - But then cache and memory would be inconsistent
- Write through: also update memory
- But makes writes take longer
  - e.g., if base CPI = 1, 10% of instructions are stores, write to memory takes 100 cycles
    - Effective CPI = 1 + 0.1×100 = 11
- Solution: write buffer
  - Holds data waiting to be written to memory
  - CPU continues immediately
    - Only stalls on write if write buffer is already full

写直达

R5-08 存储层次结构中的Cache的原理？——cache写访存（命中）

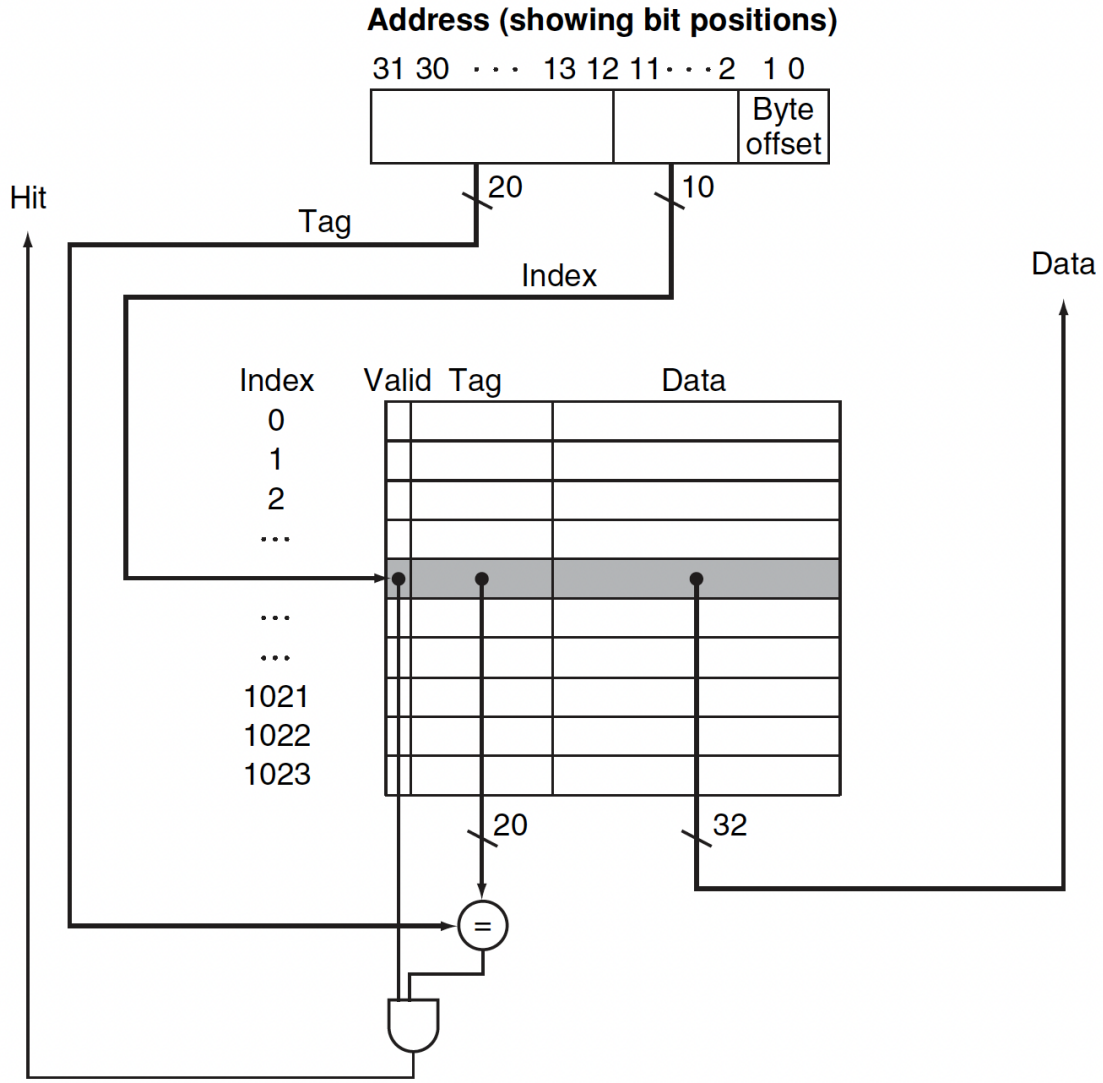
- On data-write hit, just update the block in cache
  - Keep track of whether each block is dirty
  - When a dirty block is replaced
    - Write it back to memory
    - Can use a write buffer to allow replacing block to be read first

写回法

R5-08 存储层次结构中的Cache的原理？——cache写访存（不命中）

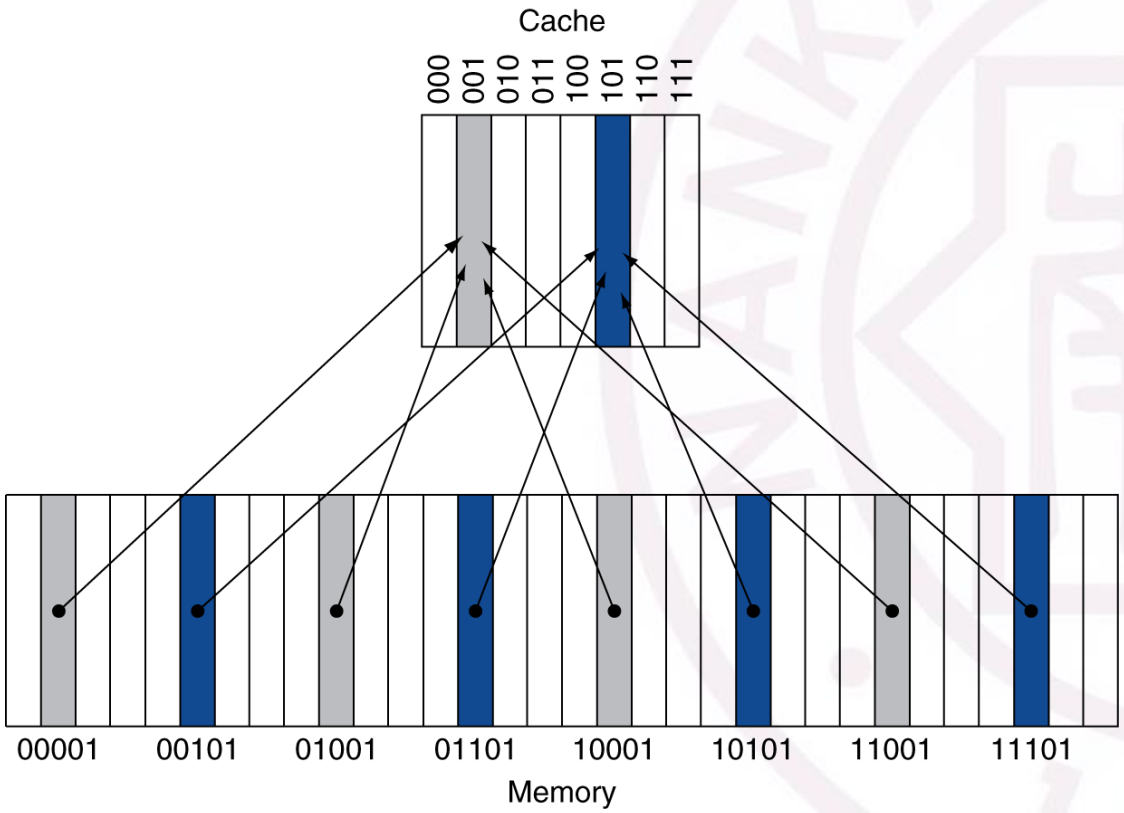
- What should happen on a write miss?
- Alternatives for write-through
  - Allocate on miss: fetch the block——写分配
  - Write around: don't fetch the block——写不分配
    - Since programs often write a whole block before reading it (e.g., initialization)
- For write-back
  - Usually fetch the block

R5-09 存储层次结构中的Cache的原理？——地址映射方式



- Location determined by address
- Direct mapped: only one choice
  - (Block address) modulo (#Blocks in cache)
- How do we know which particular block is stored in a cache location?
  - Store block address as well as the data
  - Actually, only need the high-order bits Called the tag
- What if there is no data in a location?
  - Valid bit: 1 = present, 0 = not present

R5-09 存储层次结构中的Cache的原理？——地址映射方式



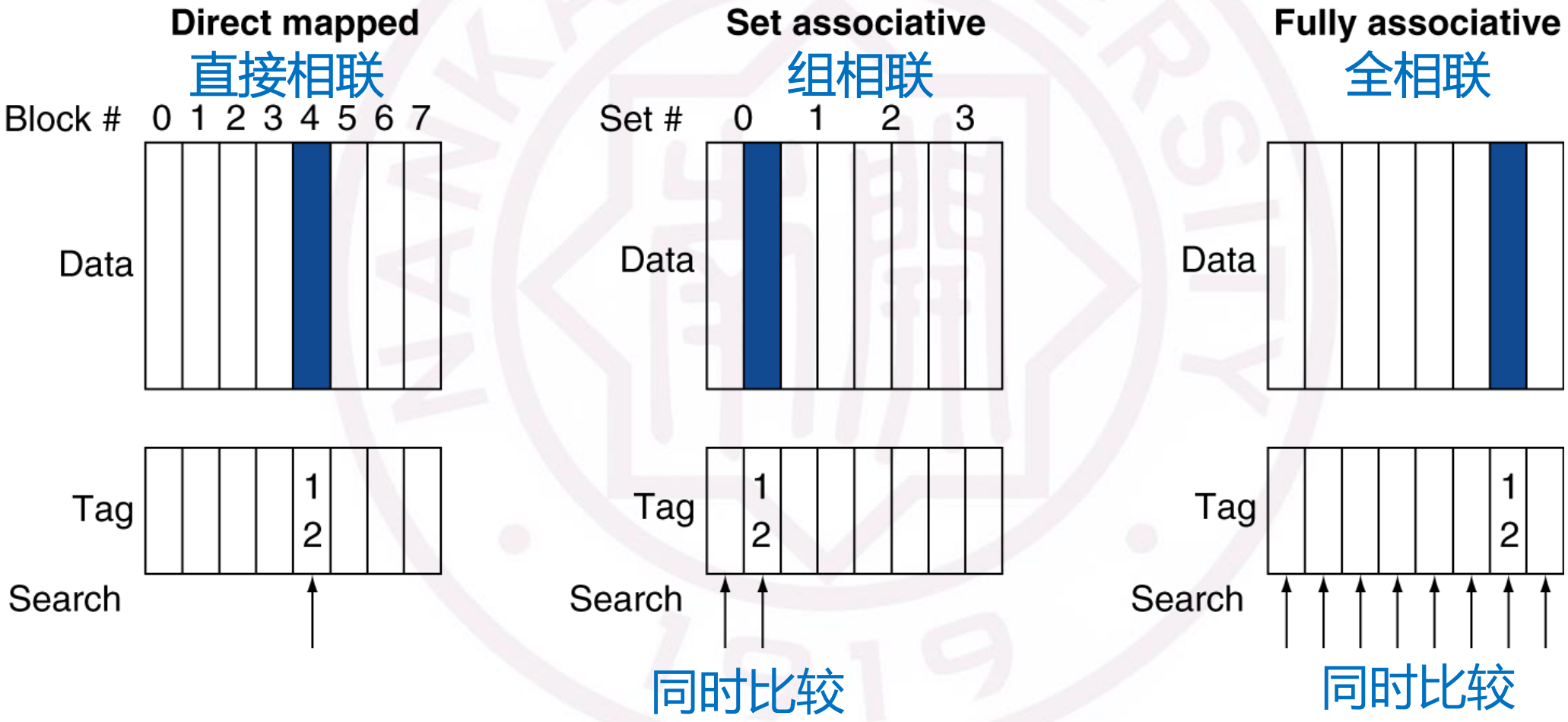
直接映射

- Larger blocks should reduce miss rate
  - Due to spatial locality
- But in a **fixed-sized cache**
  - Larger blocks ⇒ fewer of them
    - More competition ⇒ increased miss rate
  - Larger blocks ⇒ pollution
- Larger miss penalty (data transmission)
  - Can override benefit of reduced miss rate
  - Early restart and critical-word-first can help

存储器延迟越短，cache块可以越小

存储器带宽越大，cache块可以越大

R5-09 存储层次结构中的Cache的原理？——地址映射方式



R5-10 存储层次结构中的Cache的原理？——替换哪个块的方法

- Direct mapped: no choice
- Set associative
  - Prefer non-valid entry, if there is one
  - Otherwise, choose among entries in the set
- Least-recently used (LRU)
  - Choose the one unused for the longest time
    - Simple for 2-way, manageable for 4-way, too hard beyond that
- Random
  - Gives approximately the same performance as LRU for high associativity

R5-11

存储层次结构中的多级Cache?

- Primary cache attached to CPU
  - Small, but fast
- Level-2 cache services misses from primary cache
  - Larger, slower, but still faster than main memory
- Main memory services L-2 cache misses
- Some high-end systems include L-3 cache（LLC）全相联

R6-01

高维矩阵乘法优化基本思路

- **多线程并行化加速**

利用多核CPU计算资源、可采用OpenMP（Open Multi-Processing）优化

- **子块并行优化**

通过局部计算降低内存访问延迟；为OpenMP或其他并行机制提供更细粒度、更均匀的工作划分，适用于大规模矩阵，特别适合在多核CPU上运行

- **多进程并行优化**

使用MPI（Message Passing Interface）实现矩阵乘法的多进程优化，其核心思想是将大矩阵按行或块划分给不同进程，利用进程间通信协同完成计算

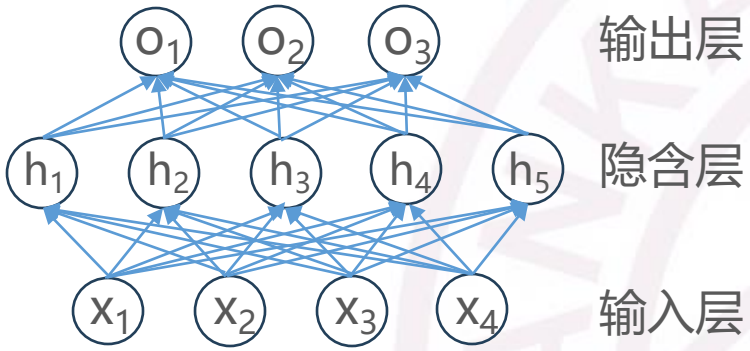
- **DCU加速计算**

通过曙光DCU（Dawn Computing Unit）硬件，加速AI训练、推理和高性能计算场景。DCU与NVIDIA GPU特性类似，支持大规模并行计算，但通常通过HIP C++编程接口进行开发

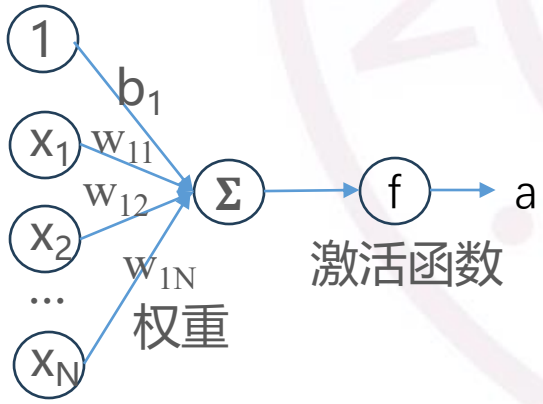
注：还有更多其他方式

R6-02

基于矩阵乘法的多层感知机(MLP)实现和性能优化



MLP网络结构



神经元结构

第一层前向传播

$$H_1 = \mathrm{ReLU}(X @ W_1 + B_1)$$

X: [1024 × 10]

W_1: [10 × 20]

B_1: [1 × 20]

H_1: [1024 × 20]

第二层前向传播

$$Y = H_1 @ W_2 + B_2$$

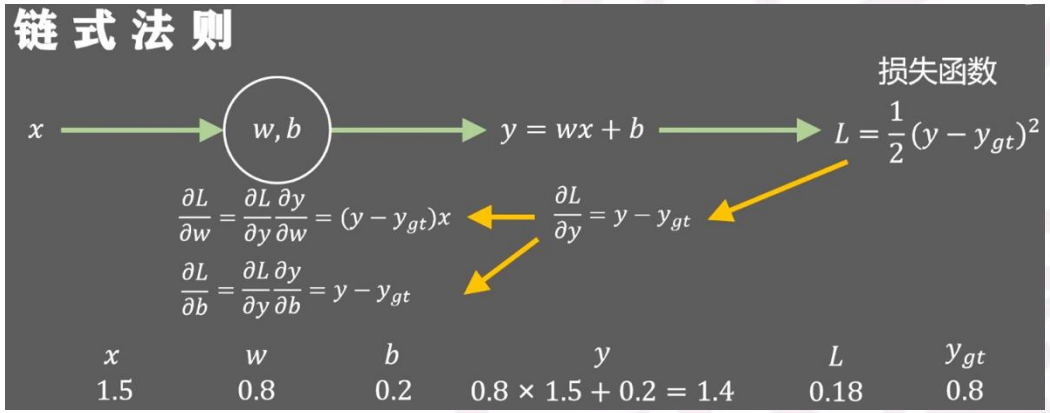
W_2 = [20 × 5]

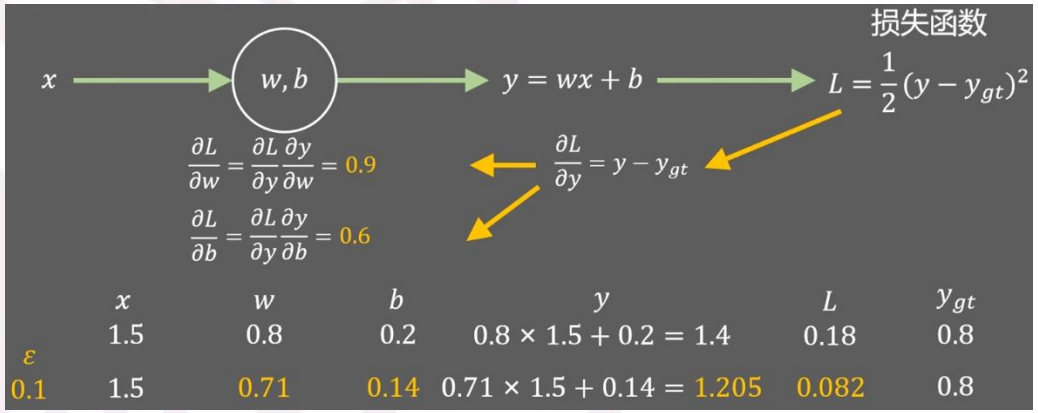
B_2 = [1 × 5]

Y = [1024 × 5]

R6-03

基于矩阵乘法的多层感知机(MLP)实现和性能优化





MLP反向传播更新*w, b* 参数

本次校内赛将**围绕基于矩阵乘法的MLP神经网络计算优化**展开，通过设计一系列挑战任务，培训并引导参赛者从算法理解、性能建模、系统优化到异构调度完成一个完整的系统创新设计

计算机设计的八大思想/原则同样贯穿指导了整个校内赛！

R6-04

计算机设计的八个伟大思想/原则在生产生活中的体现

R6-05

计算机性能如何提升

- Compiled code sequences using instructions in classes A, B, C (IC=Instruction Count)

Class	A	B	C
CPI for class	1	2	3
IC in **sequence 1**	2	1	2
IC in **sequence 2**	4	1	1

- Sequence 1: IC = 5
  - Clock Cycles
    = 2×1 + 1×2 + 2×3
    = 10
  - Avg. CPI = 10/5 = 2.0
- Sequence 2: IC = 6
  - Clock Cycles
    = 4×1 + 1×2 + 1×3
    = 9
  - Avg. CPI = 9/6 = 1.5

R6-05

计算机性能如何提升

Computer A: Cycle Time = 250ps, CPI = 2.0
Computer B: Cycle Time = 500ps, CPI = 1.2
Same ISA
Which is faster, and by how much?

$$\text{CPU Time}_A = \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A$$
$$= I \times 2.0 \times 250\text{ps} = I \times 500\text{ps}$$

← A is faster…

$$\text{CPU Time}_B = \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle Time}_B$$
$$= I \times 1.2 \times 500\text{ps} = I \times 600\text{ps}$$

$$\frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{I \times 600\text{ps}}{I \times 500\text{ps}} = 1.2$$

← …by this much

R6-05

计算机性能如何提升

假设某程序共执行了70%的算术指令、10%的取数/存数指令和30%的分支指令。

1）假设执行一条算术指令、取数/存数指令和分支指令分别需要3个周期、6个周期和2个周期，求平均CPI

2）在取数/存数指令和分支指令执行时间不变的情况下，如果要使性能提升25%，则算术运算指令的平均执行时间应该为多少？

3）在取数/存数指令执行时间不变的情况下，如果要使性能提升55%以上，则算术运算指令和分支指令的平均执行时间可能为多少（给出时间关系的相应范围即可）？

R6-06

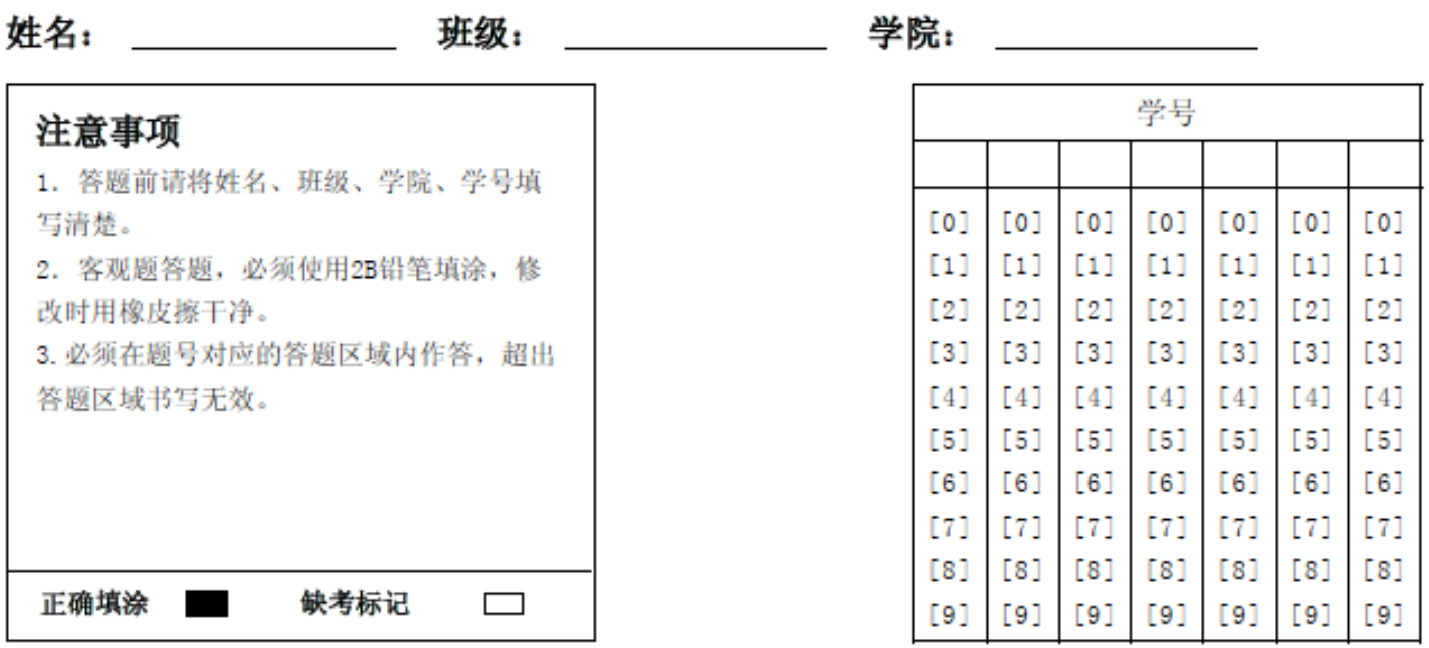
存储的高级语言如何翻译为指令?

```
i = 1;
while (i < 10) {
    for (j = 0; j < b; j++)
        A[4*j] = i + j;
    i++;
}
```

变量	寄存器	说明
i	$t0	外层循环计数器
j	$t1	内层循环计数器
b	$s0	一个已知整数
A	$s1	数组 A 的基址

```
        addi $t0, $zero, 0      # i = 0
        beq $zero $zero TEST1   # 跳转到while循环的判断
LOOP1:  addi $t1, $zero, 0      # 临时 j = 0
        beq $zero $zero TEST2       # 跳转到for循环的判断
LOOP2:  add  $t2, $t0, $t1      # tmp = i + j
        sll  $t3, $t1, 2        # offset = 4 * j
        add  $t3, $s1, $t3      # addr = A + 4*j
        sw   $t2, 0($t3)        # A[4*j] = i + j
        addi $t1, $t1, 1        # j++
TEST2:   slt  $t2, $t1, $s0      # if j < b
        beq  $t2, $zero, LOOP2.  # 如果 j >= b，跳出内层
TEST1:  addi $t0, $t0, 1             # i++
        slti  $t3, $t0, 10           # i < 10 ?
        bne  $t3, $zero, LOOP1  # 是则回到外层
```

- 电子阅卷：试卷+答题卡 （**2B铅笔** 涂写学号和选项，**黑色碳素**笔答题）



姓名：_____ 班级：_____ 学院：_____

注意事项

1. 答题前请将姓名、班级、学院、学号填写清楚。
2. 客观题答题，必须使用2B铅笔填涂，修改时用橡皮擦干净。
3. 必须在题号对应的答题区域内作答，超出答题区域书写无效。

正确填涂 ■ 缺考标记 □

学号						
[0]	[0]	[0]	[0]	[0]	[0]	[0]
[1]	[1]	[1]	[1]	[1]	[1]	[1]
[2]	[2]	[2]	[2]	[2]	[2]	[2]
[3]	[3]	[3]	[3]	[3]	[3]	[3]
[4]	[4]	[4]	[4]	[4]	[4]	[4]
[5]	[5]	[5]	[5]	[5]	[5]	[5]
[6]	[6]	[6]	[6]	[6]	[6]	[6]
[7]	[7]	[7]	[7]	[7]	[7]	[7]
[8]	[8]	[8]	[8]	[8]	[8]	[8]
[9]	[9]	[9]	[9]	[9]	[9]	[9]

- 考题：20道选择（40分）+ 4道应用题（60分），满分100

遇事不决，可问春风；
春风不语，既随本心

预祝取得好成绩